UNIVERSITY
INSTITUTE
OF LISBON

A Cross-Layer SDN Orchestration Architecture for Stateful Edge Services

Pedro Ojo Oriakhi

Master's in Telecommunications and Computer Engineering

Supervisors:

Ph.D. Professor José Moura, Assistant Professor,
Iscte - Instituto Universitário de Lisboa

Ph.D. Professor Pedro Ramos, Assistant Professor,
Iscte - Instituto Universitário de Lisboa

Month, Year

TECHNOLOGY
AND ARCHITECTURE

Department of Information Science and Technology

A Cross-Layer SDN Orchestration Architecture for Stateful Edge Services

Pedro Ojo Oriakhi

Master's in Telecommunications and Computer Engineering

Supervisors:

Ph.D. Professor José Moura, Assistant Professor,
Iscte - Instituto Universitário de Lisboa

Ph.D. Professor Pedro Ramos, Assistant Professor,
Iscte - Instituto Universitário de Lisboa

Month, Year

Write here your dedication.

Acknowledgments

Write here your acknowledgements and, if applicable, any grants or sources of funding.

Resumo

Palavras-Chave:

Abstract

Write here the abstract in English. There is a 250 word limit.

Keywords: *Keywords; separated; by; semicolons*

Contents

Acknowledgments iii

Resumo v

Abstract vii

List of Figures xi

List of Tables xv

List of Acronyms xvii

Chapter 1. Introduction 1
1.1. Context, Motivation, and Problem Statement 1
1.2. Objectives 4
1.3. Research Questions 4
1.4. Research Methodology 5
1.5. Main Contributions of this Dissertation 6
1.6. Dissertation Structure 7

Chapter 2. Background and Related Work 9
2.1. Review Methodology 9
2.2. Platform Technology Background 9
2.2.1. Edge Computing 10
2.2.2. SDN and OpenFlow 11
2.2.3. Containerisation and Docker 12
2.2.4. NoSQL and MongoDB 12
2.3. Telemetry 13
2.4. Auto Scaling 14
2.5. Load Balancing and Readiness Admission on SDN 16
2.6. SDN-Based Orchestration Platforms 18
2.7. Synthesis and Research Gaps 20

Chapter 3. Architecture and Design 23
3.1. Design Requirements 23
3.2. Architecture 24
3.2.1. Two-Domain Organisation 25
3.2.2. Virtual IP Data Paths 26

3.2.3. Control and Telemetry Paths 28
3.3. Distributed Elastic Allocation of Resources 28
3.3.1. Scaling Policy 29
3.3.2. Compute 31
3.3.3. Storage 32
3.4. Telemetry, Load Balancing, and Scaling 32
3.5. Control Workflow 34

Chapter 4. Implementation 37
4.1. Experimental Infrastructure 37
4.2. Distributed Elastic Allocation of Resources 39
4.2.1. Compute 39
4.2.2. Storage 40
4.3. Telemetry, Load Balancing, and Scaling 41
4.4. Control Workflow 42
4.5. Implementation Validation 43

Chapter 5. Experimental Results 45
5.1. Experimental Setup 45
5.1.1. Testbed and Instrumentation 45
5.1.2. Demand Model and Campaign Workloads 46
5.1.3. Metrics and Outcome Definitions 47
5.1.4. Campaign Designs and Analysis 49
5.2. Telemetry Modes (RQ1) 50
5.3. Bottleneck-Aware Scaling Action (RQ2) 54
5.4. Readiness Admission Modes under Three Message Conditions (RQ3) 59
5.5. Discussion: Synthesis Across Interfaces 67

Chapter 6. Conclusions and Future Work 71
6.1. Conclusions 71
6.2. Research Contributions 71
6.3. Limitations 71
6.4. Future Research Directions 71

References 73

Appendix A. Campaign Configuration Values 77

List of Figures

Figure 1.1 Global internet traffic between 2020 and 2025. Source: Statista, based on ITU data [3]. 1

Figure 1.2 The demand-to-capacity chain: from a demand shift to usable capacity, with the three interfaces studied in this dissertation highlighted. 3

Figure 1.3 Design Science Research Methodology 5

Figure 2.1 Software-Defined Networking architecture: the control plane is decoupled from the data plane and programs the forwarding devices through the OpenFlow protocol. 12

Figure 2.2 MongoDB replica set: a primary member serving the clients and secondary members to which the data is replicated. 13

Figure 2.3 Load balancing in an SDN network: the controller installs forwarding rules that distribute client traffic among the available backends. 16

Figure 3.1 Layered organisation of the platform in one network domain: the controller of the control layer consumes the telemetry summaries and issues the forwarding rules to the switch and the capacity changes to the resources. The data plane layer carries the client requests and the storage traffic, and the resource layer holds the edge servers, the storage members, and the telemetry aggregator. 25

Figure 3.2 Double-VIP routing sequence: a client request addressed to `VIP_SERVER` triggers load balancing in the controller, and the edge server's storage reads through `VIP_DATA` trigger load balancing across replica-set members. 27

Figure 3.3 Control and telemetry paths: edge servers and storage members send observations to the per-domain telemetry aggregator, whose summaries reach both peer controllers through a configurable telemetry interface. 28

Figure 3.4 Elastic allocation of resources: each tier is scaled on its own observation signals, with compute replicas provisioned and drained and storage members added and removed through replica-set membership, both actions issued by the controller over the infrastructure-management path. 29

Figure 3.5 The per-summary evaluation, in the fixed order of Section 3.3.1: the eligibility check, then the degradation score, then the scale-up condition,

and otherwise the idle condition, with every remaining branch returning to the next summary. 34

Figure 3.6 The control workflow, drawn as a UML diagram. The demand-to-capacity chain runs across three contexts in the same process: the packet context, the observation context, and the action context. The contexts do not block one another and they exchange state through shared structures (the telemetry state, the action queue, the capacity state, and the routing pools). The numbered stages match the chain in Figure 1.2 and show the provisioning direction. Capacity removal follows the same steps in reverse, starting with the drain. The aggregator is omitted between the observations sent by the replicas and the summary delivered to the controller. 35

Figure 5.1 Episode endpoint latency p95 by telemetry mode (served basis; per-run mean of the two domains; seven replicates per mode). Bars show the median across replicates, whiskers the min–max range, and dots the individual runs; the two highest delayed push points are the replicates where the spare capacity of the nodes already running was consumed. 53

Figure 5.2 Targeted-tier relief where the storage tier bound the episode: post-to-pre peak storage-CPU ratio per replicate (mean of the two domains) for the data-access-bound arms, against the pre-registered 0.75 threshold; bars show the mean across replicates with the standard deviation, and dots the individual replicates. The storage-only arm stays below the threshold in every replicate, the bottleneck-aware arm partially overlaps it, and the compute-only arm activated no reserve. 56

Figure 5.3 Episode endpoint latency of the data-access-bound episode at the p95 (left, logarithmic axis) and the median (right) for the compute-only, storage-only and bottleneck-aware arms and the no-action comparator. The matched storage action takes the tail and the mismatched compute action keeps the lowest median while leaving a higher tail. Bars show per-arm medians across replicates and dots the individual runs; the no-action comparator is drawn from the extension campaign and carries its cross-era label. 57

Figure 5.4 Episode endpoint latency of the compute-bound episode at the median (left) and the p95 (right, logarithmic axis) for the compute-only, storage-only and bottleneck-aware arms and the no-action comparator, all six replicates per arm on the same build. The median is the null reading: every arm serves at about 3 ms. At the p95 the onset latency is flat (about 3 ms) and a late-episode tail appears only in the arms that scale up the compute tier (medians of about 4.6 s in the compute-only arm and 1.6 s in the bottleneck-aware arm), absent in the storage-only arm and the no-action

comparator (about 7 ms). Bars show per-arm medians across replicates and dots the individual runs. 57

Figure 5.5 Execution of the compute action on the compute-bound episode, covering the compute-only and bottleneck-aware arms with the storage-only arm as the no-scaling reference. Left: the share of each domain's requests served by the added backends after the first addition (dots: individual runs and domains), over the grey share served by the original infrastructure. Right: per-backend CPU as a percentage of the backend CPU cap, median over each member's sampled windows (the original infrastructure over the episode, the added backends over their active life), against the storage-only arm's single-backend CPU on the same episode (dashed); each member alternates between serving windows at about that level and near-idle windows. The action executed in full, yet the service outcome (Figure 5.4, left) is unchanged. 58

Figure 5.6 Admission-chain intervals under an intact message, all four admission modes. (a) Provisioning to first successful response per backend over the seven seed-matched blocks (fast-binding class medians 4.1, 3.1, 10.0 and 4.0 s; slow class 13.7, 13.7 and 19.8 s, with no confirmation-check backends, because the stall did not occur in the completion runs). (b) Scaling decision to usable capacity per run (the recorded higher-rate arm, 12.0 requests per second per client, six runs per mechanism; medians 2.17 against 6.01 s; exact $p = 0.0022$, $\delta = -1.000$). Start-up class is the backend's measured socket bind time, under a second on some backends and several seconds on others; the stall is an intermittent platform cost at spawn, equal across the modes, and held fixed by the within-class comparison. Black dots are individual backends (a) and runs (b); coloured bars and marks are medians; whiskers span the min–max range. 63

Figure 5.7 Under a lost message with reduced headroom, only the modes that can re-derive readiness recover. (a) Slow share (served requests timed out or slower than 1 s), median across the seven seed-matched blocks per 10 s bin; the shaded band marks the onset window (first 150 s after the plateau starts). (b) Admitted backends per run (dots, one per block; bars, medians; whiskers, the min–max range). In the seven backend-notification runs no backend was admitted and 32–35 candidates per run were abandoned and torn down, while the fallback check, periodic polling and the confirmation check admit through their own checks (medians 5, 5 and 6 backends) with relief completing; no-fault runs admit on every mode, with zero abandoned candidates. 64

Figure 5.8 User-visible damage of the four admission modes under a premature message at the evaluated ten-second lead. Bars are the medians over the

usable pairs, dots the individual seed-matched pairs, and the dotted line the pre-registered 2 pp bound; a positive reading means the fault run’s share of slow requests was above that of its no-fault partner. The confirmation check admits at true servability, which also places it first to a first successful response after the message (11.0 s against 19.5 s under periodic polling). 66

List of Tables

Table 2.1	Search-term families by domain.	10
Table 2.2	Selection criteria.	10
Table 2.3	The three interfaces of the demand-to-capacity chain and the questions they leave open.	21
Table 4.1	Platform components as deployed.	37
Table 4.2	The four telemetry modes and their implemented sources.	41
Table 4.3	Implementation checks and their outcomes.	44
Table 5.1	Evaluation testbed configuration common to the three campaigns.	45
Table 5.2	The five request kinds that make up the workload mixes, and the tier each loads.	46
Table 5.3	Campaign workloads. Each campaign defines its own phase sequence, run in the order shown. The demand episode is the phase that applies the demand shift under test.	47
Table 5.4	Design of the telemetry campaign (RQ1).	51
Table 5.5	Telemetry realisation and what the controller saw about the load rise (mean of the seven replicates, observed range in parentheses): share of telemetry summaries delivered, and information age at decision, the time from the closing of the summary that a decision used to the decision itself.	51
Table 5.6	Telemetry modes: usable capacity, episode latency, and outcome classes (medians with min–max ranges over the seven replicates; delivered share as the observed range; no timeouts in the episode in any mode). Usable capacity is the time from episode onset to first new capacity serving traffic; the episode p95 is the served-basis endpoint latency p95 (per-run mean of the two domains).	52
Table 5.7	Pre-registered primary contrasts on the episode endpoint latency p95 (served basis, per-run mean of the two domains). Exact Mann–Whitney p-values at $n = 7$ per mode. The effect size, its interval, and the direction count are defined in Section 5.1.4. The direction count gives how many blocks (of seven) followed the predicted order.	52
Table 5.8	Design of the scaling action campaign (RQ2).	55
Table 5.9	Design of the readiness admission campaign (RQ3).	61

Table 5.10 Admission-chain intervals and healthy-regime outcomes (intact message; backend notification versus periodic polling). Medians over runs and per-backend intervals as marked; exact Mann–Whitney p-values as pre-registered. 62

Table 5.11 Admission modes under a lost message (the RQ3 campaign; six blocks, plateau pooled; the seventh block's pair is report-only under the pre-registered control-tail rule). Admitted = visibility in the routing pool; abandoned candidates in parentheses. Plateau tail = slow share over the last 300 s of the plateau; *block 4 is excluded from the fault comparison, its no-fault control tail (4.08 %) breaching the 2 pp floor. 64

Table 5.12 User-visible damage of the admission modes under a premature message at the evaluated ten-second lead (seven seed-matched pairs per mode). Damage is the change, in percentage points, in the share of requests that fail, time out, or exceed one second, against a matched no-fault run; medians over the seven seed-matched pairs. A positive reading means the fault run's share was above that of its no-fault partner. 65

Table 5.13 Synthesis of the three campaigns, classified by whether a finding held for every replicate entering the contrast (robust), depended on a condition the campaign varied or sampled (conditional), or could not be confirmed (not established). Every entry remains within its own campaign, and no quantity is combined across campaigns. 68

Table A.1 Container resource limits, flow timeouts, and client bounds per campaign. 77

Table A.2 Scaling-policy parameters per campaign. 78

Table A.3 Load-balancing weights per campaign. 78

Table A.4 Control-plane cost per configuration (descriptive; medians across the runs of each configuration). Delivery records count delivered telemetry summaries; action records count provisioned units; admission records count events or readiness checks. 79

Table A.5 Implementation knobs of the interface experiments. 80

List of Acronyms

SDN: Software-Defined Networking

VIP: Virtual IP

ZMQ: ZeroMQ

DSRM: Design Science Research Methodology

CHAPTER 1

Introduction

Edge web services that encompass monitoring, auto-scaling and routing are commonly built from separate components, each with its own policies. How well such a service absorbs a change in demand therefore depends not only on what each component does individually, but also on the interfaces between them. Coordination across those interfaces can fall short, and where it does, orchestration efficiency and reliability can degrade and may reduce the quality of service users experience [1, 2].

The remainder of this chapter is organised as follows. Section 1.1 presents the context, motivation and problem statement; Section 1.2 states the objectives; Section 1.3 lists the research questions; Section 1.4 details the methodology used to develop the thesis; Section 1.5 summarises the main contributions; and Section 1.6 outlines the rest of the dissertation.

1.1. Context, Motivation, and Problem Statement

In 2025, global internet traffic across fixed and mobile networks exceeded 7 000 exabytes[1], more than doubling since 2020 [3], as shown in Figure 1.1.

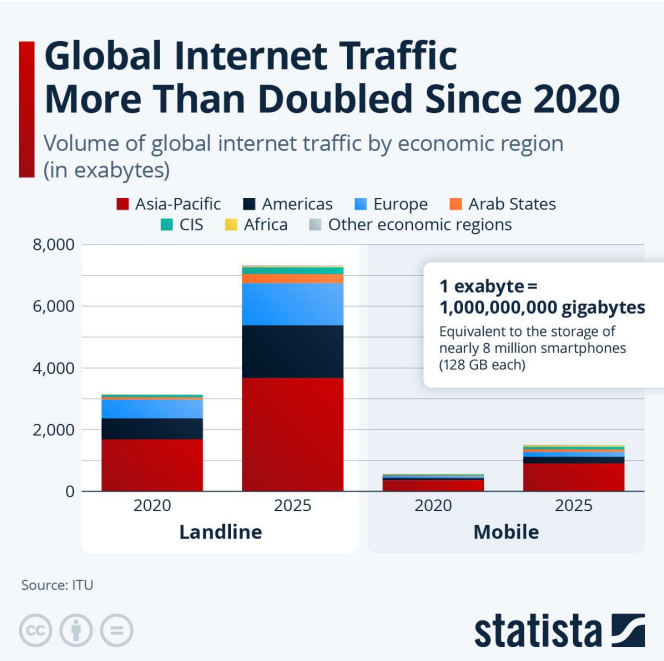


FIGURE 1.1. Global internet traffic between 2020 and 2025. Source: Statista, based on ITU data [3].

[1] One exabyte equals 10^9 gigabytes.

This growth has been driven by many technological advancements, some of which have in turn spurred the emergence of real-time interactive and data-intensive applications, among others. Examples include web streaming and cloud gaming, both services where users expect sub-second latency and high-resolution multimedia, highlighting the need for fast and capable data processing.

Cloud computing has been the backbone that enables these applications to operate at scale. It offers automatic scaling, both horizontally and vertically, a pay-per-use model and, among other aspects, a distributed offering and availability that have established it as the foundational layer supporting modern applications and services [4].

Yet, in parallel, a growing share of internet traffic now originates from mobile devices, which have become the main gateway to internet access [5]. Much of this access is therefore generated and consumed at the network edge. Edge computing brings computational and storage resources closer to the end users, allowing for much lower latency in delivering services and applications [6, 7]. Since data can be processed and stored locally, only a subset of it is sent to the cloud [6].

Despite these advantages, edge resources are dispersed, which translates into less capacity per location and greater complexity in managing the distributed edge servers. As Satyanarayanan puts it, "the dispersion inherent in edge computing raises the complexity of management considerably" [7, p. 37]. This complexity is aggravated by considerable variation in user demand over time and by the heterogeneous resource requirements of the requests served. Luo et al. further highlight that, without efficient resource scheduling, this distributed nature leads to resource waste [8]. Resource management therefore becomes increasingly important, since the limited capacity of each location must be adjusted over time to suit the constant fluctuation of user demand.

The resource management problem gains more relevance when the services themselves are stateful. Unlike stateless microservices, which can be replicated freely and absorb traffic immediately, a stateful edge service depends on data held close to the compute, so its storage tier is itself a live capacity that must follow demand alongside the compute tier. Making that capacity usable takes time, since a new storage unit must synchronise before it can serve reads, and maintaining the replicated copies consumes storage and replication bandwidth [9].

Scale-up alone does not guarantee usable capacity instantly. The orchestration must detect the demand shift, deliver that information to the decision point, provision the right kind of capacity, and redirect traffic to it. The steps hand over to one another across boundaries between functions, and what the next step acts on can be late, incomplete, mismatched, or unverified.

This sequence forms the demand-to-capacity chain, the path that an orchestration system takes to convert a change in overall user demand into capacity that serves that demand. A demand shift is first observed by monitoring and telemetry, which deliver the evidence to a decision point. The decision point selects a scaling action, such as scaling compute or scaling storage, and the action provisions new capacity. Finally the new capacity attains a readiness state (it can answer requests, not merely run) and starts receiving traffic, which is the point at

which it becomes usable. The chain is therefore a timeline through the orchestration system, and its steps are joined by interfaces.[2] The three interfaces between its steps, highlighted in Figure 1.2, are the subject of this thesis.

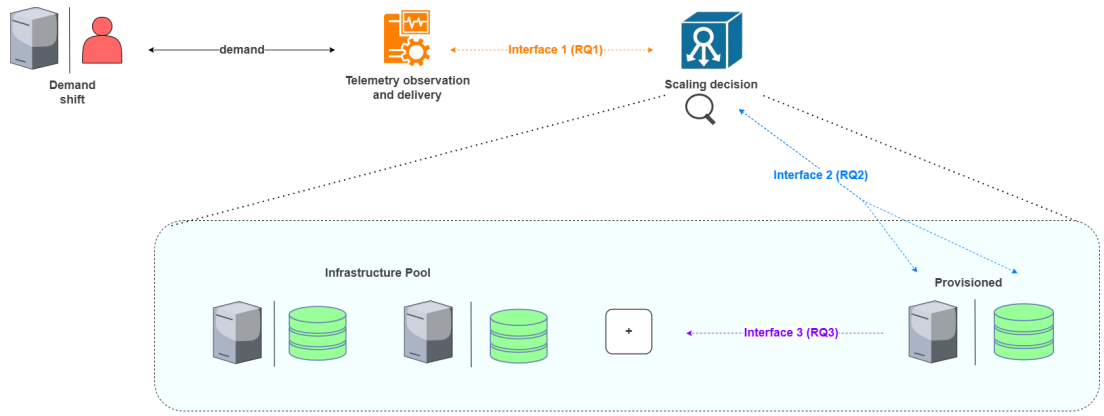


FIGURE 1.2. The demand-to-capacity chain: from a demand shift to usable capacity, with the three interfaces studied in this dissertation highlighted.

The steps described above must be coordinated under the time pressure of an ongoing demand shift. In practice, however, they are commonly performed by separate components. A monitoring system collects metrics, an auto-scaler provisions capacity, and a load balancer redirects traffic, each running under its own control loop and acting on its own view of the service. In the reviewed literature, the cost of these crossings is acknowledged but never isolated or varied as an independent variable [10, 11]. The crossings also differ in more than latency: telemetry can arrive complete and fresh, complete but delayed, or with summaries missing [12]; a capacity command can serve the tier that binds the demand or one that does not [13]; and the state that admits a backend to traffic can be late or inaccurate [14]. This thesis isolates the three interfaces and measures the effect of each of these properties on its own. The platform built for this purpose is organised as two peer Software-Defined Networking (SDN) controllers, one per network domain. Within each controller the consuming functions of the chain (telemetry consumption, scaling decisions, and traffic routing) run in the same process, so that each studied interface can be configured and varied on its own rather than observed as an incidental

[2] Throughout this dissertation, the *demand-to-capacity chain* denotes the full timeline from a demand shift to usable capacity. An *interface* is a boundary between two steps of the chain at which two functions meet, and what crosses an interface is the evidence, the command, or the state that the next step acts on. A *handoff delay* is the latency between a function's handover and the next step acting on it. The reviewed literature addresses this topic mainly at boundaries between functions that run under independent control loops, each observing, deciding, and acting on its own. The properties of what crosses (the completeness and freshness of the evidence, the match between the command and the demand, and whether a ready backend is admitted on the backend's notification alone, on the notification with a fallback check, through periodic polling, or on the notification with a confirmation check) are studied here one at a time. A check is the controller asking the backend directly whether it can serve. The *coordination gap* is the accumulated effect along the chain, which this dissertation does not measure as a whole.

coupling. Telemetry production and aggregation remain external to the controller, so how aggregated telemetry reaches the controller is an explicit boundary, while scaling and admission are exposed as lifecycle hooks.

Along this chain, three interfaces are studied in a stateful service deployed across two network domains: how telemetry reaches the controller (the telemetry mode); whether the capacity command serves the tier the demand presses on (bottleneck-aware scaling action); and whether a ready backend is admitted on the backend's notification alone, on the notification with a fallback check, through periodic polling, or on the notification with a confirmation check (readiness admission: when a ready backend starts receiving traffic).

The thesis does not claim that this SDN-based architecture is superior to existing orchestration platforms such as Kubernetes or NFV Management and Orchestration (MANO) platforms, nor that its mechanisms generalise beyond the tested infrastructure. It claims only that these three interfaces are measurable in isolation and that measurement can establish which property of each interface decides the outcome, and under what conditions. The reviewed literature has not previously characterised them this way, and no aggregate cost along the chain is claimed.

1.2. Objectives

The main objective of this thesis is to contribute to more efficient resource orchestration for stateful edge services by characterising the interfaces of the demand-to-capacity chain under controlled demand regimes, and by establishing, for each interface, which property of the crossing decides the outcome and under which conditions: how telemetry reaches the controller, whether the scaling action serves the tier the demand presses on, and whether a ready backend is admitted on its notification alone, on the notification with a fallback check, through periodic polling, or on the notification with a confirmation check.

The platform keeps controller-side coordination in-process, with how telemetry reaches the controller retained as an explicit, configurable boundary. Along with explicit lifecycle hooks for scaling and admission, this arrangement exposes the studied interfaces to direct measurement. Because the components share one process and are therefore coupled, making each interface independently configurable is itself a deliberate design choice, so that the platform can vary the telemetry mode, the scaling action taken, or the admission of ready capacity, and measure the effect.

1.3. Research Questions

This thesis seeks to answer the following three research questions:

- *RQ1*: How do the four telemetry modes (push, delayed push, polling, and sampled push) affect how the controller scales the service when load rises?
- *RQ2*: When load presses on the servers or on the data, how does scaling whichever is under pressure, rather than always scaling the servers or always scaling the data, affect service recovery and resource use?
- *RQ3*: When a new backend is provisioned, how do the four admission modes (backend notification, notification with a fallback check, periodic polling, and notification with a

confirmation check) affect whether and when the new capacity starts serving requests and what users experience, under three message conditions: the message arrives intact, never arrives, or arrives before the backend can serve?

1.4. Research Methodology

The work presented in this dissertation follows the Design Science Research (DSR) methodology [15], [16] proposed in [17], and demonstrated in Figure 1.3.

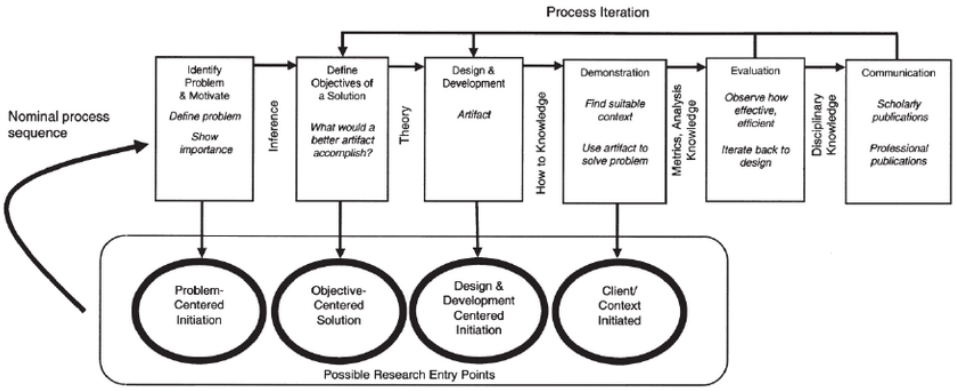


FIGURE 1.3. Design Science Research Methodology

Design science produces and evaluates purposeful artefacts that address certain real-world problems, the research contribution corresponds to the knowledge gained by designing the artefact and evaluating it against defined objectives. In Design Science Research Methodology (DSRM) the research is organised into six categories: problem identification and motivation, definition of the objectives of a solution, design and development, demonstration, evaluation, and communication.

This dissertation follows a problem-centred approach, in which the problem motivates the research and is defined as the coordination between the functions that orchestrate an edge service, where handovers that are late, incomplete, mismatched, or unverified can lower orchestration efficiency and the quality of service users experience. The problem definition is grounded in the state of the art reviewed in Chapter 2. The six activities unfolded as follows.

(1) *Problem identification and motivation.* Edge services whose monitoring, scaling and routing modules operate as independent components depend on information that crosses the boundaries between them, and how those crossings are operated can degrade orchestration efficiency. The problem is introduced in Section 1.1, and the review of the state of the art in Chapter 2 establishes both that these functions are typically studied in isolation and that single-process architectures with independently configurable interfaces are less commonly present in the reviewed literature, which motivates and delimits the problem.

(2) *Objectives of a solution.* As defined in Section 1.2, the solution is an experimental platform, not a novel algorithm, whose purpose is to make the interfaces of the demand-to-capacity chain measurable, so that their impact on resource orchestration can be evaluated through measurement and better understood.

(3) *Design and development.* The artefact is the single-process platform described in Chapters 3 and 4: two peer SDN controllers (OS-Ken/OpenFlow), one per network domain, each hosting controller-side telemetry consumption, elasticity decision-making and the double-VIP routing model, with Docker containerised service backends and a MongoDB replica set per domain serving as the storage tier, each scaled in and out through MongoDB native replication. The technology choices are justified by their utility for the platform. SDN provides a single programmable control plane through which routing behaviour can be re-programmed directly in a central manner, Docker enables fast provisioning and teardown of service backends, and MongoDB provides native replica-set replication whose membership the platform changes at run time.

(4) *Demonstration.* The artefact was deployed as a running system in an emulated edge environment with two network domains, and was driven with scheduled workload phases reproducing demand shifts in order to demonstrate that the platform operates end-to-end from demand observation through scaling action to traffic admission, with the tested interface varied across runs of each experimental campaign for evaluation.

(5) *Evaluation.* Each research question was evaluated in a dedicated experimental campaign reported in Chapter 5. The evaluation follows a pre-registered measurement and statistical procedure. A campaign is the set of independent runs that answers one research question by varying one interface of the demand-to-capacity chain while the others are held fixed; each run executes the campaign's workload and is treated as an experimental unit. The workloads are issued by a scheduled, open-loop workload driver, which applies the offered request schedule independently of the responses it receives, so that the offered load is preserved even when the service degrades. The primary comparisons are detailed in Section 5.1. Internal, external and repeatability validity threats are discussed in Section 6.3.

(6) *Communication.* The communication of the problem, the artefact and the findings is accomplished through this dissertation itself and its public defence, as well as through any resulting publications.

1.5. Main Contributions of this Dissertation

This dissertation makes the following contributions:

(1) An experimental characterisation of four telemetry modes (push, delayed push, polling, and sampled push) and of how they affect how the controller scales the service when load rises (RQ1).

(2) An experimental characterisation of bottleneck-aware scaling action selection, more specifically how deciding between compute and storage scale-up from the observed bottleneck, compared with fixed single-tier policies, affects targeted-tier relief, service recovery, and resource use under compute-bound and data-access-bound demand (RQ2).

(3) An experimental characterisation of readiness admission, specifically how the four admission modes (backend notification, notification with a fallback check, periodic

polling, and notification with a confirmation check) affect whether and when the new capacity starts serving requests and what users experience, under three message conditions: the message arrives intact, never arrives, or arrives before the backend can serve (RQ3).

(4) A comparative synthesis of the three interfaces studied (telemetry modes, scaling decisions, and readiness admission) under their respective controlled workload regimes, identifying which effects were robust, conditional, or not established.

These contributions are claimed within the evaluated two-domain testbed and workload families.

1.6. Dissertation Structure

This dissertation is organised into six chapters:

- *Chapter 1* introduces the research, presenting its context, motivation and problem statement, the objectives, the research questions that emerged, the methodology that guided the work, and the main contributions.
- *Chapter 2* reviews the background and related work. After presenting the methodology used to conduct the review, it surveys the platform technologies on which the proposed solution rests (edge platforms, SDN and OpenFlow, containers, and NoSQL document stores), then monitoring and telemetry, auto-scaling, load balancing and readiness admission on SDN, and resource orchestration on SDN, examines the default separation of monitoring, scaling and routing into independent components observed across these areas, and concludes with the research gaps that motivate this dissertation.
- *Chapter 3* presents the architecture and design of the proposed solution for edge services: the design requirements, the SDN-based architecture, the distributed elastic allocation of compute and storage resources, the monitoring and decision engine, and the control workflow.
- *Chapter 4* describes the implementation of the proposed solution: the experimental infrastructure, the implementation of the elastic allocation of compute and storage resources, the monitoring and decision engine, the control workflow, and the validation of the implementation.
- *Chapter 5* presents the experimental results. Following the demand-to-capacity chain, the experimental campaigns address the telemetry modes (RQ1), the bottleneck-aware scaling action (RQ2), and the readiness admission modes (RQ3), followed by a synthesis across the three interfaces.
- *Chapter 6* concludes the dissertation, restating the findings and the contributions, discussing the limitations of the work, and outlining directions for future research.

CHAPTER 2

Background and Related Work

This chapter presents the background and related work that motivated the research, selected through the search strategy described in Section 2.1. The review is separated into two parts. First, the technologies and concepts used to build the experimental platform are explained and surveyed, each introduced for what it is and why it suits the edge scenario. These are, more specifically, edge computing, SDN and OpenFlow, containerisation, and NoSQL document stores with a focus on MongoDB.

On top of these technologies runs resource orchestration, the layer that joins monitoring and telemetry, auto-scaling, and load balancing into one service. The review then focuses on the three interfaces of the demand-to-capacity chain: how a load rise is detected, how the scaling action is chosen when several are available, and how new capacity is admitted to traffic.

For each interface the review asks the same two questions: does the reviewed work vary this interface, and does it measure what happens downstream? This grounds the claim of Section 1.1: the literature acknowledges these interfaces but has not studied them one at a time. Section 2.6 reviews the platforms these functions run on, and Section 2.7 collects the gaps.

2.1. Review Methodology

Searches were executed in Google Scholar and IEEE Xplore for publications from 2016 onward, with the number of citations taken into consideration. Works reached through Google Scholar were retrieved from IEEE Xplore, ACM Digital Library, Springer, and ScienceDirect. Each query resulted in roughly 30 papers evaluated for selection. Table 2.1 groups the search-term families by domain, and Table 2.2 states the selection criteria applied to the retrieved hits.

Screening and citation chaining produced the reviewed literature, 37 works organised by the interface each work informs: telemetry, scaling decisions, readiness admission, and platform context. The platform technologies presented in Section 2.2 draw on eight sources, treated as context rather than as independent evidence of an RQ gap. The synthesis is recorded in Section 2.7.

2.2. Platform Technology Background

This section presents and explains the four building blocks that define the testbed evaluated in this thesis, edge computing, SDN with OpenFlow, containerisation, and MongoDB.

Each subsection states what the building block is and why it matters for the thesis scenario. How the building blocks are applied is the subject of Chapters 3 and 4.

TABLE 2.1. Search-term families by domain.

Domain	Example search terms
Monitoring and telemetry	"SDN monitoring", "SDN telemetry", "push vs pull telemetry", "monitoring delay SDN", "container monitoring edge", "edge platform monitoring"
Auto-scaling and data	"auto scaling SDN", "scaling trigger", "Kubernetes HPA end-to-end latency", "application metrics", "edge storage survey", "edge data placement"
Load balancing	"load balancing SDN", "handoff delay"
Platform context	"cross-layer orchestration", "SDN resource orchestration edge", "edge platform survey", "geo-distributed edge", "stateful edge web services"

TABLE 2.2. Selection criteria.

Inclusion	Exclusion
Addresses at least one of the three interfaces in software-defined, containerised, or edge settings	Focus outside the orchestration of service capacity (e.g., physical layer, security-specific, domain applications without an orchestration contribution)
Surveys and taxonomies that frame one of the domains	Duplicates
Seminal works reached by citation chaining	Non-English or full text unavailable
Full text freely available or accessible under ISCTE's scientific access licences	
Published from 2016 onward, with the number of citations taken into consideration, and retained on topical relevance	

2.2.1. Edge Computing

Edge computing is the placement of computational and storage resources between end devices and centralised cloud data centres, close to where data is produced and consumed. It complements rather than replaces the cloud, as latency-sensitive processing is shifted to the edge, while large-scale analytics and long-term storage remain in the cloud [6].

In essence, edge computing exists because centralised clouds alone cannot serve every workload efficiently, and bringing computation closer to the user lowers the latency of interactive and data-intensive services, and processing and storing data locally reduces the ingress bandwidth into the cloud [7].

Despite the advantages of edge computing, as mentioned previously in Section 1.1, the inherent dispersion of edge resources increases their management and orchestration complexity.

Resource heterogeneity, scarcity, and demand varying over time are all aspects that can degrade resource orchestration efficiency, making it a concept of continuous study and research [8, 18]. Scheduling that does not follow the workload leads either to over-provisioning, where resources are held but not used, or to under-provisioning, where service quality degrades [8]. Luo et al. also identify that the joint scheduling of communication, computing, and storage resources is an open direction for the field [8].

Managing these platforms remains an open challenge, and their combination with SDN is recorded as an area with room for further investigation [19]. The service evaluated in this dissertation runs in an emulated edge environment as a stateful edge service whose capacity must follow fluctuating demand, which is the management problem the remainder of this chapter reviews.

2.2.2. SDN and OpenFlow

Software-Defined Networking separates the control plane, which decides how traffic is handled, from the data plane, which forwards packets, so that forwarding behaviour can be reprogrammed centrally from a controller instead of being configured device by device.

Baktir et al. survey how edge computing benefits from this decoupling and organise the gains into benefit areas such as high-resolution control built on a centralised view and programmatic access, arguing that SDN can act as the enabler that lowers the complexity barriers and enhances the flexibility of edge deployments [20].

The OpenFlow protocol implements the decoupling through match-action flow rules installed on the switches, and is, as reviewed by Waseem et al., the most extensively used SDN protocol in the research community [21].

In OpenFlow the header fields a switch can match and rewrite are fixed by the standard, and new fields arrive with new versions of the protocol. Languages such as P4 take a different approach, letting the operator write the program that defines how packets are parsed and processed [22].

In its OpenFlow form, a single controller re-programs the forwarding behaviour of the network by installing and updating match-action flow rules, so that network behaviour can follow the life cycle of the services it carries without reconfiguring the forwarding devices individually. In this thesis, that programmability keeps the routing decisions inside the platform's control loop: which backend serves a request, when newly scaled capacity joins the traffic path, and which storage member serves a read are all decided by the controller and enacted as flow rules. The decoupling is illustrated in Figure 2.1.

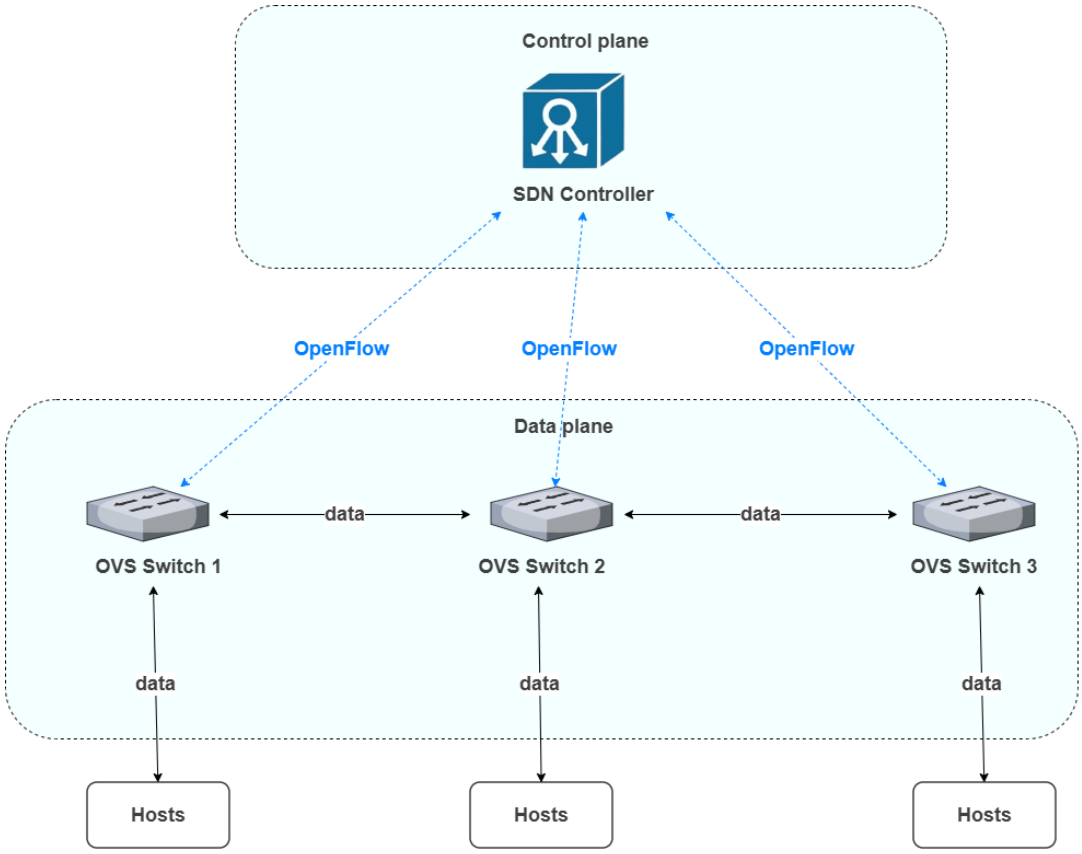


FIGURE 2.1. Software-Defined Networking architecture: the control plane is decoupled from the data plane and programs the forwarding devices through the OpenFlow protocol.

2.2.3. Containerisation and Docker

Containers package an application with its dependencies and share the host kernel, which makes them lightweight and quick to start up compared to virtual machines, properties that suit resource-constrained edge platforms. In this thesis Docker is adopted as the platform's containerisation tool because of its wide adoption for building and running containers.

2.2.4. NoSQL and MongoDB

Document-oriented NoSQL stores organise data as schema-flexible documents rather than as rows of relational tables. Because related data is embedded in a single document and the model avoids joins and fixed schemas, records can be partitioned and replicated across servers, which is how these stores scale horizontally, whereas relational deployments have traditionally grown by enlarging a single server. This model suits the heterogeneous data of edge and IoT services, whose structure varies between sources and evolves over time.

MongoDB is a document store of this family. It holds data as BSON documents within collections and provides a flexible schema. In MongoDB, redundancy is organised natively around the replica set, a group of `mongod` processes that maintain the same data set. One member is elected primary and serves the clients' writes, which it records in an operation log, the remaining members, the secondaries, replicate by applying that log asynchronously and autonomously.

Members exchange heartbeats, and when the primary fails or becomes unreachable, the secondaries hold an election and promote one of their own, so the set keeps serving without operator intervention [23]. Members can also be added to and removed from a deployment while it keeps serving, which turns the replica set into a controllable unit of storage capacity rather than a fixed configuration. Figure 2.2 shows the members of a set and the replication paths between them.

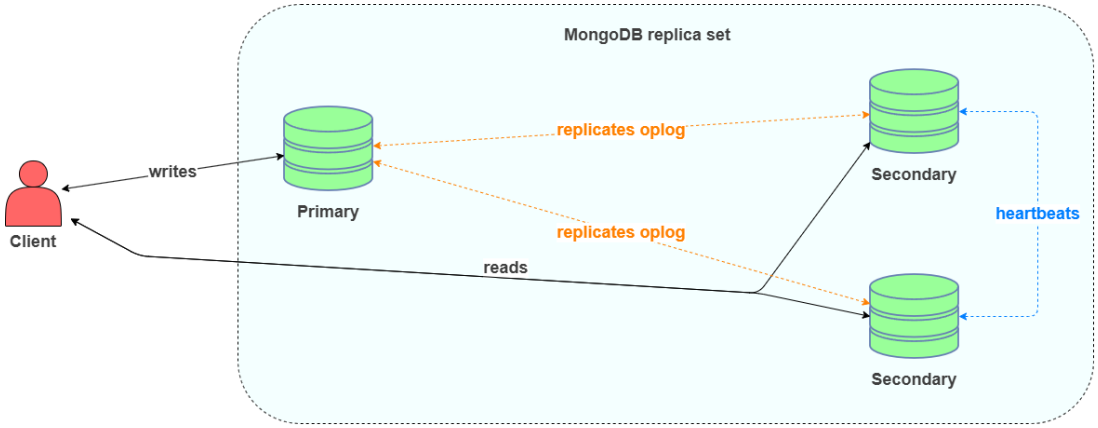


FIGURE 2.2. MongoDB replica set: a primary member serving the clients and secondary members to which the data is replicated.

This dissertation builds on that capability. The storage tier is scaled by adding and removing replica-set members while the set keeps serving, leveraging MongoDB's native replication.

2.3. Telemetry

Monitoring collects the resource and load metrics of a running service, and telemetry carries them to the controller. Every scaling decision is taken from that telemetry, so when it is late or incomplete the reaction is late or based on an incomplete picture. In an edge deployment the metrics are produced across several nodes before they reach the decision point. How telemetry arrives is therefore the first interface studied in this thesis.

Telemetry is delivered in one of two common patterns: event-driven, where the producer sends each summary as it closes, and polling, where the consumer reads the latest summary at a fixed interval. Either pattern can reach the consumer late, or with summaries missing, according to how it is configured: a read at the same interval as the summaries delivers every summary, at most one period old, while a slower read, or a producer that samples or holds summaries back, loses or delays them. The four telemetry modes compared in Chapter 5 are these two defects separately and together: push (every summary, on time), delayed push (every summary, 30 s late), sampled push (one summary in three, on time), and polling (only the latest summary at each 30 s read, so it is late and misses two in three).

Monitoring is conventionally pull-based, with the consumer requesting the latest sample on a fixed interval [12]. In orchestrators this is a periodic scrape into a time-series database, as in OSM-based NFV, where the collector polls the infrastructure into Prometheus and the control modules read it [11]. The reviewed work that comes closest to the delivery question is AdapPF, which varies the Prometheus scrape interval in a Kubernetes federation and shows that

a coarse sixty-second interval degrades scheduling accuracy relative to a five-second one, while an adaptive interval restores it and reduces monitoring traffic [24]. Yaseen records the "visibility gaps" that polling leaves between reads, and calls for monitoring tightly integrated with control [12]. Hung et al. show that telemetry collection and routing can run at one controller, but their loop does not scale the service [25]. Usman et al. leave the choice of measurement interval open [26]. Zhou and Yong change what is monitored, from CPU load to application errors, and scaling quality follows the metric, but the delivery itself is not varied, and the finding comes from a single small testbed [27].

Within the reviewed literature, no work separates late telemetry from missing telemetry, and none follows the effect of a telemetry mode through a stateful service to what its users experience. AdapPF comes closest: it varies freshness and measures a downstream consequence, but its comparison does not separate late from missing updates, and the service it measures is not stateful. Separating delay from loss, and measuring both through the service, is the gap this dissertation fills.

2.4. Auto Scaling

Auto-scaling adjusts the amount of allocated resources to the load that a service receives. When demand grows, additional capacity is provisioned, and when demand recedes, surplus capacity is released, so that the service keeps serving requests without holding resources it does not use. Auto-scaling strategies are mostly grouped into horizontal scaling, which changes the number of units, and vertical scaling, which changes the capacity of a single unit, with hybrid strategies combining the two [28]. Horizontal scaling suits the edge scenario, and is the direction this dissertation follows. Throughout this dissertation, adding a unit of capacity to a tier is referred to as scale-up, and removing one as scale-down.

In edge computing, a site's capacity is bounded by the nodes deployed there, so enlarging an individual unit has a hard ceiling, whereas adjusting the number of units lets capacity grow and shrink as demand changes. When and how many resources to add or remove therefore determines both the quality of service during a demand shift and the efficiency with which scarce resources are managed.

Qu et al. classify the auto-scalers along several characteristics, among them the scaling indicator, the resource estimation method, and the timing of the action, and Taherizadeh and Stankovski present a taxonomy for edge computing whose dimensions include adaptation intervals and architectural support for multi-tier applications, noting that a scaling approach should account for different dynamic provisioning policies at each tier. Both surveys take the scaling of service instances, whether virtual machines or containers, as the unit of analysis. The main questions the field asks are when to scale and how many instances to add, which kind of capacity is provisioned, when applicable, is not among the classification dimensions [10, 29].

With SDN, the traffic side is added, but the scaled resource remains the one the service function is built from. Llorens-Carrodeguas et al. couple SDN-based traffic steering with horizontal scaling of virtualised network functions, so that the SDN controllers re-program the forwarding paths while instances of the same pre-dimensioned function pool are activated or deactivated.

The decision reduces to whether and how many instances of that one function to activate.[11]. In a stateful edge service, however, the storage tier is itself a live capacity. When the pressure sits in the data rather than in the function instances, adding instances does not relieve the storage tier.

Predictive and learning-based approaches sharpen the same decision without widening it. Toka et al. formalise the Kubernetes horizontal autoscaler's control model to anticipate demand for edge clusters [30], Ju et al. and Cheng et al. forecast workload to adjust replica counts ahead of demand, the latter jointly with placement [31, 32], and Li et al. couple scaling and placement in a single optimisation [33]. Across these systems the optimisation lies on the forecast and adjusting the number of compute replicas over time. The improvement pursued is a better forecast of how many instances to run, which tier holds the limiting resource does not enter the scope of study.

Scaling one resource does not always make the service better, because the pressure may sit elsewhere. Nikravesh et al. quantify what that default costs in a three-tier web service. The scaling decisions taken for the business tier turn out to be insensitive to database capacity, yet reducing database capacity raises the rate of SLA violations, so service quality can be governed by the tier that the scaling decisions ignore [13]. Dogani et al. make the same observation in their survey, noting that adding resources does not necessarily raise throughput when the bottleneck lies elsewhere [34]. Among the systems covered by that survey, only a few take cross-service effects into account. Waterfall, a machine-learning auto-scaler, accounts for how scaling one microservice can shift the bottleneck towards downstream services. The mechanism remains compute-side, and no surveyed system selects a storage action.

PBScaler identifies the microservice whose degradation propagates through the invocation graph, ranks candidate bottlenecks, and optimises replica management around the identified bottleneck [35]. Bottleneck awareness therefore exists in the reviewed state of the art, but the bottleneck it locates is always a microservice and the action it takes is always replica scaling, so the choice between compute and storage capacity remains outside the action space.

The closest reviewed precedent to choosing between resource tiers is the provisioning model of Li et al., which adds both cloud compute instances and data replicas in cloud-assisted edge systems, from a forecast of future workload built over historical workload data [36]. Both actions are available, but the trigger is a forecast rather than a live observed bottleneck, and the capacity is provisioned across edge hosts and rented cloud instances, with the outcome measured in tenancy cost.

Even among the most recent surveys regarding autoscaling, El Mrabet et al. extend the classification across cloud, fog, and edge, categorising the surveyed methods by their decision mechanism, scaling strategy, and deployment layer, and the provisioned tier is not among the classification dimensions [28].

Qu et al. describe a web application as split into a front-end, application logic, and a database tier, and record that "the database tier is often considered dynamically unscalable and ignored by auto-scalers" [10, p. 6].

Overall, the literature demonstrates that the kind of capacity added under certain scenarios can have an impact, compute replicas do not relieve pressure whose limiting resource lies in the storage tier, and database capacity can govern SLA violations while a compute-tier scaling decision remains unchanged. Scaling actions are therefore not interchangeable merely because each adds resources.

Therefore in the reviewed studies, the choice between these actions remains an architectural given, settled by forecasts or by joint placement rather than chosen at runtime from a live observed bottleneck. This dissertation isolates that choice, with storage scale-up among the actions the controller may take, and measures the resulting differences in targeted-tier relief, service recovery, and resource use. How a selected action becomes usable capacity is examined next.

2.5. Load Balancing and Readiness Admission on SDN

Load balancing distributes incoming traffic among a set of available backends so that no single backend is overloaded while others remain underutilised. In SDN networks this distribution is programmed centrally, where the controller installs forwarding rules that steer each flow to a selected backend, and a load-balancing policy decides which backend receives the next flow, as illustrated in Figure 2.3.

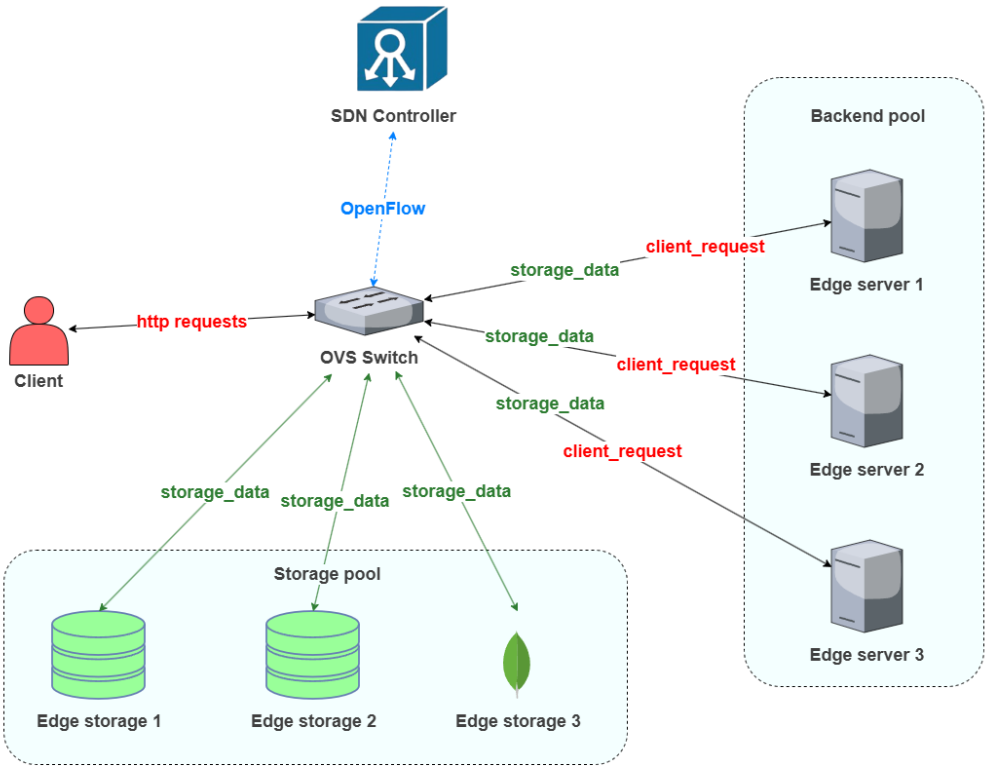


FIGURE 2.3. Load balancing in an SDN network: the controller installs forwarding rules that distribute client traffic among the available backends.

Load balancing is a common application of SDN. Belgaum et al. survey the field systematically and group its techniques into conventional and AI-based families [37]. Neghabi et al. classify the mechanisms as static and dynamic, and observe that controller state can fall out of

step between synchronizations [38]. Abdelltif et al. place the balancer on the controller itself, holding per-server load current over the OpenFlow pool [39]. Taken together, these studies treat load balancing as a selection problem. At a given routing decision, the policy decides which backend receives the next flow, choosing by criteria such as load, path properties, and resource utilisation. The set of backends receiving traffic is taken as given.

The membership of that set, however, is not fixed. Backends are added as demand rises and removed as it recedes. Capacity becomes usable only when traffic can reach it. A scaling action provisions a backend, but the provisioned backend serves nothing until it is admitted to the set of backends receiving traffic. The scaling loop that began with the demand shift closes at that admission, and the time a ready backend spends outside the set is therefore an admission delay. The reviewed work recognises the time this takes, but does not isolate the admission step.

Kurma, a load balancer for geo-distributed storage systems, is explicitly designed to operate at a granularity of seconds so that it can "work in tandem with modern elastic controllers, thereby reducing over-provisioning and SLO violations incurred during provisioning delays" [1, p. 386]. The delay between provisioning and usable capacity is therefore a recognised cost, yet the time a backend spends between becoming ready and starting to receive traffic is not measured on its own in that evaluation.

Llorens-Carrodeguas et al. couple auto-scaling with load balancing in an SDN redirector for a transparent VNF cluster, so that load balancing operates over a pool of instances that is resized while traffic is being served. Their system changes the number of active cluster members and removes flow rules for unhealthy instances using status obtained through the MANO monitoring path [11]. It therefore demonstrates that dynamic membership is already a legitimate concern. Its evaluation, however, reports activation for the whole system, so the time between a backend becoming ready and starting to receive traffic is not isolated in their evaluation.

Managing the pool itself is an established problem. Dreibholz and Mazumdar apply the IETF Reliable Server Pooling framework to a pool spanning edge and multi-cloud resources. A member registers to the pool when it joins and de-registers when it leaves, while a registrar keeps its availability current through periodic keep-alive checks. The framework's own analysis notes that "propagating load updates takes time, leading to temporary inaccuracy", and the mitigation for that inaccuracy is left to the load-balancing layer, in policies that discount stale pool state [14, p. 6].

Neghabi et al. record failure management as a concern that the mechanisms their survey covers have not taken into account, so failure-driven changes to the set of backends receiving traffic remain unaddressed [38].

The set of backends receiving traffic and the true serving state of its backends can therefore fall out of step in one of two directions. When a change reaches the routing plane late or not at all, ready capacity stays outside the set, and the nodes already serving carry the demand without it. When a backend is in the set while it cannot serve the traffic it receives, the requests routed to it fail.

This dissertation treats admission into that set as the variable: the four admission modes require different evidence before a backend is admitted (backend notification, notification with a fallback check, periodic polling, and notification with a confirmation check). The comparison measures how they affect whether and when the new capacity starts serving requests and what users experience, under three message conditions: the message arrives intact, never arrives, or arrives before the backend can serve. Chapter 5 evaluates the four modes under one regime.

2.6. SDN-Based Orchestration Platforms

Resource orchestration is the management layer that keeps a running service within its requirements by coordinating the compute, network, and storage resources of the infrastructure on which it runs. It is the activity that the technologies and tools reviewed in Section 2.2 are assembled to serve. Edge sites supply the resources, SDN supplies the programmable connectivity, containers supply the unit of compute, and the document store holds the service state. That decision-making, and its execution, is orchestration, and at the edge it is all the more demanding because each site holds scarce capacity and the demand on it fluctuates over time. For stateful services, the scaling action can address either side, because compute replicas serve requests, while the data they serve lives in the storage tier. Demand for data can therefore exist without demand for compute and vice versa.

This dissertation studies the SDN-based form of this architectural layer, in which the coordination spans the SDN controller, the container runtime, and the data store, and the challenge is to keep these layers working together as demand fluctuates. The platforms reviewed here bind these layers with different degrees of coupling, and this section examines how closely the three functions of the demand-to-capacity chain, monitoring, scaling, and routing, are held together in each, using the terms interface and coordination gap as defined in Section 1.1.

Baktir et al. organise the gains SDN brings to edge computing into benefit areas such as high-resolution control built on a centralised view and programmatic access [20]. Wang et al. survey architectures in which SDN and NFV jointly coordinate placement, traffic, and caching across the network and service layers, showing that SDN's role at the edge extends beyond forwarding into orchestration [40]. Sofia et al. argue for context-aware cross-layer orchestration of containerised edge-cloud applications serving next-generation IoT services, treating contextual information such as location and data freshness as part of the orchestration input [41]. These works document architectures or proofs of concept rather than the coordination inside a running system, and the interfaces between the functions fall outside the scope of all three.

Peng et al. exercise SDN-based resource management in vehicular multi-access edge computing as task migration and radio slicing, treating telemetry as available input rather than as an interface to be studied, and without container orchestration or tier scaling [42].

Some of these platforms bring the functions together at the SDN control point itself. Llorens-Carrodeguas et al. combine monitoring, auto-scaling, and load balancing in a single SDN redirector for a transparent VNF cluster, although metric collection and the lifecycle operations that feed and execute the decision remain external [11]; Hung et al. present an SDN controller for 5G backhaul that closes a telemetry-and-routing loop, with telemetry collection delegated to

decentralised microservices reached through an API, omitting scaling altogether [25]. The first study binds all three functions at one control point and the second binds two of the three, so combining the functions at an SDN control point has precedent in the literature and is leveraged in this dissertation. Neither system, however, reports the delay within its coordination.

At the geo-distributed scale, three systems coordinate the same functions. OneEdge keeps authoritative state at each edge site, reconciles it with a central aggregate through two-phase commits, and uses ZeroMQ (ZMQ) for communication among its distributed components. Its evaluation reports end-to-end deployment latency with a coarse breakdown into scheduling delay, WAN overhead, and container creation, but not the cost of each control-plane interaction [43]. McK8s deploys four controllers on a management cluster as Kubernetes operators over custom resources, connects member clusters through a Cilium mesh, and measures inter-cluster latencies with Serf, treating those latencies as input to placement rather than as an effect to study [44]. Kurma instances exchange load, latency, and SLO-violation measurements among datacenters and redirect requests at second granularity while elastic controllers provision capacity, naming the stages of the provisioning delay, detection, acquisition, and warm-up, before treating them as one combined quantity [1]. The coordination is built into each system, but the delay it carries is not isolated. None of their evaluations attributes the time taken by the state updates, exchanges, and redirects along those communication paths to the individual handoffs between observing demand, deciding, provisioning, and admitting capacity.

The related background is therefore organised in two setups. In the one setup, monitoring, scaling, and routing are implemented as separate components, each running its own control loop, so the evidence and commands that tie them together must cross component boundaries. The separation is not only architectural. In an evaluation across major cloud providers, Podolskiy et al. observe IaaS autoscalers and the surrounding layers acting independently, with load-balancing issues and increased latency appearing around demand shifts [2]. The measurement is observational, documenting the coordination symptom in cloud deployments rather than isolating an interface. In a second, recurring setup, the same functions are kept together at an SDN-based control point or coordinated through shared state. What neither arrangement provides, however, is a decomposition of the control process into its interfaces.Prior systems measure complete control processes or coordinate them by construction. They do not isolate how telemetry reaches the controller, scaling decisions, or readiness admission as independent variables in the demand-to-capacity path. That interface-level isolation is what this dissertation provides.

SDN-based resource orchestration is therefore an established and feasible arrangement, and the coordination it rests on is a recorded open issue. Nain et al. record resource optimisation in SDN-based edge environments among the continuing challenges of the field [19]. The interface properties studied in this dissertation are one measurable dimension of that coordination, and because the functions meet at boundaries that an SDN-based control point can expose as explicit, configurable interfaces, such a platform is a suitable experimental basis for evaluating them. The platform built in this dissertation follows that setup and exposes the telemetry mode, scaling

decisions, and readiness admission as independently configurable interfaces, as presented in Chapter 3.

2.7. Synthesis and Research Gaps

Across the reviewed literature, the cost of these crossings is acknowledged, but never measured on its own in a stateful service. Where a cost is measured, it is measured for a whole system: Kurma reports provisioning delay as one combined quantity [1], OneEdge breaks end-to-end deployment latency into scheduling, network, and container creation without attributing it to a single handoff [43], and Llorens-Carrodeguas et al. report only whole-system activation [11]. Pool membership management acknowledges that propagating state changes takes time, but leaves the mitigation to the selecting layer rather than measuring the interval [14]. The closest reviewed precedent is AdapPF, which varies monitoring freshness and measures a downstream consequence, but without separating late from missing updates and not in a stateful service [24]. The placement literature, as surveyed by Tabatabaee Malazi et al., names the interval between an instance's arrival and its readiness to serve as launch time, but the step from readiness to traffic admission lies outside its scope [45].

No reviewed work materialises the other properties of these crossings as experimental variables either. The loss of intermediate evidence is recorded as a limitation of polling (Section 2.3), the tier that a capacity action serves is settled by forecasts or by joint placement rather than chosen from a live observed bottleneck (Section 2.4), and failure-driven changes to the set of backends receiving traffic remain unaddressed by the reviewed mechanisms (Section 2.5).

Table 2.3 condenses these findings into the three interfaces and the question each leaves open.

The research questions of Section 1.3 are the experimental form of Table 2.3. This dissertation does not argue that the coordination gap is universal, only that, within the reviewed literature, these interfaces have not been isolated as independent variables. Each is measured on its own, and no aggregate cost along the chain is claimed. Chapter 3 presents the platform designed to make the questions askable and measurable.

TABLE 2.3. The three interfaces of the demand-to-capacity chain and the questions they leave open.

Interface	Open question
Telemetry to decision (RQ1)	Does the controller scale differently when updates arrive late but complete, versus when updates are missing?
Decision to scaling action (RQ2)	When compute and storage actions are both available, how much does choosing the action matched to an observed compute-bound or data-access-bound bottleneck change targeted-tier relief, service recovery, and resource use relative to fixed single-tier policies, and where do the effects concentrate?
Ready backend to traffic (RQ3)	When a newly provisioned backend becomes ready, how do the four admission modes (backend notification, notification with a fallback check, periodic polling, and notification with a confirmation check) affect whether and when the new capacity starts serving requests and what users experience, under three message conditions: the message arrives intact, never arrives, or arrives before the backend can serve?

CHAPTER 3

Architecture and Design

This chapter presents the architecture and design of the proposed orchestration platform for edge services. The design provides a platform that runs controller-side telemetry consumption, elasticity coordination, and traffic steering in one controller process per domain. The telemetry mode, the scaling decision, and the transitions from scaling action to traffic admission remain explicit observation points.

Section 3.1 defines the requirements that guided the proposed solution, and Section 3.2 presents the layered organisation of the platform and its two-domain deployment (Figure 3.1). Section 3.3 describes the distributed elastic allocation of compute and storage resources, Section 3.4 the monitoring and decision engine, and Section 3.5 the control workflow. Together, these sections realise the design requirements: the domain organisation and the data paths (DR1, DR4, DR5), the elastic allocation and the capacity changes (DR3, DR7), the observation and decision path (DR6, DR10), and the execution contexts and instrumented chain of the control workflow (DR2, DR8, DR9).

3.1. Design Requirements

The platform was designed to support edge services whose demand and resource requirements vary over time. The requirements below follow from the coordination problem and the research gaps established in Chapter 2, and guide the design presented in the remainder of the chapter.

DR1: Domain-scoped control. Each network domain must be operated by its own controller, which observes that domain's demand and acts on that domain's resources. Peer controllers exchange resource state, but the operation of a domain must not depend on a central entity.

DR2: Single-process control loop. Telemetry consumption, the scaling decision, load balancing, and the capacity changes must run in the same controller process, sharing their state directly rather than through inter-service handoffs.

DR3: Dual-tier, demand-driven elasticity. Compute and storage capacity must each follow their own demand and scale independently, and storage capacity must be changeable at run time from the platform's own code while the service keeps serving.

DR4: Stable addresses with network-enforced steering. Clients and servers must address stable virtual addresses, one for the service and one for storage reads, and the controller must enforce each routing decision on the network path, so that capacity changes never change the point of access.

DR5: Two-level load distribution. Request load must be distributed across the compute replicas and storage reads across the storage members, using the candidates' reported state, with only admitted capacity eligible.

DR6: Hierarchical reporting with freshness bounds. Edge servers and storage members must report to their domain's telemetry aggregator, and the controller must consume the summaries formed from these reports. Each summary must carry when each node last reported, and a node silent beyond a bound must be treated as absent, so that the controller acts only on capacity it can observe.

DR7: Staged capacity changes. Capacity must enter and leave service through staged transitions. Added capacity must be provisioned and registered before it can receive traffic, and removed capacity must be withdrawn from routing and drained of in-flight requests before it is released. The routing pools must contain only capacity that can serve.

DR8: Non-blocking steering. Packet handling must never wait on slow infrastructure work such as provisioning, membership joins, or removal.

DR9: Instrumented transitions. Every capacity change must be a sequence of explicit, timestamped transitions (provisioned, ready, admitted, serving), and every served request must be attributed to the node that served it, so that the chain from observed demand to the first request served by new capacity can be reconstructed.

DR10: Configurable interfaces. Telemetry delivery, the capacity action choice, and the admission mode must be independently selectable configurations of the platform.

3.2. Architecture

Within each domain the platform is organised in three layers (Figure 3.1). The control layer holds the controller, which consumes the telemetry summaries, performs load balancing, and decides the scaling actions. The data plane layer holds the software switch, which forwards the client requests and the storage traffic once the controller installs the forwarding rules. The resource layer holds the edge servers that serve requests, the storage members that hold the service state, and the telemetry aggregator that forms the summaries for the controller.

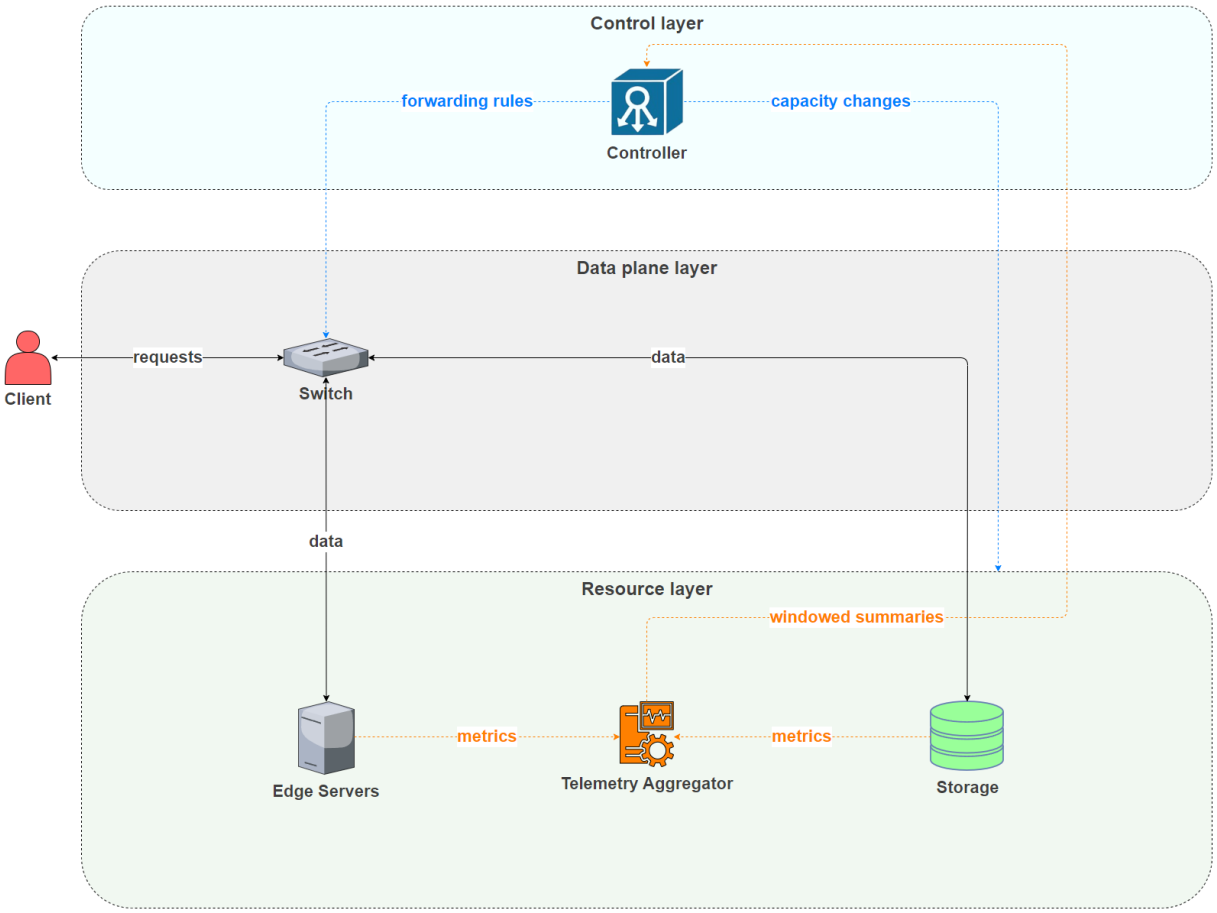


FIGURE 3.1. Layered organisation of the platform in one network domain: the controller of the control layer consumes the telemetry summaries and issues the forwarding rules to the switch and the capacity changes to the resources. The data plane layer carries the client requests and the storage traffic, and the resource layer holds the edge servers, the storage members, and the telemetry aggregator.

3.2.1. Two-Domain Organisation

The platform is organised as two network domains connected by an emulated inter-domain data link, representing the two sites of the evaluated deployment. Each domain contains client workloads, a software data plane, a pool of containerised edge servers, a replicated storage tier, a telemetry aggregator, and a controller. The peer controllers exchange topology information, and each consumes the telemetry summaries produced by the aggregators of both domains. Each controller runs telemetry consumption, elasticity coordination, and traffic steering in the same process for its own domain. Where this thesis says node without qualification, it means one deployed unit of either tier: an edge server of the compute pool, or a storage member of the replica set.

The emulated inter-domain link models the cross-domain data path, while telemetry and controller-to-controller topology exchange use host-local control channels. This distinction separates data-path delay from control-path communication.

3.2.2. Virtual IP Data Paths

Clients access edge services through a per-domain Virtual IP (VIP). Service requests are addressed to the virtual service address `VIP_SERVER`, and edge servers route storage reads through the virtual data address `VIP_DATA`, while storage writes are issued directly to the replica-set primary. This separation represents service routing and data access as distinct routing surfaces.

When a flow addressed to a virtual IP is first observed, the controller selects an eligible backend and configures the data plane to forward the flow. Subsequent matching packets are handled in the data plane without further controller involvement. Service load balancing may select a backend in either domain (a platform capability held constant in the evaluation), while cross-domain storage placement is disabled.

Load balancing is cost-based, for both service backends and storage members. The controller computes a weighted cost from each eligible backend's reported resource state (CPU utilisation, memory usage, request count, and hop distance) and installs forwarding rules for the lowest-cost one. Each metric enters the cost as a fraction of the largest value observed among the eligible candidates, so metrics with different units contribute on a common scale under configurable weights, and candidates that tie at the lowest cost are selected in round-robin. Hop distance expresses reach and in this architecture a peer-domain candidate is always assigned more hops than a local one, so local candidates are preferred at comparable resource state. For storage reads observed on the virtual data path, the same mechanism selects a replica-set member, using the corresponding storage metrics (CPU utilisation, memory usage, connections, replication lag, and hop distance). Figure 3.2 walks through the resulting sequence.

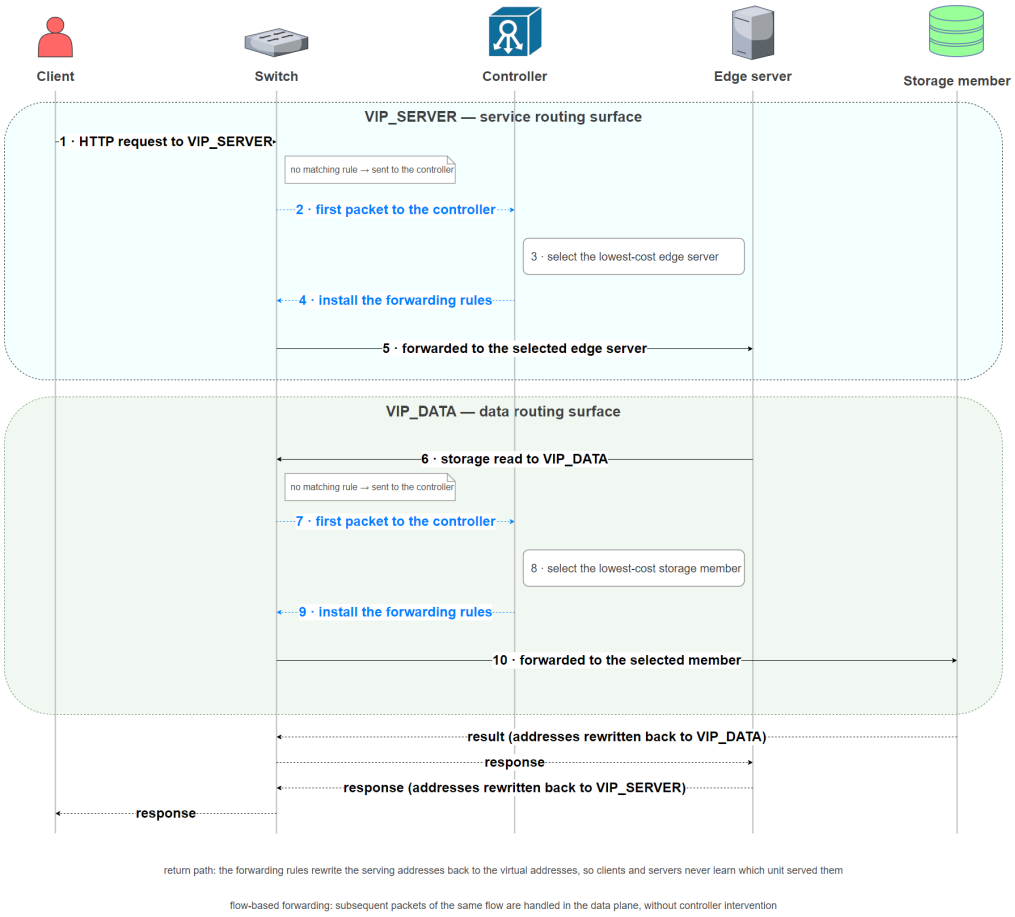


FIGURE 3.2. Double-VIP routing sequence: a client request addressed to `VIP_SERVER` triggers load balancing in the controller, and the edge server's storage reads through `VIP_DATA` trigger load balancing across replica-set members.

The load-balancing policy also defines two behaviours that accompany the cost function: a warm-lease priority, which gives a newly admitted backend bounded priority for new connections during a configured window, and a slow-start ramp, in which a newly discovered backend is held back by a cost penalty that decays over the same window. Both are configuration choices of the routing path.

A request's path decomposes into two intervals. The first is the *pre-dispatch wait*: the time between the client issuing the request and the request's dispatch in the serving process, covering the setup of its connection through the routing path and any wait ahead of the serving process. The second is the *edge service time*: the interval from request dispatch to response finalisation in the serving process, which itself decomposes into a *data-access* component (the storage operations issued for the request, including connection acquisition) and an *edge-processing* component (scheduling, lease and cache work, and response serialisation). Their sum is the request's *user-visible latency*, the client-side round trip. The two edge-service components are the tier-specific latency signals the scaling policy observes, data access for the storage tier and processing for the compute tier (Section 3.3.1). The pre-dispatch wait is observable only as a difference and carries no separate instrumentation.

3.2.3. Control and Telemetry Paths

Edge servers and storage components emit resource and request observations to their local telemetry aggregator, and each aggregator forms summaries for the controller. These summaries inform both load balancing and elasticity decisions, while topology exchange maintains awareness of peer-domain resources. The aggregation-to-controller link is the configurable telemetry interface, shown in Figure 3.3.

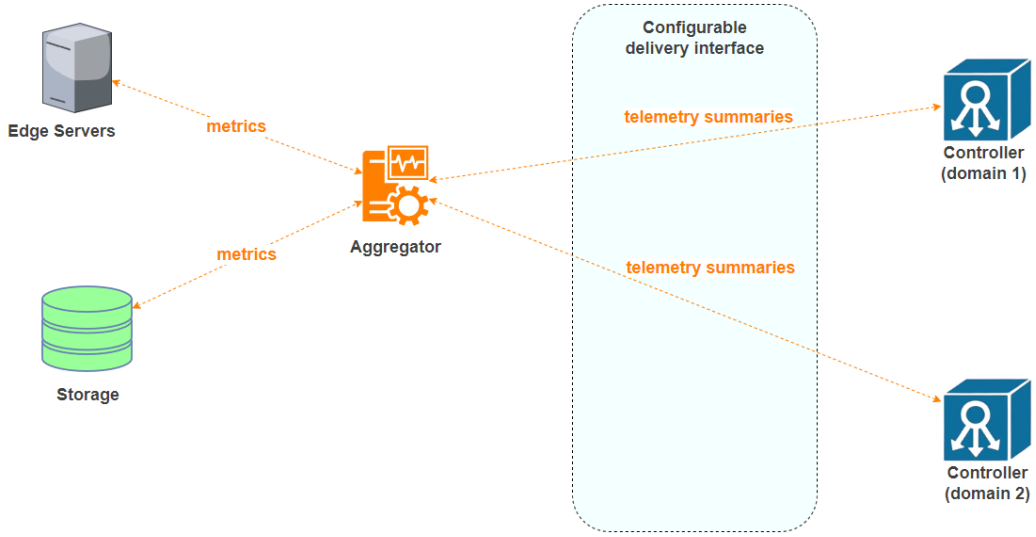


FIGURE 3.3. Control and telemetry paths: edge servers and storage members send observations to the per-domain telemetry aggregator, whose summaries reach both peer controllers through a configurable telemetry interface.

The architecture therefore separates the request data path, the telemetry observation path, and the slow infrastructure-management path while retaining their coordination within each domain's controller process. The implementation of these paths is described in Chapter 4.

3.3. Distributed Elastic Allocation of Resources

The platform scales capacity along two dimensions: compute replicas (also referred to as backends in the routing context), which serve requests, and storage members, which hold the service state. Each dimension has its own observation signals, its own action, and its own admission and release procedure, so that pressure on the two tiers is addressed separately. The two dimensions are summarised in Figure 3.4 and described in turn.

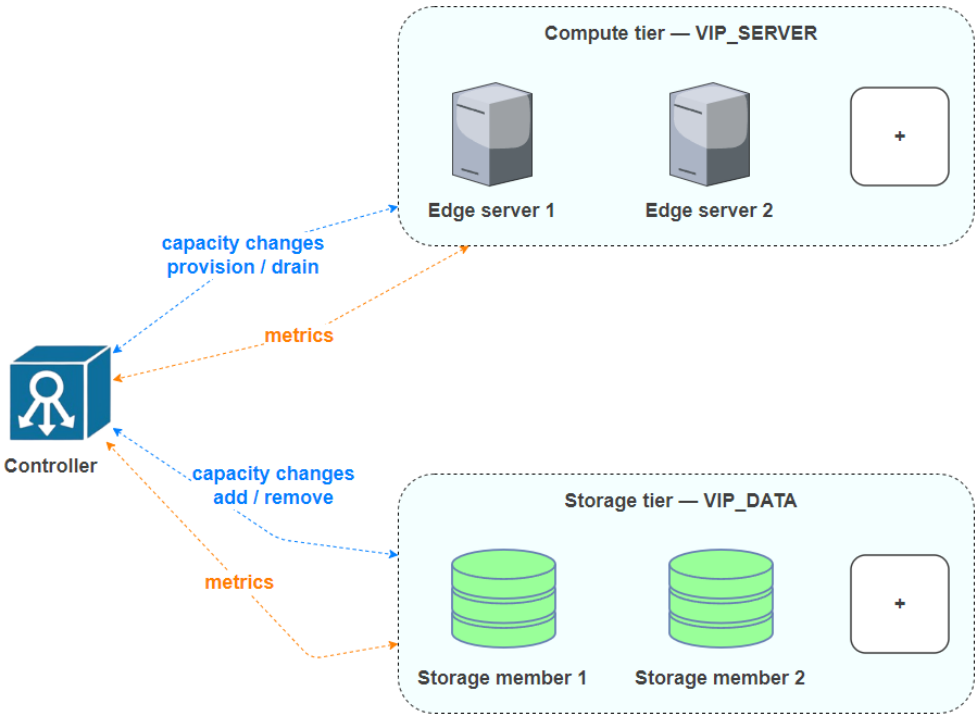


FIGURE 3.4. Elastic allocation of resources: each tier is scaled on its own observation signals, with compute replicas provisioned and drained and storage members added and removed through replica-set membership, both actions issued by the controller over the infrastructure-management path.

3.3.1. Scaling Policy

The policy is applied per resource tier, with tier-specific signals and parameters. Each tier is observed through two signals: a utilisation signal, which in this platform is the CPU utilisation of the tier's units, and a latency signal. For each summary w, let $c(w)$ denote the summary's utilisation and $t(w)$ its typical latency (the tier's latency statistic over the summary, configurable as the mean or the median). Each signal is first mapped onto the unit interval through two calibration parameters, a floor and a span. A value at or below its floor contributes nothing, a value at or above the floor plus its span contributes the maximum, and intermediate values contribute proportionally. The floor separates normal operation from pressure, and the span sets how far the signal must rise before its contribution saturates. Because both signals are mapped onto this bounded scale, they contribute on common terms despite different units, and a single extreme summary cannot dominate the score. The two normalised components are combined with weights w_{util} and w_{lat} into the degradation score

$$\begin{aligned} s(w) &= w_{\text{util}}\,\hat{c}(w) + w_{\text{lat}}\,\hat{t}(w), \\ \hat{c}(w) &= \min\left(1,\ \max\left(0,\ \frac{c(w)-c_0}{c_\Delta}\right)\right), \\ \hat{t}(w) &= \min\left(1,\ \max\left(0,\ \frac{t(w)-t_0}{t_\Delta}\right)\right), \end{aligned} \tag{3.1}$$

with the per-tier weights configured to sum to one, so the score lies between zero and one, where:

- $c(w)$ – CPU utilisation of the scored tier observed in summary w;
- $t(w)$ – typical latency of the scored tier observed in summary w;
- c_0, t_0 – signal floors: at or below these values a signal is considered normal for the tier;
- c_Δ, t_Δ – signal spans: the rise above the floor at which a signal's contribution saturates;
- $\hat{c}(w)$, $\hat{t}(w)$ – normalised components of summary w, each in $[0,1]$;
- w_{util}, w_{lat} – configurable weights of the utilisation and latency components.

The floors, spans, weights, and threshold parameters are configured independently for each tier, so the policy can be calibrated to each tier without changing the structure of the score. For the compute tier the signals are CPU utilisation and request-processing latency, while for the storage tier they are storage-member CPU utilisation and data-access latency. These signals feed only the scaling decisions, while load balancing uses the reported resource state described in Section 3.2.2.

A summary counts as degraded when its score reaches an adaptive threshold, and a scale-up requires sustained degradation across recent summaries. Let $\mathscr{W}_n$ be the set of the last W summaries in which the tier was eligible to act, and R the required number of degraded summaries; the scale-up condition is

$$\begin{aligned} |\{\, w \in \mathscr{W}_n : s(w) \geq \theta(k) \,\}| \geq R, \\ \theta(k) = \min\left(\theta_{\text{base}} + k\,\Delta\theta + \beta_{\text{peer}},\ \theta_{\max}\right), \end{aligned} \tag{3.2}$$

where:

- $\mathscr{W}_n$ – sliding set of the most recent W summaries in which the tier was eligible to act;
- W – number of eligible summaries in the scale-up window;
- R – minimum number of degraded summaries required to request a scale-up;
- k – number of dynamic units currently in the tier's pool (units provisioned by the controller beyond the initial pool);
- $\theta(k)$ – adaptive degradation threshold with k dynamic units in the pool;
- θ_{base} – base threshold;
- $\Delta\theta$ – threshold increment per additional dynamic unit, applied linearly for the compute tier; for the storage tier the increment diminishes with each additional unit, down to a configured minimum;
- $\theta_{\max}$ – upper bound of the adaptive threshold;
- β_{peer} – peer relief added to the compute-tier threshold while the peer domain is healthy enough to absorb spillover traffic (zero otherwise).

The threshold follows the number of dynamic units in the tier's pool. Each additional unit therefore faces a higher bar, and the threshold falls back as units are removed. A scale-up cooldown after each committed scale-up and a scale-down cooldown keyed on the most recent scale-up keep the pace of actions bounded, together with a per-tier cap on the number of units. The set advances only when the tier was eligible to act, and it is cleared when a scale-up occurs. A committed scale-up also clears the tier's scale-down window, so a recent scale-up is not immediately reversed.

A summary counts as idle when both signals fall below their idle thresholds,

$$c(w) < \tau_c \quad \text{and} \quad t(w) < \tau_t, \tag{3.3}$$

where:

- τ_c, τ_t – idle thresholds of the utilisation and latency signals.

A scale-down is requested only when idleness is sustained across $R_\downarrow$ of the last $W_\downarrow$ summaries, where $R_\downarrow$ is the required number of idle summaries and $W_\downarrow$ the length of the scale-down window. This window is longer than the scale-up one, which prevents a transient data-bound stall from being misread as idle capacity. Summaries whose latency exceeds a ceiling are treated as indeterminate and do not count toward idleness.

The policy evaluates each tier per summary, in a fixed order:

(1) the summary is skipped when the tier is ineligible: an infrastructure operation is in progress, the tier is at its cap, or a scale-up cooldown is active;
(2) the degradation score $s(w)$ is computed from the two normalised signals (3.1);
(3) if at least R of the last W eligible summaries reach the adaptive threshold $\theta(k)$ (3.2), the tier requests a scale-up and the summary history is cleared;
(4) otherwise, if both signals fall below their idle thresholds (3.3), idleness is tracked, and the tier requests a scale-down once idleness spans $R_\downarrow$ of the last $W_\downarrow$ summaries.

The values these parameters take in the evaluation are reported in Chapter 4.

3.3.2. Compute

Compute capacity is a per-domain pool of containerised edge servers, each capable of serving any request addressed to the domain's virtual service address. Storage reads performed by a compute replica are routed through the domain's virtual data address, or served from a Tier 1 copy where one is held (the data-placement tiers are defined in Section 3.3.3), and its writes are issued directly to the replica-set primary. Reads are left to the routing path. Compute scale-up applies the policy defined in Section 3.3.1.

A newly provisioned compute replica is recorded through an explicit registration step and becomes eligible for load balancing only once it is visible in the routing pool. How readiness reaches that pool is configurable, and four admission modes are evaluated. Backend notification admits on the message the backend sends, without a check. Notification with a fallback check admits on the same message, and a check after a configured wait recovers a message that never arrives. Periodic polling ignores the message and checks the backend on a fixed interval. Notification with a confirmation check lets the message wake a check, so that it opens the check but does not admit on its own. Because the modes that admit on the message trust it, a message that arrives before the replica can serve would reach the routing pool unchecked, while the modes that check re-derive readiness before admitting. The modes are measured under three message conditions, each held for the whole run: the message arrives intact, never arrives, or arrives before the backend can serve. Because load balancing steers new flows while installed flows remain where they were placed (Section 3.5), admission also marks the point at which the new

backend can begin to share the load. Per-backend resource quotas are configuration parameters of the compute tier.

Compute scale-down removes the least disruptive eligible replica, applying the idle condition defined in Section 3.3.1. Only replicas provisioned by the platform are eligible for removal, so the original nodes are left undisturbed. A candidate is considered only while its reports are recent, and among the eligible replicas the one carrying the least request load is removed first, with the lowest utilisation and request-processing latency used as tie-breakers. Removal proceeds in two phases: the replica is first withdrawn from the routing pool and signalled to finish its in-flight requests, and only after it drains is it released. A pending drain can be cancelled if load rebounds before the replica exits, favouring fast recovery over aggressive reclamation.

3.3.3. Storage

The replica set is the unit in which storage capacity is scaled. The storage tier of each domain runs as a replica set whose primary serves the clients' writes and whose secondaries replicate the data and absorb the remainder of the traffic. Reads may therefore be served by any replica-set member, and the consistency contract is primary writes with eventually consistent secondary reads. The evaluation does not rely on read-after-write semantics. Storage scale-up is a membership change. The controller provisions a new storage member, which joins the set as a non-voting secondary, and admits it to the data path only once the member can serve reads. Preparation is decoupled from the reaction: a readied member is held in reserve outside the routing pool, and the storage scale-up is the activation of this prepared reserve, which removes the initial readiness wait from the reaction path. A new reserve is prepared immediately afterwards.

Storage scale-down removes an idle member while the remaining members keep serving. Only members added by the platform are eligible for removal, so the set's original membership is left undisturbed, and the most recently added eligible member is removed first. The decision applies the idle condition defined in Section 3.3.1 to the storage tier's idle signals (its data-access latency and, in the configurations that enable it, its storage utilisation).

Beyond the membership change, the platform defines three data-placement tiers: Tier 0 provides direct remote reads over the virtual data path, meaning the replica-set member is deployed outside its primary network domain, Tier 1 holds a partial copy of the data in the consumer domain, and Tier 2 is the replica-set extension described above. Tier 0 and Tier 1 are platform capabilities rather than evaluated treatments. The evaluated storage action is limited to activating the prepared reserve or removing a member of the domain's own replica set.

3.4. Telemetry, Load Balancing, and Scaling

The producers and the aggregator that collects their reports are described in Section 3.2.3. Each edge server reports after every request it answers, carrying its own resource state and the experience of that request, and each storage member reports as it serves storage requests, so a busy node reports often and a quiet one is kept visible by its periodic heartbeat. The aggregator produces a summary of the most recent period at a fixed interval. Every summary, including those covering idle periods, is evaluated and recorded. Each summary also carries the time of

each node's last report, so a node that stays silent beyond an absence bound is treated as gone instead of being acted upon.

The controller consumes these summaries through an explicit, configurable interface. Four telemetry modes supply the same summaries in different forms, and they are the two defects of Section 2.3 separately and together: push delivers every summary as it is produced, complete and prompt; delayed push delivers every summary after a fixed delay, complete and late; sampled push delivers one summary in three as it is produced, partial and prompt; and polling returns only the most recent summary at a fixed read interval, partial and late. The modes are four configurations of one interface rather than two transports opposed to each other, and the read interval alone places polling in its cell: summaries that close between two reads are never fetched, and the read is not aligned with when the summaries close, so the summary it returns may have closed up to one read interval before the read.

Because the decision engine evaluates delivered summaries only, the mode sets the earliest moment a scaling reaction can begin: a decision cannot be taken before its triggering evidence arrives. The condition counts the last summaries the engine has seen, so missing summaries slow how often it is met as well as when it first can be, and the mode therefore shapes how much capacity is added during a rise. The modes let the effect of late or missing telemetry be studied while the aggregation, the workload, and the decision logic remain unchanged, so any difference in how the controller scales is attributed to the evidence it received rather than to the mechanism that carried it.

Inside the controller, the summaries feed two consumers. The first is load balancing, whose candidate state is updated from the summaries. The second is the decision engine, which evaluates each summary with the scaling policy of Section 3.3.1 and emits a typed, prioritised action request once a tier's condition is sustained. Requests that add or preserve resources take precedence over requests that remove capacity. Among the additions, a storage scale-up precedes a compute scale-up, and among the removals, a storage scale-down precedes a compute scale-down. A request's type identifies its resource tier (compute or storage) and its action (scale-up or scale-down), and the evidence behind each request is recorded in the decision log for the summary that produced it. Figure 3.5 details the per-summary evaluation, from the eligibility check to the request emissions.

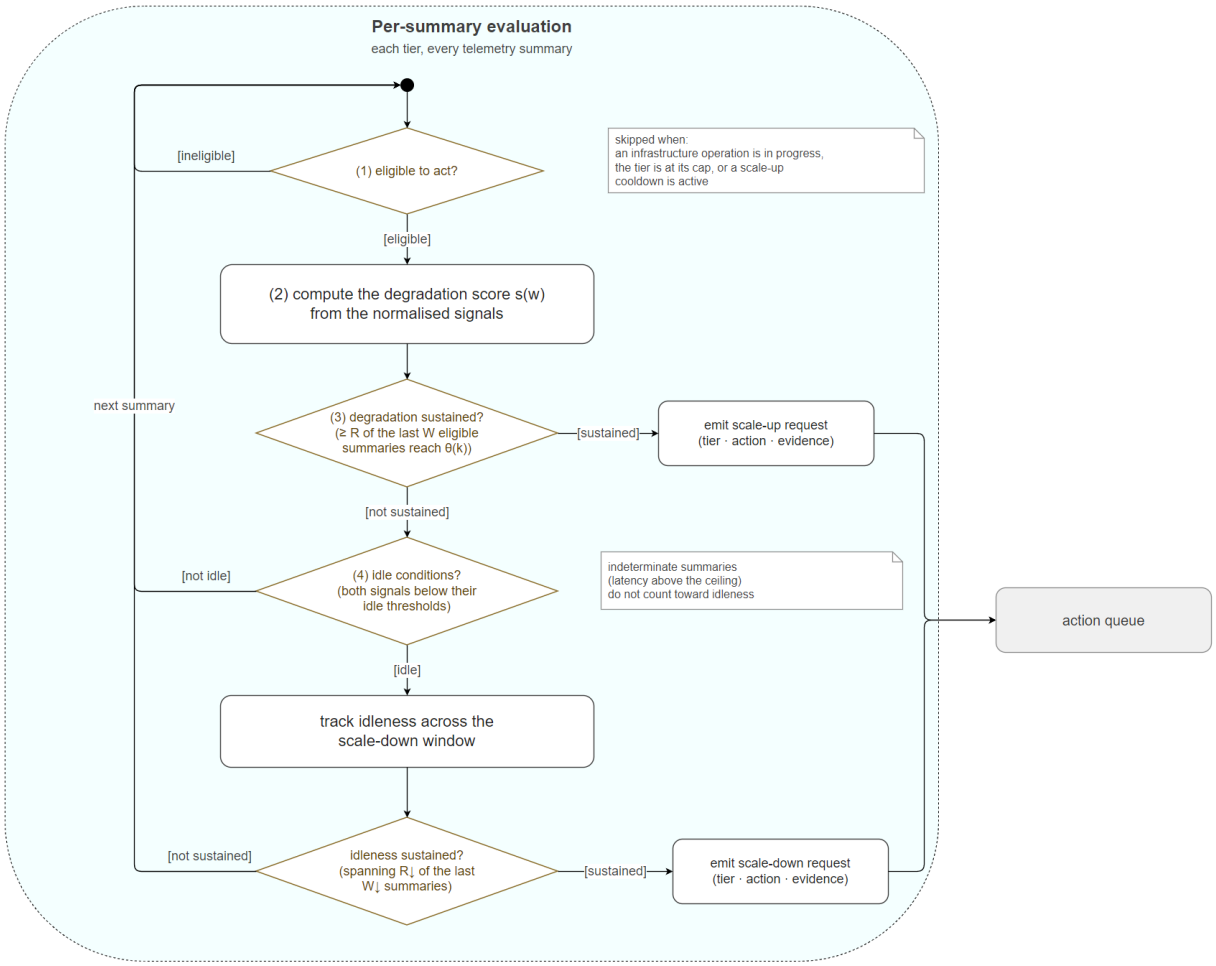


FIGURE 3.5. The per-summary evaluation, in the fixed order of Section 3.3.1: the eligibility check, then the degradation score, then the scale-up condition, and otherwise the idle condition, with every remaining branch returning to the next summary.

The decision engine may also be configured to commit to a single scaling action (compute or storage), which is the scaling interface of the demand-to-capacity chain. In the bottleneck-aware configuration, that choice distinguishes between compute-bound and data-access-bound demand and scales the matching tier when it leads by the configured classification margin, resolving close scores towards storage. A per-tier action budget can bound its actions, and the action taken and the actions rejected are recorded as evidence. The platform's capacity events, such as the completion of a drain or the readiness of a storage member, reach the same decision path, so that admission and release follow the same flow as the actions themselves.

3.5. Control Workflow

The control workflow separates three execution contexts that share one process but do not block one another. The packet context handles flow decisions and does not wait on slow work. The observation context evaluates delivered summaries and emits action requests. The action context performs infrastructure changes such as provisioning, drain, and removal.

The workflow traverses the demand-to-capacity chain introduced in Section 1.1. A change in demand alters the request experience reported by the compute replicas, and the summary

that contains the change is produced and delivered. The decision engine evaluates the delivered summary. If the tier condition is sustained, the engine emits a scaling action request, which the action context executes. The provisioned unit reaches its readiness state and is admitted to the corresponding routing pool, which for the storage tier is the activation of the prepared reserve. New flows can then be directed to it, while flows already installed on other members remain where they were placed. The chain completes when the new unit serves its first successful request. Each step is instrumented, so the timeline from demand observation to usable capacity is recorded end to end, from registration and routing-pool inclusion to the first successful request (Figure 3.6). When readiness admission is enabled, a compute replica that does not reach readiness within the configured bound, or that is never admitted within the abandonment bound, is abandoned and torn down instead of being admitted. Without readiness admission, compute replicas are registered as soon as they are provisioned and no readiness bound applies, so a replica that never becomes usable is reclaimed only by the ordinary capacity checks.

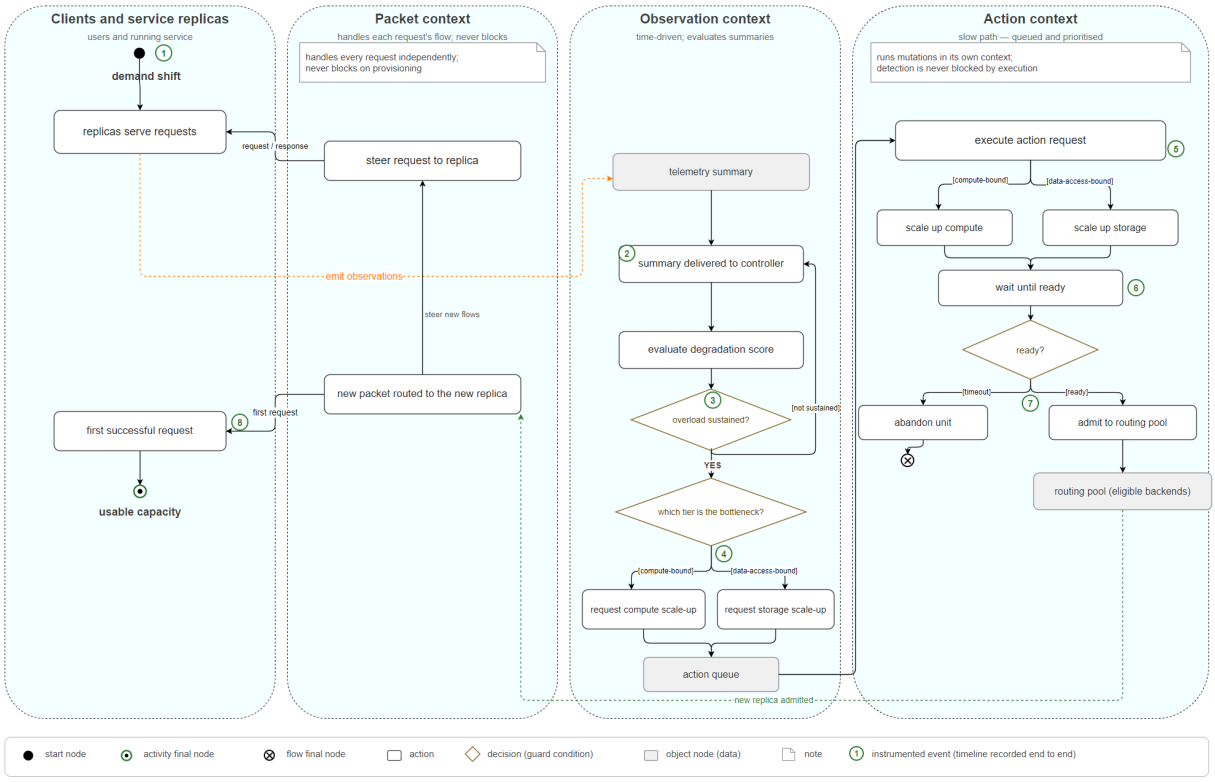


FIGURE 3.6. The control workflow, drawn as a UML diagram. The demand-to-capacity chain runs across three contexts in the same process: the packet context, the observation context, and the action context. The contexts do not block one another and they exchange state through shared structures (the telemetry state, the action queue, the capacity state, and the routing pools). The numbered stages match the chain in Figure 1.2 and show the provisioning direction. Capacity removal follows the same steps in reverse, starting with the drain. The aggregator is omitted between the observations sent by the replicas and the summary delivered to the controller.

State crosses these contexts mostly through shared structures rather than through direct calls. The observation context writes the telemetry state and the action requests, and the action context

owns the infrastructure changes and records the state of each in-flight capacity change. The routing pools are updated with the capacity transitions: a unit enters them when it is admitted and leaves them on removal, while a readied member held in reserve remains outside them until activation. The packet context reads both on each flow decision. Because the packet context is not blocked by the action context, a long-running change cannot stall request steering. Because the action requests are queued and prioritised, detection is not blocked by execution.

Every request that adds or preserves capacity precedes every request that removes capacity. The cancellation of a pending drain precedes its execution. This execution order is distinct from the scaling decision, which the observation context decides before the request is queued.

Across domains, the peer controllers exchange topology state so that each controller is aware of the peer domain's resources, while acting on its own domain's demand. The concrete realisation of this design, the technologies that implement each context, and the validation of the resulting platform are the subject of Chapter 4. The experimental characterisation of the three interfaces is presented in Chapter 5.

CHAPTER 4

Implementation

Chapter 3 specified what the platform must do. This chapter describes what was built. Section 4.1 presents the testbed and its load source, Section 4.2 the two scaled resources as implemented, Section 4.3 the observation and decision components, Section 4.4 the runtime behaviour of the controller, and Section 4.5 the checks that validate the resulting platform. Configuration values named in the text are listed per campaign in Appendix A.

4.1. Experimental Infrastructure

The platform runs on a single cloud VM host that holds both domains as Docker containers on emulated networks. The host is a KVM virtual machine running Ubuntu 22.04.5 LTS, with 4 vCPU on an AMD EPYC-Genoa processor at 2.4 GHz, 8 GB of memory, and local SSD storage. Table 4.1 lists the components as deployed.

TABLE 4.1. Platform components as deployed.

Component	Implementation	Role
Domain network	Open vSwitch bridge per domain	Carries one domain's client, service, and data traffic
SDN controller (one per domain)	OS-Ken application (Python)	VIP interception, load balancing, flow installation, and the observation and action contexts
Telemetry aggregator (one per domain)	Python process on the ZeroMQ transport	Collects producer records and produces summaries
Edge server	Flask application on the Werkzeug server	Serves the workload, reports per-request telemetry, and exposes the readiness and drain endpoints
Storage member	`mongod` with a companion sidecar	Holds one replica-set member and reports replication and resource state
Inter-domain router	Linux container with traffic shaping	Joins the two domains and applies the emulated link profile
Workload driver	Python harness on the host	Issues the scheduled open-loop load from client namespaces

The two domains are joined by a router container whose inter-domain link carries an emulated, fixed symmetric round-trip time applied with tc-netem (185 ms in the reported campaigns, with the jitter, loss, and rate knobs available and off by default). The evaluated data path is single-site. The emulated link carries no evaluated traffic, and every control-plane link is host-local.

The data plane runs OpenFlow on Open vSwitch. The platform's steering needs are elementary and standard: match ordinary addresses and ports, rewrite each connection to its backend, and deliver new connections to the controller, which answers from the same Python process that runs the control logic (DR2). OpenFlow covers this set natively, and it reaches the host's connection tracking, so per-connection identity and address translation live in the kernel rather than in the rules. A programmable data plane such as P4 could express the same steering, but its additional capabilities (custom parsers, in-switch state, and in-band telemetry) serve network-level questions that this thesis does not ask, because every decision it evaluates is taken in the controller.

Each domain has two virtual service addresses, one for client requests and one for storage reads. The data plane combines OpenFlow rules with the kernel's connection tracking. On the service path, the controller installs a DNAT/SNAT rule pair for the client and the selected backend. With flow isolation enabled the pair is scoped to a single connection and removed after its response, so every measured request triggers a fresh load-balancing decision. On the data path, a forward rule selects the member and commits a NAT mapping to it, and the kernel records the connection by its five-tuple, the pair of addresses and ports plus the protocol. Reply packets are handed back to connection tracking, which reverses the mapping, so the identity of each connection lives in the kernel's table rather than in the rule. Two consequences matter for the platform. Established connections survive rule deletion, which lets the controller bulk-delete a member's forward rules when the member leaves the pool and lets new connections re-select while established ones finish. And because the kernel keeps connections apart by their full five-tuple, the rule can be scoped to the client or, as in the reported configuration, to a single connection of it, so a pooled client spreads its connections across members. Each domain tracks its connections in its own zone, and rules that belong to a domain share a cookie so they can be removed together.

The load is exerted by a scheduled driver that runs on the host beside the testbed. Each client is a Linux network namespace attached to its domain's client network, and the driver issues requests from a namespace as HTTP calls against the domain's virtual service address, so the controller observes the namespace's source address and steers each request through the data path above. A run instantiates 24 clients per domain, 48 in total. The driver executes the run's phase schedule (Section 5.1): each phase fixes its duration, a per-client issuing rate, the fraction of clients that are active, and its request mix. Active clients issue requests continuously at the phase rate against users and content items taken from a snapshot of the seeded data set, so traffic generation does not depend on a live database, and the run's seed fixes which clients are active and the per-client request streams.

The delivery of the load is open-loop, so a client keeps issuing its requests on schedule while earlier requests are still outstanding and the offered demand is preserved when the service slows down. Three client-side bounds make the resulting failure modes explicit and finite. Each client keeps at most 1024 requests in flight, and a request that arrives when the window is full is recorded as dropped. Every request carries a 300 s client-side cap, and a request that reaches it is

recorded as a timeout. In-flight requests that outlast the 30 s phase-end drain are abandoned and recorded as cancelled. Each issued request is written to the run's request record with its phase, kind, target identifiers, issue time, outcome class, and latency, and the record, together with the phase snapshot captured per run, is the raw material of the per-run analyses of Chapter 5.

4.2. Distributed Elastic Allocation of Resources

4.2.1. Compute

A compute replica is a Flask application served by the Werkzeug server. It exposes the workload routes, a drain endpoint that moves the replica into a draining state, and a readiness endpoint. The readiness predicate is a real service round-trip, and the endpoint answers 200 only while a MongoDB ping against the domain's virtual data address succeeds. The server binds before readiness is declared, so the predicate means servability and the readiness message cannot precede the socket. After each request the replica pushes one telemetry record: the request's outcome and timing (its total time, and the MongoDB time split into reads and writes) and its own CPU and memory use, read from the container's cgroup files so the numbers describe the container rather than the host. With readiness messages enabled it also emits an `app_ready` control event when the predicate first succeeds.

The controller provisions a replica by starting its container with the tier's quotas and the run's environment, attaching it to the domain network through an Open vSwitch port, and recording it as a backend of the domain. Removal follows the two phases of Section 3.3. The drain completes once the replica's in-flight requests reach zero and a quiet period passes. Only then is its network attachment removed.

Readiness admission is implemented by a component that holds newly spawned replicas in a pending registry and admits them to the routing pool only when its configured rule is satisfied. The rule is selected by a propagation mode.

- `off`. The replica is admitted as soon as it is provisioned, and no readiness mode applies.
- Backend notification (`direct`, the fallback wait beyond the abandonment bound). The replica is admitted on the `app_ready` message itself, with no check before admission, and nothing re-checks it.
- Notification with a fallback check (`direct`, a 20 s fallback wait). The replica is admitted on the message, and a check after the wait recovers a message that never arrives.
- Periodic polling (`discovery`). The message is ignored and the controller checks the readiness endpoint at the fixed polling interval, admitting on the first check that passes.
- Notification with a confirmation check. The message wakes a check and admission requires the check to pass, and the fallback wait applies when no message arrives (`READINESS_WAKE_VERIFY_MODE=probe`).

Checks retry at a fixed interval with their own timeout. A replica that is not admitted within the abandonment bound is abandoned and fully torn down. Every admission records its source in the admission log, the message, a poll, a fallback check, or a confirmation check. A check that

disagrees with an admission made on the message is recorded as a warning and the admission stands, because the message already admitted the replica. The confirmation check never admits unverified. The admission decision is summarised in Algorithm 4.1.

Algorithm 4.1 Readiness admission of a spawned compute replica.

enqueue the replica in the pending registry
if mode is `off` **then**
    admit the replica
else if mode is `discovery` **then**
    check `/ready` on each poll and admit on the first success
else ▷ `direct`
    admit the replica on the `app_ready` message
    without a message after the fallback wait: check and admit on success
    with the confirmation check: the message wakes a check, and admission into the routing backend pool requires it to pass
end if
abandon and tear down a replica not admitted within the abandonment bound

Warm-lease priority and slow-start ramps are separate load-balancing behaviours of the routing path (Section 3.2.2) and are held constant in the readiness experiments.

4.2.2. Storage

The storage tier is built on MongoDB replica sets, chosen because DR3 requires storage capacity to change at run time from the platform's own code, while the service keeps serving. MongoDB meets this requirement with the database's own membership commands, issued by the platform's code, and each member is one container running one database process. The alternatives considered add a management component or require hand-built orchestration. A MySQL InnoDB Cluster changes membership through the cluster API of MySQL Shell, a separate management component the platform must invoke [46, 47]. A PostgreSQL streaming-replication standby has no built-in membership operation at all and must be provisioned and configured by hand [48].

Each domain's storage is a MongoDB replica set of its own, built in three steps. At network build time, the domain's storage container starts as the set's single member and becomes its primary, and the run's data set is seeded through it. At runtime, a new member starts as a bare `mongod` with a companion sidecar. The sidecar connects to a known member of the set, appends itself as a non-voting member with zero priority so it cannot affect elections, and applies the change with a single `replSetReconfig` call. When the member reaches the secondary state and can serve reads, the sidecar emits the `rs_secondary_ready` control event, and the controller admits the member into the domain's virtual data address, by default on the event, with a telemetry-observed secondary state as the fallback path. While the member serves, the sidecar polls it and reports a sample whenever the member's operation counters show activity: the

sample carries the member's replication lag, its open connections, and its CPU and memory use read from the container's cgroup files. An idle member is kept visible by the periodic heartbeat. Removal reverses the steps. The member leaves the routing pool first, and the controller then performs `rs.remove()` against the primary, waits for the removal to be confirmed, and tears the container down. The routing surface is the domain's virtual data address, and the edge server's database driver never learns the physical membership.

When the persistent reserve is enabled, the activated member is already joined and servable, and its replacement is prepared once the slot is free. The member lifecycle is bounded by the run's caps (Appendix A).

Reads are issued against the domain's virtual data address through per-request leases that the edge server manages per domain epoch. When the address's membership changes, the epoch rotates and pooled connections are retried with backoff, and reads prefer a secondary. Writes are issued directly to the replica-set primary. Selective synchronisation (Tier 1), which holds a partial copy of the data in the consumer domain, and cross-domain storage placement, which places a replica-set member in the peer domain, are capabilities of Section 3.3.3 and are disabled in the reported experiments.

4.3. Telemetry, Load Balancing, and Scaling

Each edge server reports after every request it answers, and each storage member reports as it serves storage operations, so the telemetry path is not flooded with idle traffic. The domain's aggregator collects these reports and produces a summary of the preceding 10 s every 10 s. A heartbeat every 60 s keeps a quiet but alive node visible, and a starting node is visible before its first request.

The configurable interface of Section 3.4 is implemented as alternative sources over the same summaries, so the four modes differ only in how the same summaries reach the controller. Every summary is appended to a durable, sequence-numbered log that the replaying modes read. Table 4.2 lists the four modes and their implemented sources.

TABLE 4.2. The four telemetry modes and their implemented sources.

Mode	Source	Implementation
push	`event_preserving`	ordered replay of the durable log, with each summary acknowledged
delayed push	`delayed_event_preserving`	the same replay released when each summary closes, plus a fixed 30 s delay, without burst replay
polling	`poll`	the controller retrieves only the most recent summary, every 30 s, so intermediate summaries are not delivered
sampled push	`sampled_push`	every third summary delivered as it is produced, with the intermediate ones dropped

The platform's default source pushes each summary as it is produced, without a delivery guarantee; the campaign's push mode uses the acknowledged replay, so the campaign can verify that every produced summary arrived. The replaying sources record every summary they observe in a delivery log with its network, sequence number, close time, delivery time, delay, and source mode, and this record is what the telemetry experiments of Chapter 5 read. Two kinds of message stay outside the telemetry modes. Control events are the platform's own notifications, forwarded by the aggregator immediately, without waiting for the next summary: a new replica reporting that it can serve (`app_ready`), a replica reporting that its in-flight requests have finished and it is shutting down (`drain_complete`), a storage member reporting that it is ready to serve reads (`rs_secondary_ready`), and, where per-request flow isolation is enabled, each completed workload request (`request_complete`), which releases that request's flow so the next request is routed afresh.

On the measurement side, a collector writes per-run resource records for each domain, debug records, and per-node files, and samples the CPU and memory of both controller instances. The controller's own records complete the timing evidence: the lifecycle of each provisioned unit, the elasticity events, the admission log of the readiness gate, and the decision log of the action gate, all kept per run beside the request record of Section 4.1 and the delivery log above. Inside the controller, the delivered summaries feed the two consumers of Section 3.4, and every decision is written to the decision log with its evidence. In the bottleneck-aware configuration, the action gate commits at most one action per summary and records the bottleneck class it assigned, the per-tier evidence, the action taken and the actions rejected, and the budget counters. Absence detection and scale-down evaluation run on a fixed ticker once per delivered summary rather than when it arrives. The configuration surface is environment-driven, and its values are listed in Appendix A.

4.4. Control Workflow

Each domain runs its own controller instance. The three execution contexts of Section 3.5 are implemented as one cooperative event loop and two native threads. The packet path handles flow decisions inside the OpenFlow event loop and never blocks on infrastructure work. The telemetry consumer runs on the same event loop: it receives the summaries, updates the per-backend and per-member state, evaluates the tier conditions, and posts typed alerts to a priority queue. The elasticity thread is a daemon thread that blocks on that queue and performs the infrastructure changes, and the readiness gate runs its own worker thread for checks.

State crosses the contexts through the shared structures of Section 3.5. The telemetry consumer writes the per-node state and the alerts between its yield points, the elasticity thread hands its results to the routing pools through the same registration the packet path reads, and the packet path reads the pools on each flow decision.

On the first packet of a new connection, no flow rule matches and the packet is sent to the controller. The controller selects a unit with the cost function of Section 3.2.2, installs the rules described in Section 4.1, and sends the packet on. Subsequent packets follow the installed rules. The rules carry a configurable idle timeout, which resets as the connection stays active, and

a configurable hard timeout, after which the rule expires and the next packet triggers a fresh load-balancing decision. The evaluated values are listed per campaign in Appendix A. When a unit leaves a pool, its rules are removed. Algorithm 4.2 summarises the first-packet path.

Algorithm 4.2 First-packet handling for a new connection.

1: the first packet of a new connection finds no flow rule
2: send the packet to the controller
3: select a unit from the eligible pool with the cost function
4: install the routing rule for the client and the unit (Section 4.1)
5: send the packet on, and later packets follow the rule

A request waits ahead of the edge service in three places, which the instrumentation separates. The first is the connection setup through the virtual service address. The second is the first packet's round trip to the controller and the installation of its rules. The third is the queue on the selected replica before the application reads the request. The first two are part of the steering path described here, and the third accrues on the serving side. The waiting time is read as a difference, user-visible latency minus the reported edge service time.

The controller keeps its state in memory. The routing pools, the per-node telemetry, and the lifecycle records of in-flight operations are not written durably, so a restart begins from an empty state and re-derives what it needs from the network: the topology worker re-observes the switches and rebuilds the hop cache and the routing pools, the telemetry resumes and repopulates the per-node state, and units without fresh reports receive the configured default in load balancing. On the switch connection, the stale flows are flushed and the catch-all rule is reinstalled, so later connections are selected again. Because a restart resets this state, the evaluated runs exclude mid-run controller restarts.

4.5. Implementation Validation

Before any experiment campaign, the implementation was validated in three ways. The locked configuration was exercised end to end and checked for stability, each mechanism was exercised and its effect reproduced, and the telemetry path was checked against the mode it implements.

TABLE 4.3. Implementation checks and their outcomes.

Check	Condition	Outcome
Integrated stability	Identical runs of the locked configuration, with all mechanisms enabled	Failure rate within the envelope (0.18 % and 0.41 % against a 3 % gate), with no tracebacks
Variability	Identical replicates after the scale-down cooldown fix	Compute-phase variability removed, with 0.23 % overall in the verification run
Data-path routing	Stale-rule failures during member removal	Compute-phase failures fall from 56–65 % to 1.4 %, with no connection-epoch rotations
Storage reserve	Activation of the prepared reserve	The activated reserve carries data-path traffic, and liveness is stable
Selective synchronisation	Bidirectional hotspot activation	Data-access latency falls from 84.5 ms to 3.58 ms, with a clean drain in both directions
Telemetry path	Summary continuity and per-mode fidelity	Every produced summary delivered or recorded as an aged-out gap, completeness and measured delay match the configured mode, and overhead stays bounded across modes

Correctness is read from this behaviour rather than from code inspection. The mechanisms were exercised under the canonical workload, and their effects were reproduced across replicate runs. The telemetry-path checks verify the modes directly. The summary log records every produced summary, and a summary that is never delivered is recorded as an aged-out gap rather than silently lost. Completeness, sequence order, and delivery delay are logged per summary and match the configured mode, and the aggregator's overhead and per-summary payload stay bounded and constant across modes. The readiness path is additionally exercised with the fault instrumentation of the fault-robustness experiments, where a replica can be configured to report readiness before it can serve, to answer workload requests with errors while the readiness predicate stays satisfied, or to lose its readiness message entirely. The instrumentation and its values are listed in Appendix A. The empirical characterisation of the three interfaces is presented in Chapter 5.

CHAPTER 5

Experimental Results

5.1. Experimental Setup

This section defines the conditions under which the three research questions' experimental evaluations were executed: the testbed and its instrumentation, the demand model and the workload of each campaign, the measured outcomes, and the design of each campaign.

5.1.1. Testbed and Instrumentation

The evaluation was performed on the platform of Chapters 3 and 4, deployed as a two-domain testbed on a single host. Each domain runs its own data plane, compute replicas, storage tier, telemetry aggregator, and SDN controller, and the two domains are joined by an emulated inter-domain link. Both domains are driven by the same scheduled, open-loop workload driver, described more extensively in Section 4.1 as a driver that applies the workload's request schedule regardless of the responses it receives, so that the demand offered to the service is independent of the capacity available to serve it.

Table 5.1 summarises the configuration common to the three campaigns; the per-campaign resource caps, quotas, and parameter values are listed in Appendix A.

TABLE 5.1. Evaluation testbed configuration common to the three campaigns.

Component	Configuration
Host	both domains on a single KVM-based cloud virtual machine running Ubuntu 22.04.5 LTS, with 4 vCPU on an AMD EPYC-Genoa processor at 2.4 GHz, 8 GB of memory, and local SSD storage; all components run as CPU-capped containers
Inter-domain link (emulated)	fixed symmetric round-trip latency of 185 ms (tc-netem)
Data plane	one Open vSwitch per domain, programmed over OpenFlow
Controllers	one OS-Ken controller per domain; traffic steering, telemetry consumption, and elasticity decisions run in the controller process
Edge tier	Docker compute replicas per domain; a fixed set of servers extended by dynamic units during scale-up
Storage tier	one MongoDB replica set per domain; members added and removed through native replication
Telemetry	one aggregator per domain; summaries of the preceding 10 s, produced every 10 s
Workload driver	open-loop scheduled driver: 24 clients per domain (48 in total), a bounded in-flight window of 1024 requests, a client-side request cap of 300 s, and a 30 s phase-end drain

The testbed runs the two domains on a single host, which keeps the control paths host-local. The three campaigns ran on hosts of the same type, identical apart from the local disk, which is smaller on the host of the telemetry campaign (80 GB against 160 GB) because no instance with

the same disk size was available when that host was provisioned. Telemetry and controller-to-controller topology exchange do not traverse the inter-domain link, which carries cross-domain data traffic only. The evaluation workloads reference locally owned content only, so storage reads remain within the replica set of the domain that serves each request, and cross-domain storage placement is disabled. Load balancing may place a request on a peer-domain replica at a hop-cost penalty. This occurred in about 2 % of requests with a recorded serving backend in the telemetry and scaling action campaigns and 0.5 % in the readiness admission campaign, and it is a property of the platform's load-balancing policy rather than a treatment. The campaigns therefore measure the coordination interfaces under controlled and repeatable conditions.

Each run is an independent experimental unit and starts from the same set of nodes and configuration.

Before each run, the data set is generated from a fixed data seed and loaded into the domain's replica-set primary: 3000 content items and 100 user profiles per domain, with the collection indexes created by the same setup. The setup also exports a snapshot of the seeded items and profiles, and the driver draws its request targets from that snapshot, so the offered demand is fixed by the run's seed and does not depend on the live database. Between runs, the persistent containers, the data state, and the controller state are reset.

During a run, the platform records the timeline of the demand-to-capacity chain end to end: demand episode onset, telemetry summary completion and controller observation, scaling decision and action taken, provisioning start and completion, application-readiness confirmation, admission to the routing pool, first routed request, and first successful response. The load distribution of the served demand across the replicas is also recorded. The records behind these timestamps are described in Section 4.3, and every timing reported in this chapter is read from them. The rest of this section defines the workloads that drive these timelines, the outcomes extracted from them, and the design under which each campaign varied exactly one interface.

5.1.2. Demand Model and Campaign Workloads

The demand model varies two axes: the request intensity over time, and the request mix, which determines whether an episode binds the compute tier or the storage tier. The spatial axis, that is, which domain owns the requested content, is held constant in the evaluation, as described above.

The request mix is a weighted distribution over five request kinds defined in Table 5.2.

TABLE 5.2. The five request kinds that make up the workload mixes, and the tier each loads.

Request kind	What it does	Tier loaded
Content lookup	Reads a single content item (a point read).	Storage
Feed ranking	Performs a filtered item retrieval of candidate content, then ranks the candidates in the edge server.	Compute
Service pressure	Introspects the edge server's recent request activity.	Compute
Content update	Writes an update to a content item.	Storage
Content aggregation	Computes a collection-level aggregation over content items.	Storage

A workload is a timed sequence of demand phases. Each phase fixes, for its duration, the offered request intensity, as a per-client request rate and an active-client fraction, and the request mix. The phase that applies the demand shift under test is referred to as the demand episode. Each campaign defined its own workload, calibrated and locked before the campaign started, and every run stores the phase schedule it executed.

The phases around the episode have fixed roles across the campaigns. A baseline window opens every run at a low, steady load (rate 1.0 per client, 10 % of clients active) and establishes the healthy pre-shift operating point. A recovery gap is a quiet phase with a low issuing load (rate 0.5, 5 % of clients) and acts as a post-episode quiet period that lets the platform settle and separates the stressed window from the post-shift windows. A demand drop returns the load to a low, steady level (rate 1.0, 10 %) and is the release window, in which the capacity the episode required is scaled down and post-shift service quality is compared with the baseline. An idle tail closes the run at a near-idle load (rate 0.05, 5 %), so that every configuration reaches sustained idleness and the release of surplus capacity can complete. Table 5.3 summarises the workload of each campaign.

TABLE 5.3. Campaign workloads. Each campaign defines its own phase sequence, run in the order shown. The demand episode is the phase that applies the demand shift under test.

Campaign	Phase sequence	Demand episode
Telemetry Modes (RQ1)	Baseline (60 s) → load rise (180 s) → recovery gap (120 s) → demand drop (420 s) → idle tail (420 s)	Rate 1.2 per client, all clients active; mixed load: 30 % service pressure, 35 % content lookups, 15 % feed ranking, 10 % each of updates and aggregations
Bottleneck-Aware Scaling Action (RQ2)	Baseline (30 or 60 s) → one of two episodes → recovery gap (60 or 120 s) → demand drop (360 or 420 s)	Compute-bound: 600 s at rate 1.5 per client, pure service pressure. Data-access-bound: 480 s at rate 5.0 per client, 90 % content lookups
Readiness Admission Modes under Three Message Conditions (RQ3)	Baseline (60 s) → compute-bound plateau (600 s) → recovery gap (120 s) → demand drop (420 s) → idle tail (420 s)	Rate 2.0 per client, pure service pressure; per-backend CPU quota 0.12

5.1.3. Metrics and Outcome Definitions

For the telemetry campaign, the primary outcome is usable-capacity latency: the time from the onset of the load rise to the first new capacity serving traffic, reconstructed from the recorded timeline. The time to the first scaling decision is reported as a descriptive outcome only, because the telemetry mode determines when the evidence that triggers a decision reaches the controller, and a decision cannot be taken before that evidence arrives.

For the scaling action campaign, the outcomes are read per arm, each arm being one scaling policy under one demand regime: the targeted-tier relief, as the change in the receiving tier's utilisation and latency between the pre-registered windows before and after the action, and

the service recovery, as the episode service quality against the no-action comparator, with the resource use reported alongside. Saturation, as this campaign evaluates it, is the capacity-bound state in which a tier runs at its capacity limit, arriving requests queue and completion slows, rather than the state of an overloaded server whose own processing slows.

The scaling action results use the request-path latency quantities defined in Section 3.2.2: user-visible latency, edge service time, its data-access and edge-processing components, and the pre-dispatch wait. The edge service components are aggregated as per-request means over the demand episode. The readiness admission campaign reports the segment of the chain from the scaling decision to usable capacity, which isolates the admission path.

Service quality is summarised per run through percentiles (p50, p95) of the endpoint latency over the demand episode. Latency percentiles are computed on the served basis, over completed requests. Outcome classes are reported distinctly as offered requests, completed requests, failed requests, timeouts (each recorded at the client-side cap of 300 s), and client-side cancellations and drops at phase end. Completion is reported on the served basis, completed requests over non-abandoned requests, and on the offered basis, completed requests over offered requests. Unless stated otherwise, each run contributes the mean of its two domains, and any cross-domain asymmetry above the pre-registered symmetry screen is reported.

The outcomes each campaign reports follow the facets of its research question.

- *The telemetry campaign* answers its question in three parts. The first is what the controller saw: the delivered fraction, how much of the produced telemetry reached it, and the information age at decision, how old the summary behind a decision was when the decision was taken. The second is how much capacity the controller added and when the new capacity started serving traffic: the number of compute scale-ups per run, and the usable-capacity latency. The third is what users experienced: the episode latency percentiles. Both latency measures are defined at the start of this subsection.
- *The scaling action campaign* answers its question along its three axes, per arm: targeted-tier relief, as the relief obtained in the tier that received the action; service recovery, as the episode service quality against the no-action comparator; and resource use, as the scaling actions taken and the compute and storage node-minutes of the units those actions provision. A node-minute is one provisioned unit counted for each minute it exists, so the totals characterise how much capacity-time a policy keeps in service beyond the nodes that are always running, not how busy that capacity is. One unit kept for thirty minutes and three units kept for ten minutes both total thirty node-minutes. The totals are normalised per thousand recorded requests. The load spread across the replicas of the receiving tier, as each replica's share of the served demand and its utilisation, reports the mechanism through which the action executed.
- *The readiness admission campaign* answers its question in three parts. When the new backend serves is analysed along the admission chain: the intervals from verified readiness to admission, from admission to the first routed request, and from provisioning to the first successful response, and the relief of the existing nodes once the new backend

begins serving. The cost of a lost message is measured in the slow-share, the fraction of served requests that time out or exceed one second, over the full plateau (the campaign's demand episode), the onset window (the first 150 s of the plateau), and the plateau tail (the last 300 s of the plateau), and in the number of admissions, the candidates abandoned and torn down. The cost of a premature message is read as the change in the same user-visible measure against a matched no-fault run. A backend counts as admitted when it becomes visible in the routing pool, not when its registration is recorded.

Control overhead is reported as the CPU utilisation and resident memory of the controller processes, and as the volume of control records, along each campaign.

5.1.4. Campaign Designs and Analysis

All three campaigns follow the same discipline of single-variable manipulation. Each campaign varies exactly one interface of the demand-to-capacity chain and holds the other two constant, so that the measured differences are attributable to the interface under test. The policies compared within a campaign are baselines that encode architectural properties, such as a fixed single-tier scaling policy or a periodic polling mechanism, and not competing aspects. Each campaign was pre-registered with the treatments, primary outcomes, screening rules, and analysis tests all fixed before data collection.

No results are pooled across campaigns. Every contrast is computed within its own campaign's design, and where a research question draws on more than one campaign the campaigns are reported separately.

Each campaign organises its runs so that every planned contrast is measured over matched demand. A seed fixes the demand of a run: which clients are active in each phase and each client's request stream, targets, payloads, and timing jitter. The runs are organised into blocks, a block holding one run of every treatment the campaign compares, sharing one seed, so the treatments entering a contrast meet identical demand; the execution order within a block is counterbalanced across blocks, so testbed drift cannot align with a single treatment. Each campaign fixes its seed range before data collection, and no seed is reused within a campaign.

The number of replicates per campaign was fixed before data collection so that a fully separated contrast remains resolvable under the pre-registered test. Under complete separation between two treatment sets of n runs, the exact Mann–Whitney U test has a p-value floor of $2/\binom{2n}{n}$: 0.0022 at six per treatment and 0.0006 at seven. The exact test shows that six and seven runs per arm are enough to discount run-to-run variation as the source of the observed difference. The campaigns use these sizes: six per arm in the scaling action and readiness admission campaigns, with the readiness-admission relief comparison drawn from a separate set at seven runs per arm, and seven per mode in the telemetry campaign. That campaign takes an extra replicate because its runs showed a real run-to-run spread even when repeated under the same configuration and seed.

The reported contrasts follow the tests fixed before data collection. Two campaigns use the exact Mann–Whitney U test: it ranks the runs of the two treatments together and reports the

probability of a separation at least as strong as the observed one when the treatments are in fact equivalent. The exact form enumerates every way the observed values could be split, which six or seven runs per arm keep feasible. Alongside each p-value the campaigns report Cliff's δ, the probability that a run of one treatment exceeds a run of the other minus the reverse probability, because significance alone does not show how separated the runs are. Values near 0 mean the treatments overlap; at -1 or 1 every run of one treatment lies below or above every run of the other, a complete separation that further replicates can only preserve or weaken. Reported intervals are 95 % intervals, the pre-registered level.

Each campaign uses these tests and intervals on its own measures. The telemetry campaign reports each δ with a 95 % bootstrap confidence interval, and a direction count, the number of seed-matched blocks consistent with the predicted order. The scaling action campaign works with ratios instead of groups: for each arm it reports the median ratio across runs, with a 95 % confidence interval, and compares it with the pre-registered threshold, because its question is whether a ratio passes a target, not whether two sets of runs differ. The readiness admission campaign reports how long each new backend takes from verified readiness to admission, compared within the backend's measured start-up class (fast- or slow-binding): on this testbed the socket bind intermittently stalls for several seconds when a backend starts during an active run, a shared platform cost recorded per backend, random across backends and equal across the compared modes, which the within-class comparison removes. It reports its event-loss comparison as differences in the slow share, in percentage points, within each matched block, and its premature-message comparison as per-mode damage against a matched no-fault run. It reports no confidence intervals, because its pre-registered checks were thresholds and agreement across blocks. No correction is made for testing several contrasts at once, because each campaign fixed a small number of named contrasts in advance.

Screening rules are applied before the contrasts as runs entering a contrast must pass the campaign's per-run checks on data-path integrity, test-harness behaviour, and delivery fidelity; cross-domain symmetry is screened, with breaches above the screening line reported as flags rather than exclusions, and the defined exclusions are removed and reported. Pre-registered criteria that are not met are reported as such, together with the observed evidence and its documented causes. The full design of each campaign, its treatments, held constants, workload, replication, exclusions, and pre-registered test, is summarised in a table at the start of its results section. The concrete values of the parameters held constant are listed per campaign in Appendix A.

5.2. Telemetry Modes (RQ1)

The campaign asked how the four telemetry modes change the way the controller scales the service when load rises. Its answer is reported in the three parts of Section 5.1.3: what the controller saw, how much capacity it added and when that capacity started serving traffic, and what users experienced during the rise. The modes were compared over the same load rise (Section 5.1). Push (A) is the reference: every summary reaches the controller as it closes. Delayed push (B) delivers every summary, 30 s late. Sampled push (D) delivers one summary

in three, on time. Polling (C) delivers the same one-in-three share as sampled push, but the summary it returns had already closed when the read began, so its evidence arrives old as well as sparse (Section 3.4). Relative to the reference, one property changes at a time: delay without missing summaries (B), missing summaries without delay (D), and both together (C), so the cost of late telemetry is separated from the cost of missing telemetry. The pre-registered expectation was the ordering $A < B < C \approx D$ on both usable capacity and the episode latency p95.

The summary period, the scaling policy, the routing policy, and the workload were held constant across modes. Each mode was run seven times, once per seed-matched block (seeds 3001–3007, 28 runs in total), so the four modes are re-tested under a different demand realisation in each block; each run is one experimental unit. Table 5.4 summarises the design.

TABLE 5.4. Design of the telemetry campaign (RQ1).

Item	Design
Treatment	Four telemetry modes over the otherwise identical platform: push (A); delayed push, the same summaries with a fixed 30 s lag (B); polling every 30 s (C); and sampled push, every third summary (D).
Held constant	Summary period (10 s), scaling and routing policy, workload, topology, resource limits.
Workload	Baseline (60 s) $\rightarrow$ load rise (180 s, rate 1.2 per client, mixed load) $\rightarrow$ recovery gap (120 s) $\rightarrow$ demand drop (420 s) $\rightarrow$ idle tail (420 s).
Replication	Seven seed-matched blocks, one run per mode per block (28 runs, seeds 3001–3007), counterbalanced order.
Exclusions	None; all 28 runs passed the platform checks.
Test	Four pre-registered contrasts on the episode latency p95, defined in Section 5.1.4: delay (A–B, D–C) and missing evidence (A–D, B–C).
Expectations	Ordering $A < B < C \approx D$ on usable capacity and the episode latency p95; sampled push expected strictly worst; quality outside the load rise unaffected.

Under push and delayed push the controller received every completed summary, with the information age at decision lowest under push and highest under delayed push; under polling and sampled push it received about one third of them (Table 5.5). The same load rise therefore reached the controller through four different views: every summary as it closed, the same stream one fixed delay behind, whatever summary was on hand at each read, and one summary in three.

TABLE 5.5. Telemetry realisation and what the controller saw about the load rise (mean of the seven replicates, observed range in parentheses): share of telemetry summaries delivered, and information age at decision, the time from the closing of the summary that a decision used to the decision itself.

Mode	Delivered (%)	Information age (s)
push (A)	100.0	3.5 (1.2–7.5)
delayed push (B)	100.0	33.3 (30.6–38.4)
polling (C)	32.5–33.6	16.2 (13.8–19.8)
sampled push (D)	32.8–33.6	11.7 (5.3–15.9)

Every mode scaled up both tiers, and what differed was when the new capacity became usable. First new capacity serving traffic was 28.5 s after the onset in A, 59.6 s in B, 79.6 s in C and 83.2 s in D (Table 5.6), so polling and sampled push took nearly three times as long as push, while delayed push landed about 31 s after push, close to its fixed 30 s lag. A $<$ B and B $<$ C held in all seven blocks; C and D were not cleanly separated (D later by 3.6 s at the median, C $<$ D in five of seven blocks). The time to the first scale-up decision is descriptive only, because the telemetry mode determines when the triggering evidence arrives. The number of actions is descriptive in the same way: polling and sampled push ran 2 to 4 compute scale-ups per run, against 6, the whole dynamic pool of three units per domain, in push and delayed push. The trigger needs several qualifying summaries in a row, and a stream carrying one summary in three accumulates them more slowly, so the lossy modes added less as well as later.

TABLE 5.6. Telemetry modes: usable capacity, episode latency, and outcome classes (medians with min–max ranges over the seven replicates; delivered share as the observed range; no timeouts in the episode in any mode). Usable capacity is the time from episode onset to first new capacity serving traffic; the episode p95 is the served-basis endpoint latency p95 (per-run mean of the two domains).

Mode	Delivered (%)	Usable capacity (s)	Episode p95 (s)	Timeouts	Cancelled (%)
push (A)	100.0	28.5 (25.1–32.9)	3.7 (2.8–5.2)	0	5.6
delayed push (B)	100.0	59.6 (55.1–61.5)	10.8 (5.0–65.3)	0	6.5
polling (C)	32.5–33.6	79.6 (73.7–81.8)	35.0 (27.0–39.6)	0	5.4
sampled push (D)	32.8–33.6	83.2 (75.1–88.1)	34.5 (31.4–37.6)	0	6.1

The user-visible cost followed the scaling time (Table 5.7, Figure 5.1). With about two thirds of the summaries absent, the episode endpoint p95 reached 35.0 s in C and 34.5 s in D against 3.7 s in A, roughly a ninefold penalty, with complete separation ($\delta = -1.000$, exact $p = 0.0006$). Delay degraded the outcome in aggregate, to 10.8 s against 3.7 s ($\delta = -0.959$, $p = 0.0012$), but not uniformly: two of the seven delayed push runs collapsed to the level of polling and sampled push (65.3 s and 64.7 s), while the other five stayed at 5.0–12.6 s.

TABLE 5.7. Pre-registered primary contrasts on the episode endpoint latency p95 (served basis, per-run mean of the two domains). Exact Mann–Whitney p-values at $n = 7$ per mode. The effect size, its interval, and the direction count are defined in Section 5.1.4. The direction count gives how many blocks (of seven) followed the predicted order.

Contrast	Medians (s)	Exact p	Cliff's δ [95 % CI]	Direction
Delay A–B	3.73 vs 10.77	0.0012	-0.959 [-1.000, -0.755]	7/7
Delay D–C	34.50 vs 35.04	0.456	-0.265 [-0.837, 0.388]	2/7
Loss A–D	3.73 vs 34.50	0.0006	-1.000 [-1.000, -1.000]	7/7
Loss B–C	10.77 vs 35.04	0.209	-0.429 [-1.000, 0.429]	5/7

Note. Direction counts the blocks consistent with each contrast's predicted order. The delay contrasts (A–B, D–C) compare evidence that differs in lateness alone, with the delivered share unchanged; the loss contrasts (A–D, B–C) compare evidence that differs in how much arrives, prompt in the first and delayed in the second. The D $>$ C prediction reflects the pre-campaign flag that sampled push could be strictly worst; its low count (2/7) is the deviation reported in the text.

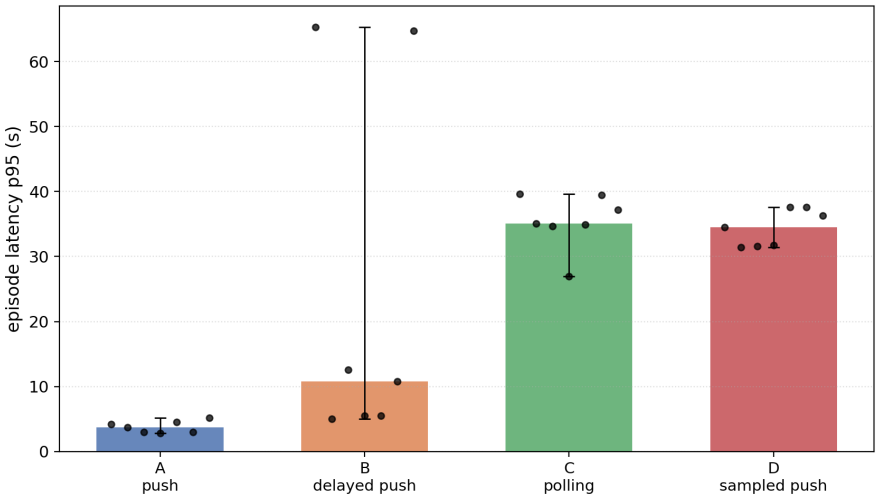


FIGURE 5.1. Episode endpoint latency p95 by telemetry mode (served basis; per-run mean of the two domains; seven replicates per mode). Bars show the median across replicates, whiskers the min–max range, and dots the individual runs; the two highest delayed push points are the replicates where the spare capacity of the nodes already running was consumed.

The spread of delayed push comes from the platform state that each run samples, not from the decision engine. Across all seven runs the first scale-up decision fell between 52.0 s and 60.4 s and usable capacity between 55.1 s and 61.5 s. What decides the outcome is whether the spare capacity of the nodes already running, their absorption margin, covers the time they absorb the load alone: 25–33 s in A, 55–62 s in B, 74–82 s in C and 75–88 s in D. A slow run can exhaust that margin: across runs of identical configuration the episode's median compute processing time varied sixfold (16–94 ms). A's absorption window was covered in every run, so the variance left no trace; C's and D's were exhausted in all seven; delayed push sat at the boundary, covered in five runs and exhausted in the two that sampled a slow state.

Two pre-registered expectations did not hold, and both are informative. The full ordering broke because the two collapsed delayed push runs landed at C's level, so B and C do not separate (exact $p = 0.209$), and sampled push was not strictly worst, since C and D are statistically indistinguishable ($p = 0.456$). What the campaign measured instead is two faults of different kinds: missing summaries act robustly in every replicate, while delay acts conditionally on the margin it consumes. One delayed push run showed a cross-domain asymmetry above the campaign's screening line (domain 2 p95 21.1 s against 4.0 s in domain 1); it was the only run above the line.

The differences are confined to the load rise. Outside it, the four modes were indistinguishable, with baseline, recovery and demand-drop p95 at about 1.05–1.08 s, no timeouts inside the episode, and cancellation and drop shares flat at roughly 5–7 %.

In sum, on the evaluated testbed, the telemetry mode decided how much capacity the controller added, when that capacity started serving traffic, and what users experienced while the load rise was absorbed. Missing summaries drove the user-visible cost in every replicate, while

delay cost in proportion to the absorption margin it consumed. Delay and loss are different faults, not two severities of one.

5.3. Bottleneck-Aware Scaling Action (RQ2)

The campaign compared three scaling-action policies under the same demand shifts: a fixed compute-only policy, a fixed storage-only policy, and bottleneck-aware selection, each evaluated on a compute-bound episode (600 s of pure service pressure) and a data-access-bound episode (480 s, 90 % content lookups). Each episode is named after the tier its load presses on: the compute tier in the first, the storage member in the second. Whether the named tier actually becomes capacity-bound (Section 5.1.3) is itself one of the results. The six-arm campaign contributes six replicates per arm (five seed-matched and one cross-seed; 34 of 36 runs valid, with two runs excluded as documented container memory-cap incidents), and a comparative extension contributes the element the original design lacked: no-action comparators (a policy that scales nothing), against which the consequence of any action can be measured. Comparisons that draw on both campaigns carry labels recording the memory cap and software build of each side.

Each policy stands for an operator choice about which tier the action serves: scale only the tier that runs the requests, scale only the tier that stores and serves the data, or scale the tier the episode presses on. That tier is the bottleneck and bottleneck-aware selection reads the per-tier telemetry, classifies the episode as compute-bound or data-access-bound, and scales the matching tier, while each fixed policy acts on its single tier alone.

The pre-registered expectation was that the aligned action would relieve the tier that bound the episode and improve the user-visible service against inaction and that the mismatched action would harm service relative to inaction.

Table 5.8 summarises the design of the original campaign.

TABLE 5.8. Design of the scaling action campaign (RQ2).

Item	Design
Treatment	Three scaling policies (compute-only, storage-only, and bottleneck-aware selection) each under two demand regimes (compute-bound and data-access-bound), giving six arms, plus no-action comparators (a policy that scales nothing) contributed by the comparative extension, six runs per episode, against which the benefit and harm ratios are computed. The fixed policies scale their own tier only, while bottleneck-aware selection classifies the episode from the tier telemetry and scales the matching tier. Compute scale-up adds a compute replica; storage scale-up activates a prepared reserve in the same domain.
Held constant	Telemetry mode, routing policy, per-tier resource caps, action budget, cooldowns, backend-admission condition; Tier 1 selective synchronisation and cross-domain storage placement disabled.
Workload	Compute-bound episode (600 s, rate 1.5 per client, pure service pressure) or data-access-bound episode (480 s, rate 5.0 per client, 90 % content lookups); each with baseline, recovery, and demand-drop phases.
Experimental unit	Run; per-arm criterion over replicates.
Replication	Six arms and six replicates in six counterbalanced blocks (36 runs; seeds 42 and 43; five replicates on the primary seed and one cross-seed replicate per arm); 34 valid.
Exclusions	Two runs excluded as documented container memory-cap incidents (kills under the 256 MB and 512 MB caps); per-run screens as above.
Test	Pre-registered arm-level criterion: median of the replicate p95 ratios between the pre-activation window (from the episode onset to the reserve activation, 30–60 s of completed requests) and the post-activation window (from 120 s after the added member is ready to the episode end, over 300 s), with a 95 % CI excluding 1.0, computed on the seed-matched replicates (five per arm, or fewer where an exclusion applies); the cross-seed replicate reported separately; tier-relief and resource-use outcomes alongside.
Expectations	The aligned action is expected to relieve the tier that binds the episode and to benefit user-visible service over inaction; the mismatched action is expected to bring no relief to the binding tier and to harm service. The aligned action's value over inaction on the compute axis is carried as a descriptive measure. Each expectation is assessed against a pre-registered coefficient, reported with its outcome.

The relief check asked whether the aligned action relieved the tier that bound the episode (Figure 5.2). Where the storage tier bound the episode, it did: the prepared reserve was activated in every run where the storage action was selected (11 of 11) and shared the reads, and the storage-member peak CPU fell to 0.57–0.66 of its pre-activation level in all twelve storage-only domains. The bottleneck-aware arm followed the same direction with eight of its ten domains below the threshold. On the compute-bound episode no relief is reported because the compute tier never entered the capacity-bound state, so there was no queue to relieve. The action did spread the load across the added backends, with no user-visible change (Figure 5.5).

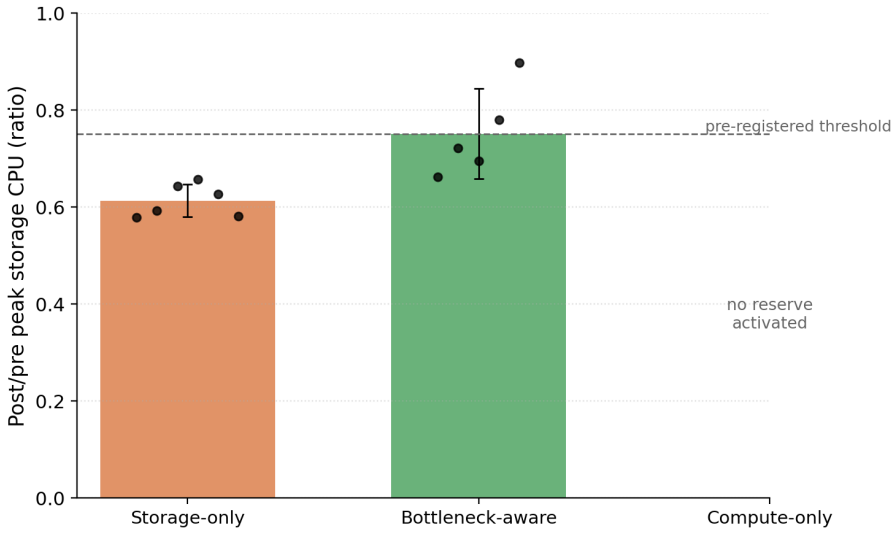


FIGURE 5.2. Targeted-tier relief where the storage tier bound the episode: post-to-pre peak storage-CPU ratio per replicate (mean of the two domains) for the data-access-bound arms, against the pre-registered 0.75 threshold; bars show the mean across replicates with the standard deviation, and dots the individual replicates. The storage-only arm stays below the threshold in every replicate, the bottleneck-aware arm partially overlaps it, and the compute-only arm activated no reserve.

The service outcome is regime-concentrated (Figures 5.3 and 5.4). On the data-access-bound episode the tier-matched storage action delivered a large user-visible benefit over inaction as the episode endpoint latency p95 was about 0.6 s against about 1.6 s under no action, a ratio of about 0.37 against the pre-registered benefit threshold of 0.85 (secondary: p50 about 60 ms against about 280 ms). The bottleneck-aware arm also beat no action on the same episode but by less (about 0.67 against the 0.37 of the fixed storage arm). Its own second-tier action is the documented cause: after storage relief it adds compute as well, and in two of five replicates a compute add coincided with a tail spike.

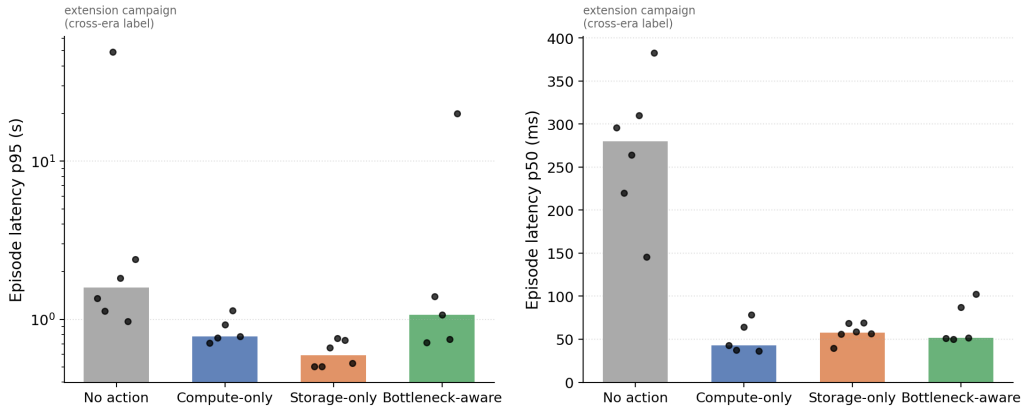


FIGURE 5.3. Episode endpoint latency of the data-access-bound episode at the p95 (left, logarithmic axis) and the median (right) for the compute-only, storage-only and bottleneck-aware arms and the no-action comparator. The matched storage action takes the tail and the mismatched compute action keeps the lowest median while leaving a higher tail. Bars show per-arm medians across replicates and dots the individual runs; the no-action comparator is drawn from the extension campaign and carries its cross-era label.

On the compute-bound episode no policy differed from no action in service as every arm served at a p50 of about 3 ms. No recovery is attributable to the compute action, as the onset latency is flat (about 3 ms). The load is the same in every arm and stays below the tier's capacity, so the late-episode tail that appears in the scaling arms is not due to the load but to scaling the compute tier mid-episode. A later pre-registered escalation found no regime in which the compute deficit was both capacity-bound and measurable, so the compute-bound episode exercises the compute tier without ever producing a service deficit for an action to recover.

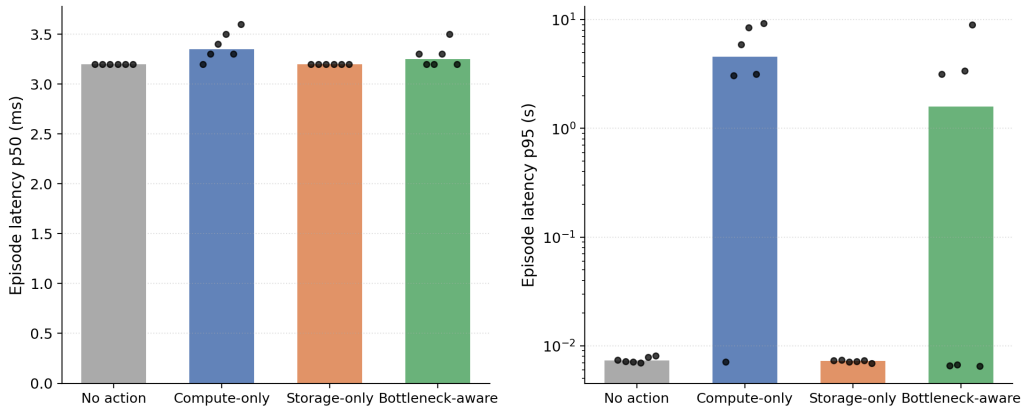


FIGURE 5.4. Episode endpoint latency of the compute-bound episode at the median (left) and the p95 (right, logarithmic axis) for the compute-only, storage-only and bottleneck-aware arms and the no-action comparator, all six replicates per arm on the same build. The median is the null reading: every arm serves at about 3 ms. At the p95 the onset latency is flat (about 3 ms) and a late-episode tail appears only in the arms that scale up the compute tier (medians of about 4.6 s in the compute-only arm and 1.6 s in the bottleneck-aware arm), absent in the storage-only arm and the no-action comparator (about 7 ms). Bars show per-arm medians across replicates and dots the individual runs.

The data-access reading follows from where the queue builds: with no action the serving storage member carries the full read load alone, the matched action moves a share of the reads to the reserve and cuts the data-access component of the wait (about 79 ms to 52 ms), and the mismatched action never reaches the data path. The data-access component and the edge-processing component are the per-tier latency signals the selection observes (Section 3.3.1).

The scaling action on the compute-bound episode was executed in full (Figure 5.5): the added replicas took most of each run's requests (medians of about 71 and 72 % in the compute-only and bottleneck-aware arms), and the total compute spent rose to about four times the single-backend load. The domain's traffic alternated between members rather than loading them at once: in the windows a backend carried traffic it served the full episode rate at about the single-backend CPU (mostly 55–61 % against 61 %), so the all-window medians in the panel (46–56 %) sit below that level only through the idle windows each member also has.

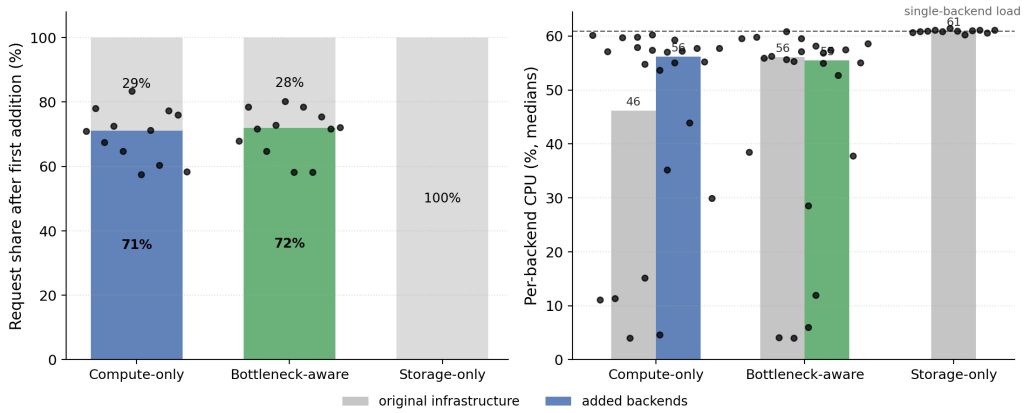


FIGURE 5.5. Execution of the compute action on the compute-bound episode, covering the compute-only and bottleneck-aware arms with the storage-only arm as the no-scaling reference. Left: the share of each domain's requests served by the added backends after the first addition (dots: individual runs and domains), over the grey share served by the original infrastructure. Right: per-backend CPU as a percentage of the backend CPU cap, median over each member's sampled windows (the original infrastructure over the episode, the added backends over their active life), against the storage-only arm's single-backend CPU on the same episode (dashed); each member alternates between serving windows at about that level and near-idle windows. The action executed in full, yet the service outcome (Figure 5.4, left) is unchanged.

Resource use separates the policies where service does not: on the compute-bound episode both compute-scaling arms spent compute without changing the service, and on the data-access-bound episode the storage action's spend became tier relief and user-visible benefit, while the mismatched compute action's spend relieved nothing. In the evaluated envelope, what reached users was decided by the tier a policy served, not by the size of its spend.

Selection commitment holds across regimes. The bottleneck-aware classifier committed to the detected bottleneck in every evaluated regime: on the data-access-bound episode it activated the storage reserve first and scaled compute only after storage relief, and on the compute-bound episode it scaled compute without activating storage. Classifier-versus-episode agreement was 88–93 % on the data-access-bound episode. The compute units the arm adds after storage relief

follow from the condition (edge utilisation rose past its threshold while every request is served through the edge), not from a compute deficit.

The compute-bound episode shows no such mirror, because pure service pressure holds the storage tier near idle, so no sustained storage request arises after the compute action.

Degradation was bounded on both sides. On the data-access-bound episode the mismatched action came in between the matched action and no action (about 0.8 s at the p95 against about 0.6 s and 1.6 s): it helped less than the matched action while still beating inaction, so the pre-registered harm expectation was not met. On the compute-bound episode the cost of acting is the late-episode tail of the scaling arms (about 4.6 s and 1.6 s at the p95, against about 7 ms elsewhere), which forms while a replica is prepared mid-episode.

One pre-registered criterion remained formally unmet: the storage tail-latency test. It asks whether the request tail latency (p95) clearly improved once the reserve was active, comparing the window before the activation with the window after it. The tail did come out lower after the action, about a quarter lower on the median (a ratio of 0.727), but not clearly enough for the test: the interval around that improvement still spans no change, so the criterion counts as unmet. The documented cause is a measurement asymmetry between the two pinned windows: the long post-activation window always contains the sparse completed-request tail above 10 s, which the short pre-activation window rarely does, inflating its p95 independently of the action. The storage reading therefore rests on the tier relief and on the labelled benefit over inaction, and the unmet gate is carried as such. Across the five pre-registered criteria the set closes as follows: storage relief met, benefit over inaction met, harm from the mismatched action not met, compute recovery not attributable to the action, and storage tail latency unmet under its own rule. Within the operating envelope, the only regime in which an action improved user-visible service was the one where the served tier actually bound.

In sum, the results support one conditional conclusion: the aligned action relieved the tier that bound the episode and delivered the labelled user-visible benefit over inaction, and the benefit is absent where no capacity-bound queue forms, however fully the action executes. The selector committed to the detected bottleneck in every evaluated regime, while the campaign reports no add-attributable compute recovery and no demonstrated advantage of bottleneck-aware selection over a matched fixed policy. The absence of that advantage is partly a property of the envelope as the experiment failed to produce a compute-bound episode that was capacity-bound within the tested range, so on that axis no policy, fixed or adaptive, had a service deficit to recover, and selection had no service difference to separate itself with.

The reported effects hold for the evaluated two-domain testbed, its workloads, and its resource limits. The platform behaviours observed at those limits are carried to Chapter 6 as limitations and future work.

5.4. Readiness Admission Modes under Three Message Conditions (RQ3)

This section evaluates readiness admission: when a newly provisioned backend is admitted into the routing pool and starts receiving traffic. The backend sends a message when it can serve, and each mode turns that message into traffic differently. The four modes are compared under three

message conditions, each against its pre-registered expectation, with the readiness criterion, the load-balancing policy, the workload, and the resource limits held fixed within each compared set, so that the mode is the only variable (Table 5.9).

The modes pair the direct approach with a safeguard. Backend notification admits on the message alone and nothing re-checks it, its fallback sitting beyond the abandonment bound. Notification with a fallback check is event-driven with a fallback for a lost message: it waits 20 s for the message and checks the backend if none arrives. Periodic polling ignores the message and checks the backend every 10 s. Notification with a confirmation check treats the message as a prompt: it wakes a check, and admission requires the check to pass. A check passes only once the backend can serve.

The conditions reproduce how a message can behave in practice: it arrives when sent, it never arrives, or it arrives before the backend can serve.

TABLE 5.9. Design of the readiness admission campaign (RQ3).

Item	Design
Treatment	Readiness-admission coordination, compared across four admission modes: backend notification, notification with a fallback check (the control, which recovers a lost message without verifying it), periodic polling, and notification with a confirmation check.
Message conditions	Each condition is held for the whole run. Intact: the message arrives as sent. Lost: every message is dropped. Premature: the message arrives before the backend can serve, at a ten-second lead in the evaluated matrix.
Held constant	Readiness criterion (the same check), load-balancing policy, routing weights, pool state, workload shape, per-backend CPU quota (fixed within each compared set, with per-set quota values listed in Appendix A).
Workload	Admission timing with an intact message; fault comparison on a baseline followed by a 600 s compute-bound plateau (rate 2.0 per client, pure service pressure) under fixed reduced headroom (per-backend CPU quota 0.12); the premature-message runs reuse the same plateau.
Experimental unit	Run; the fault comparison is per block, admission intervals additionally analysed per backend within runs.
Replication	Four admission modes over three message conditions, seven seed-matched blocks per mode (intact and lost at seeds 5201–5207; premature at seeds 6301–6307, each block paired with a matched no-fault run), arm order rotated per block; the admission timing is characterised on separate intact-source campaigns, each with its own fixed workload and quota: a per-backend campaign of seven runs per mechanism, a higher-rate arm of six runs per mechanism at 12.0 requests per second per client, a relief comparison of seven runs per mode, and a poll-interval sensitivity arm of seven runs reported descriptively. Samples are never pooled across these sets.
Exclusions	One block excluded from the fault comparison (no-fault control tail breach); pre-registered health floors on the no-fault arm.
Test	Admission timing: exact Mann–Whitney U on the per-backend admission intervals, stratified by the backend's start-up class (fast- vs slow-binding). Fault comparison: co-primary pre-registered slow-share differences, in percentage points (pp), over the full plateau and the onset window (at least 5 pp per window; medians at least 7 pp), exact Mann–Whitney U of the per-block differences, relief completion on the plateau tail, and boundedness of the backend-notification degradation (slow share at most 25 %). Premature message: per-mode damage in the common user-visible measure against a same-seed no-fault run, at the evaluated ten-second lead.
Expectations	While the message is intact, backend notification admits earlier in every start-up class, the admission interval is bounded by the configured polling period, and the new-backend load and the relief of the existing nodes are unchanged between the modes once admission occurs. When the message is lost under reduced headroom, a decisive slow-share difference for backend notification over the full plateau and the onset window, at least 5 pp per window (medians at least 7 pp), bounded overall (slow share at most 25 %), no redistribution under backend notification, recovery by the modes that re-derive readiness with relief completing on the plateau tail, and the no-fault arm holding the pre-registered health floors. Under a premature message, the modes that admit on it are exposed while the modes that check hold their no-fault level.

While the message is intact, the expectation was that the modes that admit on the message would admit earlier in every start-up class, that the checking mode would wait at most one polling period, and that the modes would not differ once admission occurred. Notification with a fallback check behaves like backend notification while the message arrives, because its fallback only fires when none does, so the priced contrast is between admitting on the message and checking for it. The measured gap is complete, with a median of 0.001 s against 6.984 s from verified readiness to admission ($\delta = -1.000$, earlier in every backend), with the full chain in Table 5.10 and Figure 5.6. The head start reached the user end to end. Provisioning to the first successful response was about six seconds shorter under backend notification (11.27 s against 17.70 s, exact $p = 0.0005$), and relief reached the loaded tier about four seconds sooner (scaling decision to usable capacity 2.17 s against 6.01 s, exact $p = 0.0022$). Once admitted, both modes served the first request in about a second, and the load left the existing nodes by the same amount under both (relief $p = 0.209$): the mode changes when the redistribution begins, not how much it moves. The expectation held. The checking mode pays its polling granularity and nothing more, as the pre-registered poll-interval sensitivity arm shows (six runs at a 15 s interval: the admission interval 15.22 s against 9.62 s at the 10 s interval, and the time from the scaling decision to usable capacity 16.97 s against 11.45 s, both rising by about the added five seconds; descriptive, Cliff's $\delta = -0.444$), and the delay stayed inside the spare capacity of the existing nodes, the margin that the lost-message part of the campaign deliberately removes. No user-visible cost of the seconds-level delay appeared in any tested healthy regime. The confirmation check kept that pace. Its backends reached the first successful response in a median of 4.0 s against 4.1 s under backend notification, a near-tie, and close to six seconds before periodic polling (10.0 s). The comparison covers the fast-binding class only, because the intermittent bind stall of the recorded campaign did not occur in the completion runs (no backend bound slower than a tenth of a second), so the slow class carries no reading for this mode.

TABLE 5.10. Admission-chain intervals and healthy-regime outcomes (intact message; backend notification versus periodic polling). Medians over runs and per-backend intervals as marked; exact Mann–Whitney p-values as pre-registered.

Interval	Backend notification	Periodic polling	Test
Verified readiness→admission (s)	0.001	6.984	$\delta = -1.000$; earlier in every start-up class
Provisioning→first successful response (s)	11.27	17.70	exact $p = 0.0005$, $\delta = -0.594$
Scaling decision→usable capacity (s)	2.17	6.01	exact $p = 0.0022$, $\delta = -1.000$
Admission→first routed request (s)	0.4–1.2	0.4–1.2	confirmatory; not separated
Relief of the existing nodes (per-run median, pp; seven runs per mode)	16.0	19.3	exact $p = 0.209$; not separated

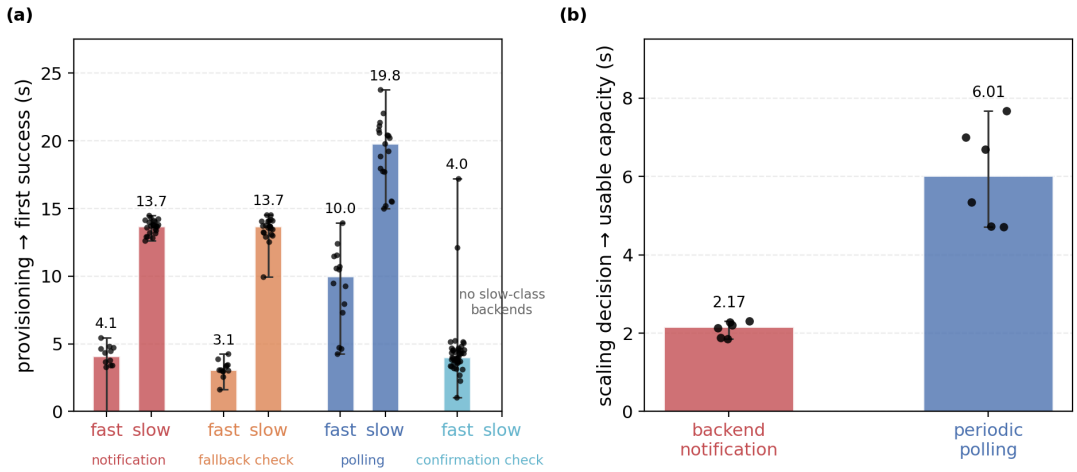


FIGURE 5.6. Admission-chain intervals under an intact message, all four admission modes. (a) Provisioning to first successful response per backend over the seven seed-matched blocks (fast-binding class medians 4.1, 3.1, 10.0 and 4.0 s; slow class 13.7, 13.7 and 19.8 s, with no confirmation-check backends, because the stall did not occur in the completion runs). (b) Scaling decision to usable capacity per run (the recorded higher-rate arm, 12.0 requests per second per client, six runs per mechanism; medians 2.17 against 6.01 s; exact $p = 0.0022$, $\delta = -1.000$). Start-up class is the backend's measured socket bind time, under a second on some backends and several seconds on others; the stall is an intermittent platform cost at spawn, equal across the modes, and held fixed by the within-class comparison. Black dots are individual backends (a) and runs (b); coloured bars and marks are medians; whiskers span the min–max range.

When the message is lost under reduced headroom, the expectation was that a mode without a re-derivation path would leave the new capacity dark at a decisive but bounded cost, while the modes that re-derive readiness would recover and relief would complete. The modes diverge in kind (Table 5.11 and Figure 5.7). The reduced headroom is deliberate: the fault comparison runs at a reduced per-backend CPU quota, so the existing nodes carry thin spare capacity and the cost of a lost admission is exposed rather than absorbed by slack. A mode that admits only on the message never learns the backend is ready, so backend notification admitted no backend in any loss run, every candidate abandoned and torn down (32–35 per block). The modes that re-derive readiness recover, because they observe it again through their own checks. Notification with a fallback check admitted after its fallback wait, about 21 s after provisioning, and periodic polling within one polling period, beginning about four seconds sooner (medians 17.1 s against 21.4 s). In the premature grid's loss runs (seeds 6301–6303), periodic polling also recovered through its check, landing $+1.3$ pp from its matched no-fault runs.

The consequence reached users. Under backend notification the slow share, the fraction of served requests that timed out or exceeded one second, rose to 8.3–14.4 % over the plateau, concentrated in the onset window, where the difference against the recovering modes had a median of 28.0 pp (full plateau 7.64 pp, exact $p = 0.0040$), and it stayed bounded at a 10.6 % plateau median, under the pre-registered ceiling of 25 % on the slow share. The existing nodes

kept serving the episode throughout, so what the failure removed was the spare capacity, not the service.

TABLE 5.11. Admission modes under a lost message (the RQ3 campaign; six blocks, plateau pooled; the seventh block's pair is report-only under the pre-registered control-tail rule). Admitted = visibility in the routing pool; abandoned candidates in parentheses. Plateau tail = slow share over the last 300 s of the plateau; *block 4 is excluded from the fault comparison, its no-fault control tail (4.08 %) breaching the 2 pp floor.

Mode	Re-derivation	Admitted (abandoned)	Plateau slow (%)	Plateau tail (%)
Backend notification	none; fallback inert	0 (32–35)	8.3–14.4	1.0–1.3, except b2: 11.4
Notification with a fallback check	20 s fallback check	4–6 (0)	1.9–3.6	≤1.5
Periodic polling	10 s poll	4–5 (0)	2.0–5.0	≤1.4, except b4*: 4.08
Notification with a confirmation check	20 s absence fallback	6–7 (0)	1.1–4.1	0.7–3.7

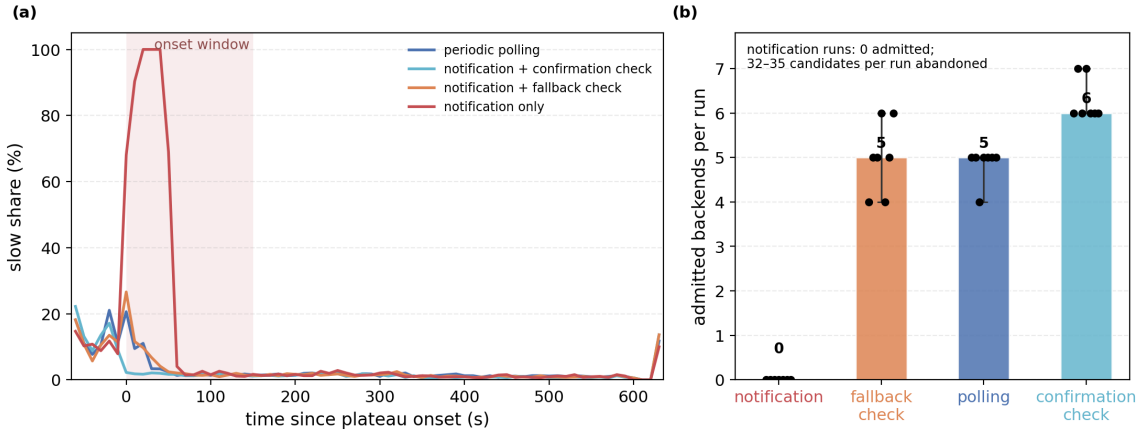


FIGURE 5.7. Under a lost message with reduced headroom, only the modes that can re-derive readiness recover. (a) Slow share (served requests timed out or slower than 1 s), median across the seven seed-matched blocks per 10 s bin; the shaded band marks the onset window (first 150 s after the plateau starts). (b) Admitted backends per run (dots, one per block; bars, medians; whiskers, the min–max range). In the seven backend-notification runs no backend was admitted and 32–35 candidates per run were abandoned and torn down, while the fallback check, periodic polling and the confirmation check admit through their own checks (medians 5, 5 and 6 backends) with relief completing; no-fault runs admit on every mode, with zero abandoned candidates.

The expectation held. Backend notification admitted no backend and paid a bounded but decisive cost to users, while the modes that re-derive readiness recovered and relief completed. The separation follows from what each mode can re-derive: a dropped message is unrecoverable for a mode that admits only on it, its fallback sitting beyond the abandonment bound, while the fallback check and the periodic poll observe readiness again through their own checks. One

qualifier bounds the reading: the pre-registered conjunction was not fully met, the shortfall confined to two marginal no-fault window breaches in the health-floor count that do not touch the comparison, with the block-4 exclusion resting on a separate pre-registered control-tail check recorded in Table 5.11. The confirmation check recovered through the same absence fallback, admitting 6–7 backends per run with none abandoned. Its paired slow share against its own fault-free partners stayed within the 2 pp bound in every one of the seven blocks (median change -0.4 pp), so the recovery carried no user cost.

Under a premature message at the evaluated lead of ten seconds, the expectation was that the modes that admit on the message would be exposed while the modes that check would absorb it (Table 5.12 and Figure 5.8). Damage is read in the same user-visible measure as above: the change in the share of requests that fail, time out, or exceed one second, against a matched no-fault run. Backend notification carried a median damage of $+14.4$ pp against its matched no-fault runs, positive in all six usable pairs, with one pair dropped for a breach of the no-fault health floors. Notification with a fallback check shared the exposure (median $+11.7$ pp, positive in six of seven pairs). Periodic polling stayed at its no-fault level (median -0.1 pp, within the 2 pp bound in all seven pairs). The same-fault contrasts confirm the separation, $+17.0$ pp and $+16.2$ pp over the seven blocks, positive in every block in both. The expectation held, and the justification is the evidence each mode trusts: the fallback check reacts to a message that never arrives, not to one that arrives early, so it shares the exposure, while the periodic poll's check cannot pass before the backend can serve, so the timing of the message has nothing to act on. The confirmation check was immune in the same way: a median damage of -0.9 pp over the seven seed-matched pairs, within the 2 pp bound in six of them (the exception a $+3.1$ pp pair). It admitted at true servability, reaching the first successful response 11.0 s after the message against 19.5 s under periodic polling. Backend notification was $+16.9$ pp more damaged than it, positive in every block, and its own damage against periodic polling is a near-tie (-0.2 pp). Two of its runs carried the small pre-bind transient recorded for the pilot (two early successes), reported per run; the cell stays assessable.

TABLE 5.12. User-visible damage of the admission modes under a premature message at the evaluated ten-second lead (seven seed-matched pairs per mode). Damage is the change, in percentage points, in the share of requests that fail, time out, or exceed one second, against a matched no-fault run; medians over the seven seed-matched pairs. A positive reading means the fault run's share was above that of its no-fault partner.

Mode	Median damage (pp)	Positive pairs	Within 2 pp
Backend notification	$+14.4$	6/6	0
Notification with a fallback check	$+11.7$	6/7	2
Periodic polling	-0.1	3/7	7/7
Notification with a confirmation check	-0.9	1/7	6/7

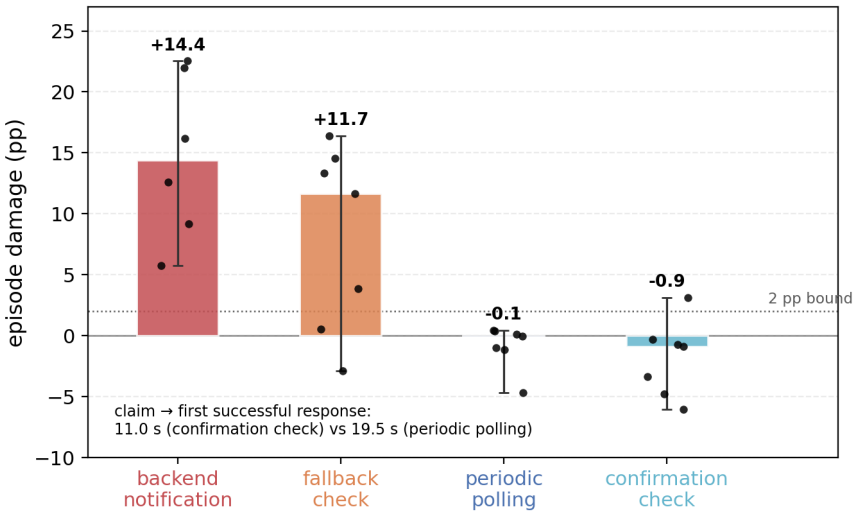


FIGURE 5.8. User-visible damage of the four admission modes under a premature message at the evaluated ten-second lead. Bars are the medians over the usable pairs, dots the individual seed-matched pairs, and the dotted line the pre-registered 2 pp bound; a positive reading means the fault run's share of slow requests was above that of its no-fault partner. The confirmation check admits at true servability, which also places it first to a first successful response after the message (11.0 s against 19.5 s under periodic polling).

Across the pre-registered criteria, the set closes as follows: the intact direction, the one-period band, and the post-admission invariance held; the lost separation and boundedness held, with the full conjunction missed on two marginal no-fault breaches; and the premature criterion held in both directions, the modes that admit on the message exposed and the modes that check holding their no-fault level. For the confirmation check, the intact direction and the lost no-harm reading held in the classes with evidence, and the premature criterion held outright, its admission landing at true servability.

In sum, while the message was intact, the admission interval was bounded by the polling period and passed without user-visible cost. When the message was lost under reduced headroom, backend notification paid a bounded service-quality cost while the modes that re-derive readiness recovered. And when the message arrived early, the modes separated by what their evidence can see. A premature message cost the modes that admit on it while the modes that check priced it at nothing. Readiness-admission coordination is therefore at once a latency choice (about seven seconds of added delay while everything works) and a reliability choice, deciding whether a ready backend's capacity is delayed or lost when the message is lost, and whether a premature message reaches traffic before the backend can serve. Because admission fills the routing pool, the same choice sets what the load distribution can rely on when load has triggered the scaling. A lost message hides ready capacity from the pool. A premature message exposes capacity that cannot yet absorb traffic. The reported effects hold for the evaluated two-domain testbed, its workloads, and its resource limits.

5.5. Discussion: Synthesis Across Interfaces

This section reads the three campaigns together. The campaigns ran on the same platform, used the same metric definitions and instrumentation, and followed the same discipline of varying one interface while holding the others fixed. Each campaign, however, drove its own workload under its own demand regime, so the three sets of results are not pooled and no quantity is combined across them, consistent with the commitment in Section 2.7 that no aggregate coordination cost is claimed. What can be compared is the pattern each campaign revealed: which effects held for every replicate, which depended on a condition the campaign varied or sampled, and which the campaign could not establish. This classification is the comparative synthesis promised in Section 1.5, stated per interface in Table 5.13, and it prepares the answers summarised in Chapter 6.

TABLE 5.13. Synthesis of the three campaigns, classified by whether a finding held for every replicate entering the contrast (robust), depended on a condition the campaign varied or sampled (conditional), or could not be confirmed (not established). Every entry remains within its own campaign, and no quantity is combined across campaigns.

Interface	Robust	Conditional	Not established
Telemetry Modes (RQ1)	Loss of intermediate evidence. New capacity became usable nearly three times later than under push, and the episode p95 penalty was about ninefold, with complete separation in every replicate	Delay: costly in proportion to the absorption margin it consumes, with two of seven delayed push runs collapsing to the level of polling and sampled push	The full ordering A $<$ B $<$ C $\approx$ D, and polling versus sampling not separated (C $\approx$ D)
Bottleneck-Aware Scaling Action (RQ2)	Storage action under data-access-bound demand. Tier relief in every storage-only domain (12/12) and in most bottleneck-aware domains (8/10), with user-visible benefit over inaction (labelled comparison); the classifier committed to the detected tier in every evaluated regime, with classifier-versus-episode agreement of 88–93 % on the data-access-bound episode	A capacity-bound queue in the served tier as the condition for benefit. Where no queue formed, full compute execution recovered nothing and the provisioning left a late-episode tail	Compute-side recovery attributable to the action, no demonstrated advantage of bottleneck-aware selection over a fixed matched policy under the tested conditions, the pre-registered harm expectation not met (the measured ordering is the matched action, then the mismatched action, then the bottleneck-aware action, then no action), and the pre-registered storage tail-latency test, reported unmet
Readiness Admission Modes under Three Message Conditions (RQ3)	Loss without re-derivation leaves capacity dark (zero admissions in every loss run), and a premature message damages only the modes that admit on it (damage medians up to +14.4 pp) while the modes that check stay flat	The loss cost under the reduced headroom the campaign fixed, and premature-message damage for the modes that admit on the message while the modes that check stay flat at the evaluated ten-second lead	The lost-message reading's full pre-registered conjunction (two marginal no-fault window breaches)

Read across the campaigns, the measured effects share a shape. They appear at the edge of the operating envelope, where a margin is consumed, and not in the steady state. The telemetry modes were indistinguishable outside the load rise, and the cost of delay appeared only when the absorption margin of the existing nodes ran out. Missing summaries stretched the window in which the existing nodes had to cope alone, and the margin was exhausted in every replicate. The action improved user-visible quality only where the episode built a capacity-bound queue in the served tier. Where no queue formed, the compute action recovered nothing and the

provisioning itself left a tail. The admission delay cost nothing while the nodes held spare capacity, and the lost message consumed that margin under the reduced headroom the campaign fixed. On the evaluated testbed, the three interfaces decide how a demand shift is weathered, not how the service performs once the shift has passed.

The campaigns also separate failure modes that monitoring and scaling are often discussed as one. Across the telemetry modes, missing summaries and late summaries are different faults. Missing summaries cost about nine times the episode tail, with complete separation in every replicate, while late summaries cost in proportion to the margin they consume. Delay was harmless in five delayed replicates and decisive in the two that sampled a slow platform state. In the action interface, execution and benefit are separate. The compute action spread the load, yet users saw no recovery because no capacity-bound queue existed to drain, and the storage action delivered benefit precisely where the queue existed. The action also carried a cost of its own: on the compute-bound episode the provisioning left a late-episode tail in both compute-scaling arms, and on the data-access-bound episode the bottleneck-aware arm's additional compute action coincided with a tail spike in two of its five replicates, so provisioning is not invisible to users while an episode is live. In the admission interface, latency and reliability are separate. Backend notification saved about seven seconds while the platform was healthy, and lost the admission entirely when its single message was dropped, while the modes that re-derive readiness turned the same loss into a bounded delay, and a premature message cost only the modes that admitted on it. Because admission fills the routing pool, the choice also sets what the load distribution can rely on once scaling has been triggered.

The interfaces also differ in the control records they leave behind, but not in the controller cost that carries them. Controller CPU stayed near 14 % in the telemetry episodes, near 11 % across the admission plateau, and within 2–22 % across the action episodes depending on the regime (the storage-only policy at the low end, the compute-adding policies at the high end), with memory near 70 MB everywhere. The records differ as designed: about 335 deliveries per run under push and delayed push against about 105–114 under polling and sampled push, check records in place of messages under periodic polling, and eight or six compute scale-ups per run where the compute policies acted (configuration medians in Table A.4).

Chapter 2 left the three questions of Table 2.3 open, and the campaigns are their experimental form. Read back against those questions, the results answer them within the reviewed scope. On the telemetry modes, the reviewed works do not trace a single mode through a stateful service's scaling path, and the closest precedent varies freshness for a single consumer. The campaign distinguishes the two ways evidence can be incomplete: missing intermediate summaries drive the user-visible cost in every replicate, while a complete but delayed path costs in proportion to the absorption margin it consumes. On the action, the reviewed auto-scaling works fix the action type or evaluate coordination in composite. The campaign shows that user-visible value follows the tier that binds the episode, and that classification had no demonstrated advantage over a fixed policy matched to the regime in the operating envelope, where no compute-bound episode became capacity-bound and no policy had a service deficit to recover,

which bounds the expected value of choosing the action at runtime. On admission, the reviewed placement work names readiness but leaves the step to traffic outside its scope. The campaign measures that step as a bounded delay while the platform is healthy, and as an all-or-nothing loss under a dropped message without re-derivation, which the modes that re-derive readiness convert into a bounded delay. These statements characterise, within the reviewed literature, the interfaces that the reviewed works leave open. They do not assert that the coordination gap is universal.

Three considerations follow for the design of services like the one evaluated, all bounded to the evaluated testbed. First, when telemetry must survive a load rise, completeness is the property to protect: dropping intermediate summaries drove the user-visible cost in every replicate, while a complete but delayed path degraded only where the absorption margin was thin. Second, a scaling action should be conditional on the tier that actually binds the episode: full execution of an action on a tier that was not the bottleneck spent node-minutes without recovering the service, and the act of provisioning itself left a tail while an episode was live. Third, readiness admission is a reliability path as much as a latency path: a bounded re-derivation cost about seven seconds while the platform was healthy, and preserved the provisioned capacity when the message was lost. On the evaluation side, the campaigns show that these interfaces are exercised by the transient and the margin.

These statements inherit the scope and qualifiers of the campaigns that produced them: a two-domain testbed on a single host with an emulated inter-domain link, a stateful edge service over MongoDB replica sets, and the evaluated workload families and resource limits. Cross-domain storage placement remained held-constant platform context throughout, and its characterisation is deferred to future work. The limitations of the evaluation are detailed in Chapter 6.

CHAPTER 6

Conclusions and Future Work

6.1. Conclusions

6.2. Research Contributions

6.3. Limitations

6.4. Future Research Directions

References

[1] K. L. Bogdanov, W. Reda, G. Q. Maguire Jr., D. Kostić, and M. Canini, "Fast and accurate load balancing for geo-distributed storage systems," in *Proceedings of the ACM Symposium on Cloud Computing*, Carlsbad CA USA, Oct. 2018, pp. 386–400. DOI: 10.1145/3267809.3267820 [Online]. Available: https://doi.org/10.1145/3267809.3267820

[2] V. Podolskiy, A. Jindal, and M. Gerndt, "Iaas reactive autoscaling performance challenges," in *2018 IEEE 11th International Conference on Cloud Computing (CLOUD)*, San Francisco, CA, USA, Jul. 2018, pp. 954–957. DOI: 10.1109/CLOUD.2018.00144 [Online]. Available: https://doi.org/10.1109/CLOUD.2018.00144

[3] International Telecommunication Union, "Facts and figures 2025: Internet traffic," ITU, Oct. 2025. Accessed: Jul. 13, 2026. [Online]. Available: https://www.itu.int/itu-d/reports/statistics/2025/10/15/ff25-internet-traffic/

[4] D. Gurung, S. M. Z. U. Rashid, Z. u. Abdeen, and S. Rath, "Cloud revolution: Tracing the origins and rise of cloud computing," in *2026 IEEE 16th Annual Computing and Communication Workshop and Conference (CCWC)*, Las Vegas, NV, USA, Jan. 2026, pp. 1100–1106. DOI: 10.1109/CCWC67433.2026.11393790 [Online]. Available: https://doi.org/10.1109/CCWC67433.2026.11393790

[5] T. Gaudiaut, *Mobile accounts for nearly 60 percent of web traffic*, Statista Chart of the Day, Data from StatCounter. Licensed under CC BY-ND 3.0, Mar. 2026. Accessed: Jul. 19, 2026. [Online]. Available: https://www.statista.com/chart/35951/share-of-mobile-device-website-traffic-worldwide-since-2010/

[6] K. Cao, Y. Liu, G. Meng, and Q. Sun, "An overview on edge computing research," *IEEE Access*, vol. 8, pp. 85 714–85 728, 2020. DOI: 10.1109/ACCESS.2020.2991734 [Online]. Available: https://doi.org/10.1109/ACCESS.2020.2991734

[7] M. Satyanarayanan, "The emergence of edge computing," *Computer*, vol. 50, no. 1, pp. 30–39, Jan. 2017. DOI: 10.1109/MC.2017.9 [Online]. Available: https://doi.org/10.1109/MC.2017.9

[8] Q. Luo, S. Hu, C. Li, G. Li, and W. Shi, "Resource scheduling in edge computing: A survey," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 4, pp. 2131–2165, 2021. DOI: 10.1109/COMST.2021.3106401 [Online]. Available: https://doi.org/10.1109/COMST.2021.3106401

[9] M. Breitbach, D. Schäfer, J. Edinger, and C. Becker, "Context-aware data and task placement in edge computing environments," in *2019 IEEE International Conference on Pervasive Computing and Communications (PerCom)*, Kyoto, Japan, Mar. 2019, pp. 1–10. DOI: 10.1109/PERCOM.2019.8767386 [Online]. Available: https://doi.org/10.1109/PERCOM.2019.8767386

[10] C. Qu, R. N. Calheiros, and R. Buyya, "Auto-scaling web applications in clouds: A taxonomy and survey," *ACM Computing Surveys*, vol. 51, no. 4, pp. 1–33, Jul. 2018. DOI: 10.1145/3148149 [Online]. Available: https://doi.org/10.1145/3148149

[11] A. Llorens-Carrodeguas, I. Leyva-Pupo, C. Cervelló-Pastor, L. Piñeiro, and S. Siddiqui, "An sdn-based solution for horizontal auto-scaling and load balancing of transparent vnf clusters," *Sensors*, vol. 21, no. 24, p. 8283, Dec. 2021. DOI: 10.3390/s21248283 [Online]. Available: https://doi.org/10.3390/s21248283

[12] N. Yaseen, "From counters to telemetry: A survey of programmable network-wide monitoring," *Network*, vol. 5, no. 3, p. 38, Sep. 2025. DOI: 10.3390/network5030038 [Online]. Available: https://doi.org/10.3390/network5030038

[13] A. Y. Nikravesh, S. A. Ajila, and C.-H. Lung, "The impact of database layer on auto-scaling decisions in a 3-tier web services cloud resource provisioning," in *2017 IEEE 41st Annual Computer Software and Applications Conference (COMPSAC)*, Turin, Jul. 2017, pp. 401–406. DOI: 10.1109/COMPSAC.2017.235 [Online]. Available: https://doi.org/10.1109/COMPSAC.2017.235

[14] T. Dreibholz and S. Mazumdar, "Towards a lightweight task scheduling framework for cloud and edge platform," *Internet of Things*, vol. 21, p. 100 651, Apr. 2023. DOI: 10.1016/j.iot.2022.100651 [Online]. Available: https://doi.org/10.1016/j.iot.2022.100651

[15] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–106, Mar. 2004. DOI: 10.2307/25148625 [Online]. Available: https://doi.org/10.2307/25148625

[16] J. vom Brocke, A. Hevner, and A. Maedche, "Introduction to design science research," in *Progress in IS*, 2020, pp. 1–13, ISBN: 9783030467807. DOI: 10.1007/978-3-030-46781-4_1 [Online]. Available: https://doi.org/10.1007/978-3-030-46781-4_1

[17] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, Dec. 2007. DOI: 10.2753/MIS0742-1222240302 [Online]. Available: https://doi.org/10.2753/MIS0742-1222240302

[18] F. Liu, G. Tang, Y. Li, Z. Cai, X. Zhang, and T. Zhou, "A survey on edge computing systems and tools," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1537–1562, 2019. DOI: 10.1109/JPROC.2019.2920341 [Online]. Available: https://doi.org/10.1109/JPROC.2019.2920341

[19] A. Nain, S. Sheikh, M. Shahid, and R. Malik, "Resource optimization in edge and sdn-based edge computing: A comprehensive study," *Cluster Computing*, vol. 27, no. 5, pp. 5517–5545, Feb. 2024. DOI: 10.1007/s10586-023-04256-8 [Online]. Available: https://doi.org/10.1007/s10586-023-04256-8

[20] A. C. Baktir, A. Ozgovde, and C. Ersoy, "How can edge computing benefit from software-defined networking: A survey, use cases, and future directions," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 4, pp. 2359–2391, 2017. DOI: 10.1109/COMST.2017.2717482 [Online]. Available: https://doi.org/10.1109/COMST.2017.2717482

[21] Q. Waseem, W. Isni Sofiah Wan Din, A. Aminuddin, M. Hussain Mohammed, and R. F. Alfa Aziza, "Software-defined networking (sdn): A review," in *2022 5th International Conference on Information and Communications Technology (ICOIACT)*, Yogyakarta, Indonesia, Aug. 2022, pp. 30–35. DOI: 10.1109/ICOIACT55506.2022.9972067 [Online]. Available: https://doi.org/10.1109/ICOIACT55506.2022.9972067

[22] A. Liatifis, P. Sarigiannidis, V. Argyriou, and T. Lagkas, "Advancing sdn from openflow to p4: A survey," *ACM Computing Surveys*, vol. 55, no. 9, pp. 1–37, Jan. 2023. DOI: 10.1145/3556973 [Online]. Available: https://doi.org/10.1145/3556973

[23] MongoDB, Inc., *Replication*, MongoDB Manual, 2026. Accessed: Sep. 13, 2026. [Online]. Available: https://www.mongodb.com/docs/manual/replication/

[24] C.-K. Huang and G. Pierre, "Adappf: Self-adaptive scrape interval for monitoring in geo-distributed cluster federations," in *2023 IEEE Symposium on Computers and Communications (ISCC)*, Gammarth, Tunisia, Jul. 2023, pp. 417–423. DOI: 10.1109/ISCC58397.2023.10218080 [Online]. Available: https://doi.org/10.1109/ISCC58397.2023.10218080

[25] M.-H. Hung, C.-C. Teng, C.-P. Chuang, C.-S. Hsu, J.-W. Gong, and M.-C. Chen, "A sdn controller monitoring architecture for 5g backhaul networks," in *2022 23rd Asia-Pacific Network Operations and Management Symposium (APNOMS)*, Takamatsu, Japan, Sep. 2022, pp. 1–4. DOI: 10.23919/APNOMS56106.2022.9919988 [Online]. Available: https://doi.org/10.23919/APNOMS56106.2022.9919988

[26] M. Usman, S. Ferlin, A. Brunstrom, and J. Taheri, "A survey on observability of distributed edge & container-based microservices," *IEEE Access*, vol. 10, pp. 86 904–86 919, 2022. DOI: 10.1109/ACCESS.2022.3193102 [Online]. Available: https://doi.org/10.1109/ACCESS.2022.3193102

[27] H. Zhou and C. H. Yong, "Implement hpa for nginx service using custom metrics under kubernetes framework," *IEEE Access*, vol. 12, pp. 189 722–189 734, 2024. DOI: 10.1109/ACCESS.2024.3509876 [Online]. Available: https://doi.org/10.1109/ACCESS.2024.3509876

[28] M. I. S. El Mrabet, A. Malki, and M. Malki, "A comprehensive survey and taxonomy of autoscaling techniques in cloud and edge environments," in *2025 Fourth International Conference on Theoretical and Applicative Aspects of Computer Science (ICTAACS)*, Skikda, Algeria, Dec. 2025, pp. 1–8. DOI: 10.1109/ICTAACS69003.2025.11399335 [Online]. Available: https://doi.org/10.1109/ICTAACS69003.2025.11399335

[29] S. Taherizadeh and V. Stankovski, "Auto-scaling applications in edge computing," in *Proceedings of the International Conference on Big Data and Internet of Thing*, London United Kingdom, Dec. 2017, pp. 158–163. DOI: 10.1145/3175684.3175709 [Online]. Available: https://doi.org/10.1145/3175684.3175709

[30] L. Toka, G. Dobreff, B. Fodor, and B. Sonkoly, "Machine learning-based scaling management for kubernetes edge clusters," *IEEE Transactions on Network and Service Management*, vol. 18, no. 1, pp. 958–972, Mar. 2021. DOI: 10.1109/TNSM.2021.3052837 [Online]. Available: https://doi.org/10.1109/TNSM.2021.3052837

[31] L. Ju, P. Singh, and S. Toor, "Proactive autoscaling for edge computing systems with kubernetes," in *Proceedings of the 14th IEEE/ACM International Conference on Utility and Cloud Computing Companion*, Leicester United Kingdom, Dec. 2021, pp. 1–8. DOI: 10.1145/3492323.3495588 [Online]. Available: https://doi.org/10.1145/3492323.3495588

[32] K. Cheng, S. Zhang, C. Tu, X. Shi, Z. Yin, S. Lu, Y. Liang, and Q. Gu, "Proscale: Proactive autoscaling for microservice with time-varying workload at the edge," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 4, pp. 1294–1312, Apr. 2023. DOI: 10.1109/TPDS.2023.3238429 [Online]. Available: https://doi.org/10.1109/TPDS.2023.3238429

[33] Y. Li, H. Zhang, W. Tian, and H. Ma, "Joint optimization of auto-scaling and adaptive service placement in edge computing," in *2021 IEEE 27th International Conference on Parallel and Distributed Systems (ICPADS)*, Beijing, China, Dec. 2021, pp. 923–930. DOI: 10.1109/ICPADS53394.2021.00121 [Online]. Available: https://doi.org/10.1109/ICPADS53394.2021.00121

[34] J. Dogani, R. Namvar, and F. Khunjush, "Auto-scaling techniques in container-based cloud and edge/fog computing: Taxonomy and survey," *Computer Communications*, vol. 209, pp. 120–150, Sep. 2023. DOI: 10.1016/j.comcom.2023.06.010 [Online]. Available: https://doi.org/10.1016/j.comcom.2023.06.010

[35] S. Xie, J. Wang, B. Li, Z. Zhang, D. Li, and P. C. K. Hung, "Pbscaler: A bottleneck-aware autoscaling framework for microservice-based applications," *IEEE Transactions on Services Computing*, vol. 17, no. 2, pp. 604–616, Mar. 2024. DOI: 10.1109/TSC.2024.3376202 [Online]. Available: https://doi.org/10.1109/TSC.2024.3376202

[36] C. Li, J. Bai, Y. Ge, and Y. Luo, "Heterogeneity-aware elastic provisioning in cloud-assisted edge computing systems," *Future Generation Computer Systems*, vol. 112, pp. 1106–1121, Nov. 2020. DOI: 10.1016/j.future.2020.06.022 [Online]. Available: https://doi.org/10.1016/j.future.2020.06.022

[37] M. R. Belgaum, S. Musa, M. M. Alam, and M. M. Su'ud, "A systematic review of load balancing techniques in software-defined networking," *IEEE Access*, vol. 8, pp. 98 612–98 636, 2020. DOI: 10.1109/ACCESS.2020.2995849 [Online]. Available: https://doi.org/10.1109/ACCESS.2020.2995849

[38] A. A. Neghabi, N. Jafari Navimipour, M. Hosseinzadeh, and A. Rezaee, "Load balancing mechanisms in the software defined networks: A systematic and comprehensive review of the literature," *IEEE Access*, vol. 6, pp. 14 159–14 178, 2018. DOI: 10.1109/ACCESS.2018.2805842 [Online]. Available: https://doi.org/10.1109/ACCESS.2018.2805842

[39] A. A. Abdelltif, E. Ahmed, A. T. Fong, A. Gani, and M. Imran, "Sdn-based load balancing service for cloud servers," *IEEE Communications Magazine*, vol. 56, no. 8, pp. 106–111, Aug. 2018. DOI: 10.1109/MCOM.2018.1701016 [Online]. Available: https://doi.org/10.1109/MCOM.2018.1701016

[40] A. Wang, Z. Zha, Y. Guo, and S. Chen, "Software-defined networking enhanced edge computing: A network-centric survey," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1500–1519, Aug. 2019. DOI: 10.1109/JPROC.2019.2924377 [Online]. Available: https://doi.org/10.1109/JPROC.2019.2924377

[41] R. C. Sofia, D. Dykeman, P. Urbanetz, A. Galal, and D. A. Dave, "Dynamic, context-aware cross-layer orchestration of containerized applications," *IEEE Access*, vol. 11, pp. 93 129–93 150, 2023. DOI: 10.1109/ACCESS.2023.3307026 [Online]. Available: https://doi.org/10.1109/ACCESS.2023.3307026

[42] H. Peng, Q. Ye, and X. S. Shen, "Sdn-based resource management for autonomous vehicular networks: A multi-access edge computing approach," *IEEE Wireless Communications*, vol. 26, no. 4, pp. 156–162, Aug. 2019. DOI: 10.1109/MWC.2019.1800371 [Online]. Available: https://doi.org/10.1109/MWC.2019.1800371

[43] E. Saurez, H. Gupta, A. Daglis, and U. Ramachandran, "Oneedge: An efficient control plane for geo-distributed infrastructures," in *Proceedings of the ACM Symposium on Cloud Computing*, Seattle WA USA, Nov. 2021, pp. 182–196. DOI: 10.1145/3472883.3487008 [Online]. Available: https://doi.org/10.1145/3472883.3487008

[44] M. A. Tamiru, G. Pierre, J. Tordsson, and E. Elmroth, "Mck8s: An orchestration platform for geo-distributed multi-cluster environments," in *2021 International Conference on Computer Communications and Networks (ICCCN)*, Athens, Greece, Jul. 2021, pp. 1–10. DOI: 10.1109/ICCCN52240.2021.9522318 [Online]. Available: https://doi.org/10.1109/ICCCN52240.2021.9522318

[45] H. Tabatabaee Malazi, S. R. Chaudhry, A. Kazmi, A. Palade, C. Cabrera, G. White, and S. Clarke, "Dynamic service placement in multi-access edge computing: A systematic literature review," *IEEE Access*, vol. 10, pp. 32 639–32 688, 2022. DOI: 10.1109/ACCESS.2022.3160738 [Online]. Available: https://doi.org/10.1109/ACCESS.2022.3160738

[46] Oracle Corporation, *Mysql adminapi*, MySQL Shell User Guide, 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://dev.mysql.com/doc/mysql-shell/en/admin-api-userguide.html

[47] Oracle Corporation, *Innodb cluster*, MySQL 8.0 Reference Manual, 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/mysql-innodb-cluster-introduction.html

[48] PostgreSQL Global Development Group, *Log-shipping standby servers*, PostgreSQL Documentation, 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://www.postgresql.org/docs/current/warm-standby.html

APPENDIX A

Campaign Configuration Values

This appendix lists the concrete configuration values of the three campaigns reported in Chapter 5, complementing the design summaries given at the start of each results section. The values are transcribed from the campaign configuration records; every run additionally stored the effective configuration it executed (`controller_env_snapshot.env`, `aggregator_env_snapshot.env`) and its phase schedule (`phases_snapshot.json`), so the per-run values are recoverable from the run artefacts. Where a campaign spans several demand regimes or evidence sets, the arms state the value per regime or set.

TABLE A.1. Container resource limits, flow timeouts, and client bounds per campaign.

Parameter	Telemetry Modes (RQ1)	Bottleneck-Aware Scaling Action (RQ2)	Readiness Admission Modes under Three Message Conditions (RQ3)
Compute replica CPU quota (vCPU)	0.25	0.15 (compute-bound arms) / 1.20 (data-bound arms)	0.12 (fault arms); 0.15 / 0.30 (intact-source sets)
Compute replica memory limit	256 MB	256 MB → 512 MB (from run 15; platform hardening)	256 MB
Storage-member CPU quota (vCPU)	0.08	0.08 (compute-bound arms) / 0.15 (data-bound arms)	0.08
Storage-member memory limit	512 MB	512 MB	512 MB
Clients per domain	24	24	24
MongoDB connections per edge server	6	12	6
Dynamic-unit cap (compute / storage)	3 / 3	6 / 6	12 / not exercised
Inter-domain link RTT (emulated)	185 ms	185 ms	185 ms
Service-path flow timeouts (idle / hard)	30 s / 60 s	30 s / 60 s	30 s / 120 s
Client bounds (in-flight window / request cap / phase-end drain)	1024 / 300 s / 30 s	1024 / 300 s / 30 s	1024 / 300 s / 30 s

Note. The scaling action campaign's compute-replica memory limit was raised from 256 MB to 512 MB at run 15 (platform hardening after the two documented memory-cap incidents; the affected runs were re-run, and one compute-bound arm was re-run at the binding configuration). In the readiness admission campaign the fault arms ran at the reduced quota of 0.12 vCPU and the intact-source admission-timing sets at 0.15 and 0.30 vCPU; storage scaling is not exercised in that campaign.

TABLE A.2. Scaling-policy parameters per campaign.

Parameter	RQ1	RQ2	RQ3
Latency signal statistic	mean	median	mean
Compute score weights (utilisation / latency)	0.60 / 0.40	0.60 / 0.40	0.40 / 0.60
Compute signal floors (CPU % / latency ms)	10 / 25	10 / 25	4.5 / 20
Compute scale-up threshold (base / max)	0.18 / 0.85	0.18 / 0.85	0.25 / 0.85
Storage score weights (utilisation / latency)	0.00 / 1.00	0.60 / 0.40	0.60 / 0.40
Storage signal floors (CPU % / latency ms)	1.5 / 60	35 / 10	1.5 / 60
Storage scale-up threshold (base / max)	0.35 / 0.55	0.35 / 0.55	0.20 / 0.55
Scale-up cooldowns (compute / storage)	45 s / 120 s	45 s / 120 s	45 s / 120 s
Compute scale-down (cooldown / window / below-criterion)	180 s / 3 of 6 / 25 % · 40 ms	180 s / 3 of 6 / 25 % · 40 ms	180 s / 7 of 12 / 15 % · 20 ms
Storage scale-down (cooldown / window)	30 s / 3 of 5	30 s / 3 of 5	120 s / 9 of 15
Action budget per tier	—	4	—
Bottleneck classification margin	—	0.05	—

Note. The latency statistic is the per-summary statistic that feeds the decision signals; the telemetry campaign keeps the mean statistic by design, the scaling action campaign runs the median statistic, and the readiness admission campaign runs the platform base decision configuration. In the telemetry modes column the zero utilisation weight makes the storage signal latency-only; the action budget and the classification margin apply only to bottleneck-aware selection.

TABLE A.3. Load-balancing weights per campaign.

Parameter	RQ1	RQ2	RQ3
Backend weights (CPU / RAM / requests / hops)	0.30 / 0.10 / 0.20 / 0.28	0.30 / 0.10 / 0.20 / 0.28	0.20 / 0.20 / 0.20 / 0.28
Storage-member weights (CPU / RAM / connections / lag / hops)	0.20 / 0.10 / 0.20 / 0.20 / 0.30	0.20 / 0.10 / 0.20 / 0.20 / 0.30	0.20 / 0.20 / 0.10 / 0.20 / 0.30

Note. Weights of the weighted cost functions that select a backend, and a storage member, for a new flow (Section 3.2.2). The readiness admission campaign pins these weights; the other two campaigns use the platform base weights.

TABLE A.4. Control-plane cost per configuration (descriptive; medians across the runs of each configuration). Delivery records count delivered telemetry summaries; action records count provisioned units; admission records count events or readiness checks.

Campaign	Configuration	Control records per run	CPU	Memory
Telemetry	A push	340 deliveries; 6 compute scale-ups	14.3 %	69 MB
Telemetry	B delayed push	332 deliveries; 6 compute scale-ups	14.7 %	69 MB
Telemetry	C polling	105 deliveries; 4 compute scale-ups	14.5 %	72 MB
Telemetry	D sampled push	114 deliveries; 3 compute scale-ups	14.0 %	73 MB
Action	compute-only, compute-bound	8 compute scale-ups	2.7 %	72 MB
Action	storage-only, compute-bound	none	2.1 %	66 MB
Action	bottleneck-aware, compute-bound	8 compute scale-ups	2.7 %	67 MB
Action	compute-only, data-access-bound	6 compute scale-ups	22.0 %	69 MB
Action	storage-only, data-access-bound	reserve activations	15.6 %	69 MB
Action	bottleneck-aware, data-access-bound	6 compute scale-ups and reserve activations	22.3 %	70 MB
Action	no action (comparator)	none	2.2 / 17.0 %	66 / 69 MB
Admission	backend notification	4 events	10.9 %	67 MB
Admission	periodic polling, 10 s	4 check records	11.5 %	66 MB
Admission	periodic polling, 15 s	4 check records	11.0 %	66 MB

Note. Controller CPU is the mean over the episode phase for delivery and action, and over the plateau for admission; memory is the corresponding mean. Every action configuration records four storage adds for the run-start reserve preparation, excluded from the storage rows above; the no-action comparator records 2.2 % on the compute-bound episode and 17.0 % on the data-access-bound one. Counts are medians across that configuration's runs.

TABLE A.5. Implementation knobs of the interface experiments.

Knob	Meaning
READINESS_PROPAGATION	Compute-admission mode. `off` registers a replica immediately, `direct` admits on the `app_ready` message, and `discovery` checks on the polling interval.
READINESS_EVENT_FALLBACK_S	Direct mode. The time the gate waits for the message before checking anyway. 20 s in the reported admission runs and 130 s in the backend-notification configuration, where the check cannot begin before the abandonment bound.
DISCOVERY_POLL_INTERVAL_S	Polling interval. 10 s in the primary arms and 15 s in the sensitivity arm.
READINESS_WAKE_VERIFY_MODE	`probe` enables the confirmation check on the direct base. The message wakes a check, and admission requires the check to pass.
READINESS_PROBE_TIMEOUT_S, _RETRY_S, _MAX_S	Check timeout (5 s), retry interval (1 s), and abandonment bound (120 s).
EDGE_READY_PREMATURE_S	Fault instrumentation. The replica reports readiness this many seconds before it can serve, ten seconds in the evaluated premature runs.
EDGE_READY_SEMANTIC_LIE_S	Fault instrumentation. Workload requests fail while the readiness predicate stays satisfied, for this window counted from bind (1200 s).
READINESS_EVENT_DROP_MODE	Fault instrumentation. The readiness message is dropped entirely.
TELEMETRY_SOURCE	Telemetry mode source. `event_preserving` implements push, `delayed_event_preserving` implements delayed push (`DELAY_S`, 30 s), `poll` implements polling (`POLL_INTERVAL_S`, 30 s in the campaign), and `sampled_push` implements sampled push (`SAMPLE_EVERY`, every third summary); `zmq` is the platform's raw push default.
VIP_FLOW_ISOLATION, EDGE_FLOW_ISOLATION	One fresh routing decision per measured request, with per-connection service-path flows.